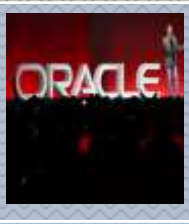
2020



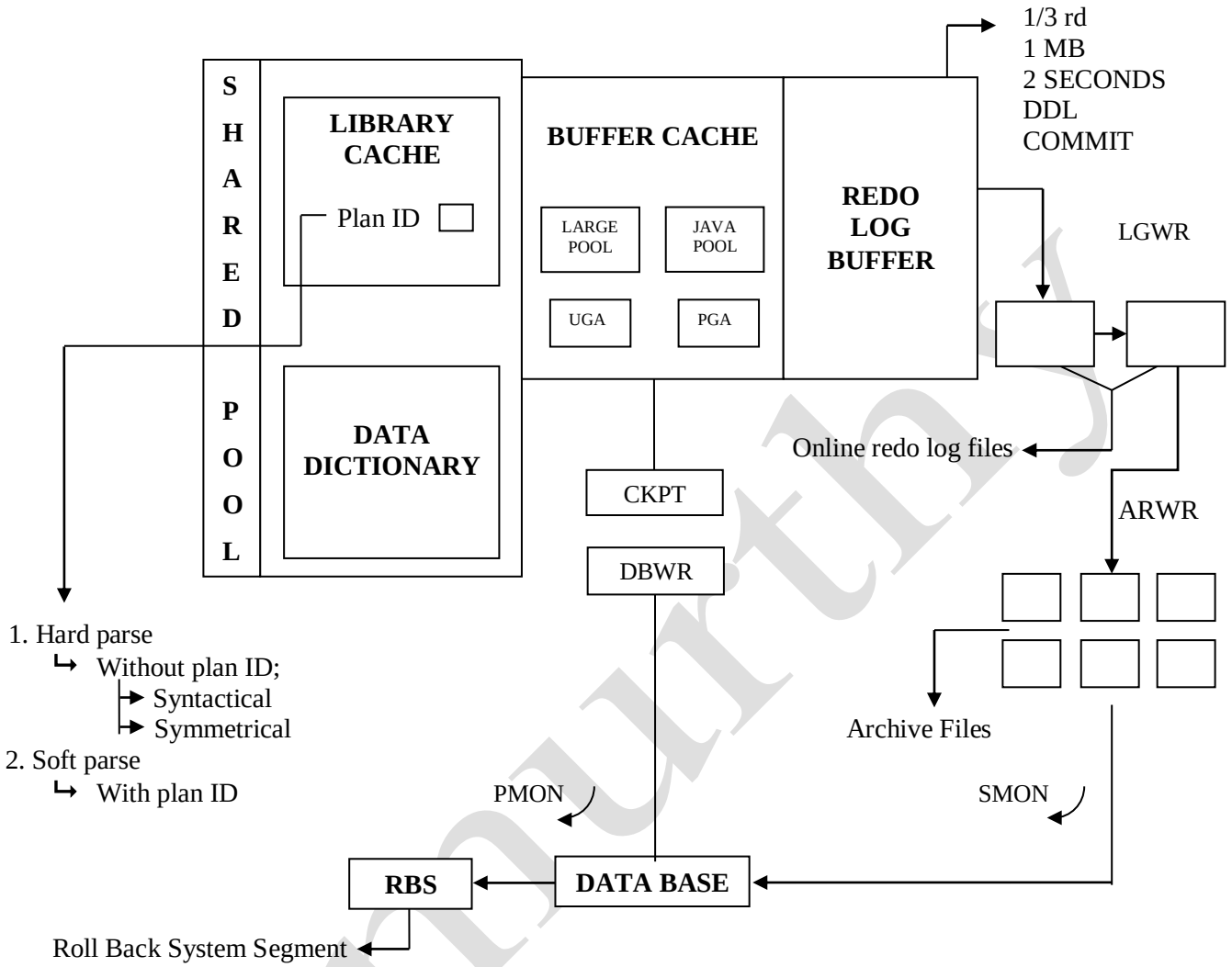
ORACLE

Dedicating to M.Rajasekhar Sir

Written by RAMANA MURTHY ATTADA
talk2armurthy@gmail.com

CONTENTS

ORACLE ARCHITECTURE



EXPLANATION:

When we installed the oracle it occupies some space from hard disk and some space from RAM.

The space occupied from RAM (750MB) - SYSTEM GLOBAL AREAR (SGA).

The space occupied from hard disk - ORACLE DATA BASE.

That System Global Area will be divided into three parts

SGA = SHARED POOL+ BUFFER CACHE+ READ LOG BUFFER

SHARED POOL again divides into two parts

SHARED POOL = LIBRARY CACHE + DATA DICTIONARY

LIBRARY CACHE:

If we write any insert, update, delete, select or any other statement whenever we execute that statement that data will go and store in LIBRARY CACHE in the RAM.

If we run any query for the 1st time two checking's will be applied for that query in LIBRARY CACHE.

1. Syntactical - it will check the syntax of the query
2. Symmetrical - it will check whether the tables, columns are existing or not.
   - ↳ These two checking's are done in hard parse only.
   - ↳ Again if we run same query it does not goes for these two checking's because when we are executing the query for 1st time system will generate some plan id for that query. 2nd time if we run same query it will go for soft parse by using that plan id.
   - ↳ Whether the query will go for the hard parse/soft parse will be decided by the plan id.

NOTE: If we write any query system will plan how to execute this query that process is called optimization.

DATA DICTIONARY:

Objects list exists in this data dictionary, Data dictionary nothing but `SELECT * FROM ALL_OBJECTS` etc..,

BUFFER CACHE:

Recent data will be exists in buffer cache (insert, update), all **:**NEW records will be present in this buffer cache only.

- **UGA:** When we are connecting to the schema and we executing any select statement or update statement etc., for that we need some memory that memory is **User Global Area (UGA).** When we are calling any packaged program that entire package will store in UGA in the RAM. If we are joining any tables and that join is nested loop join then those leading, probing table's data will be store in this UGA.

- **PGA:** When we executing bigger programs those are done in **Program Global Area (PGA)**. Mostly all the hash join operations are done in the PGA because we are joining the tables which are having huge data. Sometimes UGA will be exists in the PGA when PGA need memory.

- **LARGER POOL:** Which operations need larger memory those are done in larger pool.

- **JAVA POOL:** When we call any java related programs from the data base those are store in this JAVA POOL.

REDO LOG BUFFER:

Whatever the data in buffer cache will be go to redo log buffer but that data doesn't present more time in read log buffer. That data will be written in online redo log files when 1/3rd of redo log buffer is filled (or) 2seconds completed (or) 1MB of redo log buffer is crossed (or) any DDL operation is done (or) commit applied on read log buffer. In these five conditions if anyone is satisfied LGWR will move the data from redo log buffer and we don't know which condition will be satisfied 1st because it is depends on redo log buffer size.

LGWR is log writer it moves the data from redo log buffer to online redo log files.
AKWR is archive writer it moves the data from online redo log files to archive files.
Both are used for backup and recovery which records are deleted and committed.

- **DBWR:** It is a background processor and it will move the data from buffer cache to database when you give commit in buffer cache.
- **CKPT:** It is check point processor and it says to DBWR 'that buffer cache is filled and committed take that data and store in DB (only committed data)'.

  These DBWR, CKPT, LGWR, ARWR are called background process.
- **RBS:** If we update any data :NEW records can be exists in buffer cache and :OLD data records will be exists Roll Back System segment (RBS). If we delete any data that will be exists in this RBS after deleting if we give rollback, system will bring the data from RBS and stores where we have deleted.
- **SMON:** System Monitoring. Which extends are empty, which blocks are empty and in which block data should be inserted and what is pctused all these things handled by this SMON. And rare cases it will move the data from archive files to database when the data is committed in buffer cache but DBWR won't move the data to database then SMON will move that data from archive files to database and we already know that how the data will be moved from buffer cache to archive files by LGWR and AKWR processors.
- **PMON:** Program Monitoring. It will check all that background processors are working are not. If any background processor is not working properly then it will immediately shutdown the database. Means if SQL engine or PL/SQL engine or LGWR etc.., anyone is not working properly then it will shutdown the database.

We have number of background processor in oracle, to see all those background processors we should go for '`SELECT * FROM V$BGPROCESS`'.

ORACLE INSTANCE:

Oracle Instance = Background Processor + Temporary Memory + Permanent Memory

- Background processor (DBWR, CKPT, LGWR, ARWR etc..,).
- Temporary Memory (SGA).
- Permanent Memory (DATABASE).

TYPES OF FILES IN ORACLE:

- **Parameter File:** System configuration is exists in this parameter file. Means it contains the information about (SGA size, block size, time format, buffer size, shared pool size).

  ↳ `SELECT * FROM V$PARAMETER;`

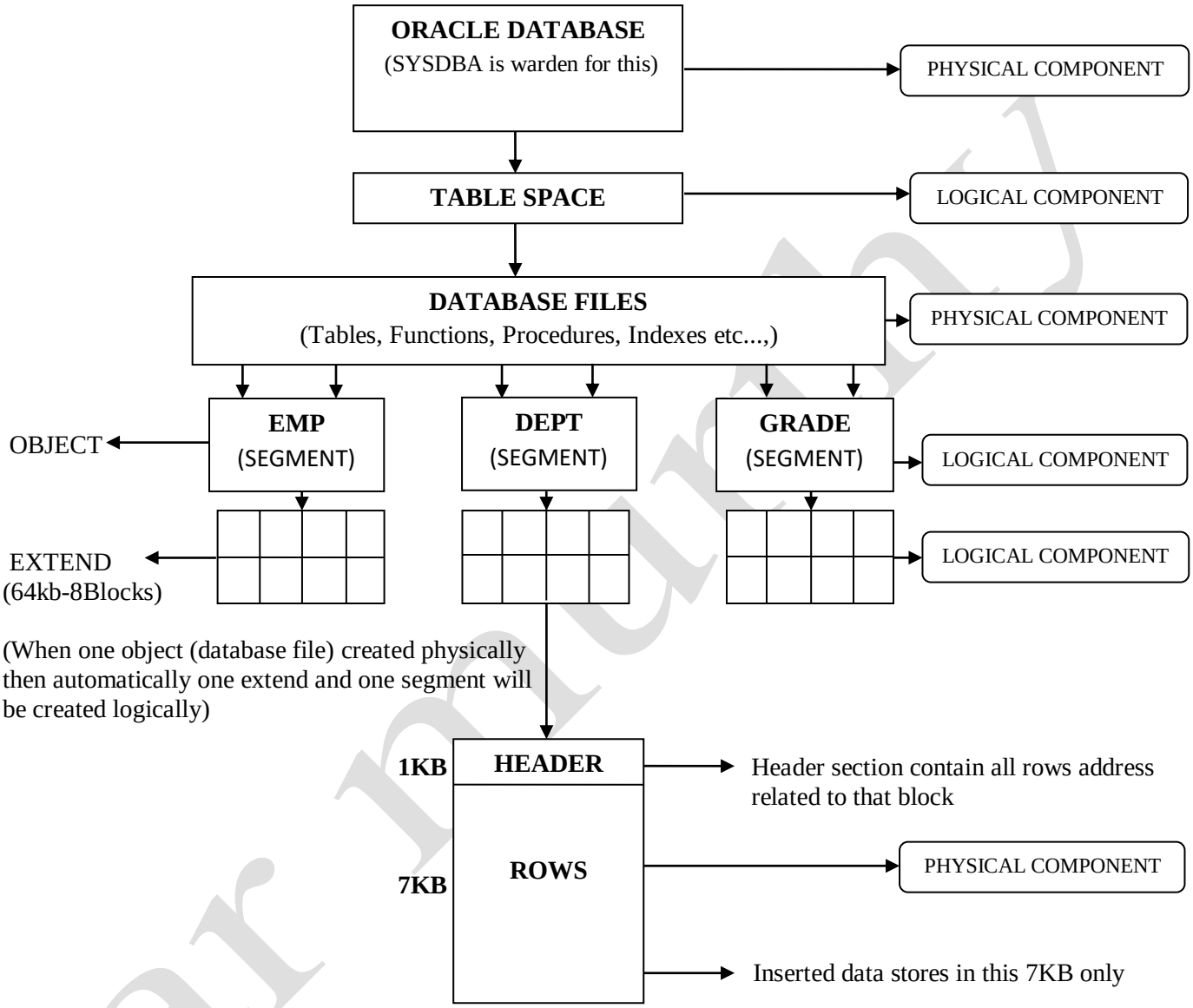
- **Database File:** Database files information exist in this means it contains information about (No. of blocks, block size, total size, total size occupied by all blocks) in a tablespace.

  ↳ `SELECT * FROM V$DATAFILE;`

- **On line redo log file:** LGWR will export the data from redo log buffer to on line redo log file. It works in five conditions 1/3rd of buffer is filled or 1MB of buffer is filled or 25secs completed in red log buffer or when you give commit the when you write any DML. In these five conditions if anyone is satisfied LGWR will move the data from redo log buffer.
- **Archive files:** It contains backup and recovery files exported by ARWR.
- **Control file:** Control file contain timestamp of the database when oracle is installed and also it will contain remaining files location address (database file, redo log file etc..,).

ORCLE DATABASE

Oracle database is some area in hard disk. It will be created automatically when we installing the oracle;



PHYSICAL COMPONENTS	LOGICAL COMPONENTS
• ORACLE DATABASE	• TABLESPACE
• DATABASE FILES	• SEGMENT
• BLOCKS	• EXTEND

TABLESPACE

Table space is some area in database. It is a logical component and one tablespace can contain any number of database files.

TABLESPACE CREATION:

- ```
  CREATE TABLESPACE RAMANA_DEFAULT DATAFILE
  'E:\ORACLE\ORADATA\ORCL\RAMANA_DEFAULT.DBF' SIZE 300M;
  ```

- ```
  CREATE TEMPORARY TABLESPACE RAMANA_TEMP TEMPFILE
  'E:\ORACLE\ORADATA\ORCL\RAMANA_TEMP.DBF' SIZE 50M;
  ```

NOTE: Above address (`'E:\ORACLE\ORADATA\ORCL\.DBF'`) found in `SELECT * FROM V$DATAFILE;`

To extend tablespace:

- ```
  ALTER TABLESPACE RAMANA_DEFAULT ADD DATAFILE
  'E:\ORACLE\ORADATA\ORCL\RAMANA_DEFAULT2.DBF' SIZE 50M;
  ```

In every project we have to maintain different tablespace's for different type of objects. If you want to store the object in particular tablespace you have to write that table space name at the time of creation of that object. If you don't write the tablespace name then it is go and store in the default tablespace which you have given at the time of schema creation.

Ex:

TABLESPACE NAME	TABLESPACE USE
RAMANA_DEFAULT	For default use
RAMANA_TABLES	For tables
RAMANA_INDEXES	For indexes
RAMANA_TEMPORARY	For all temporary operations in the schema like global temporary tables, group by, order by etc.,

NOTE: If you create any global temporary table that data will be stored in temporary tablespace by default. And if you want to do order by, group by operations in separate tablespace then you should go for temporary tablespace.
  ```

SCHEMA

Schema is a collection of database objects. Schema objects are logical structures created by users. You can create and manipulate schema objects with SQL.

SCHEMA CREATION:

Schema Permissions:

```
CREATE USER AR_MURTHY IDENTIFIED BY ACR143 DEFAULT TABLESPACE
RAMANA_DEFAULT TEMPORARY TABLESPACE RAMANA_TEMPORARY;
GRANT CONNECT,DBA TO AR_MURTHY;
GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO AR_MURTHY;
REVOKE READ, WRITE ON DIRECTORY DATA_PUMP_DIR FROM AR_MURTHY;
```

Schema Permissions:

```
CREATE ROLE EQUITAS_MERCHANT;
GRANT CONNECT,CREATE TABLE,CREATE SYNONYM,CREATE PUBLIC
SYNONYM,CREATE PROCEDURE,CREATE TRIGGER TO EQUITAS_MERCHANT;
GRANT EQUITAS_MERCHANT TO RBL_PROJECT;
```

NOTE: 1. If you want to give permission for any schema you should give from SYSDBA only.
2. `CREATE PROCEDURE` is enough for procedure, function, packages, view but for triggers we have mention separately.

To compile entire schema:

```
BEGIN
  DBMS_UTILITY.COMPILE_SCHEMA ('AR_MURTHY');
END;
```

'**DBMS_UTILITY**' is predefined package. It is useful when there is any invalid objects in our schema it will compile all those objects at one shot. If depending object is dropped or modified then program will go to invalid status. Means one table is dropped then all the programs which are depending on that table will go to invalid status after that even we have created that table then also programs will be invalid, if we want to valid the programs we should compile the programs. So we can go for this concept to compile all those programs at once.

This compilation method is also useful when we are sending our SVN portal script to client, if we write this syntax at end of the script it will compile that entire schema, even there is any order mistake in the SVN script then also no problem because it will compile entire schema again so there is no chance to get invalid objects.

To see all invalid objects in schema:

```
SELECT * FROM USER_OBJECTS WHERE STATUS='INVALID';
```

To see all dependency programs on an object:

```
SELECT * FROM ALL_DEPENDENCIES WHERE REFERENCED_NAME='EMP';
```

To see created date and modified date of any object:

```
SELECT OBJECT_NAME,CREATED,LAST_DDL_TIME FROM USER_OBJECTS
WHERE OBJECT_NAME='SP_ERROR_LOG';
```

To see in how many programs you have used one concept:

```
SELECT * FROM ALL_SOURCE WHERE TEXT LIKE '%BULK COLLECT%'
AND OWNER='AR_MURTHY';
```

DATABASE TABLE

TABLE CREATION:

- ```
  CREATE TABLE STUDENT_DTLS( ROLL_NO NUMBER(5) CONSTRAINT PK_ROLL
  PRIMARY KEY USING INDEX TABLESPACE RAMANA_INDEXES,NAME
  VARCHAR2(30),PHONE NUMBER(10))TABLESPACE RAMANA_TABLES;
  ```

- ```
  CREATE TABLE EMP_DTLS AS SELECT E.EMPNO,E.ENAME,D.DNAME
  FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;
  ```

- *THEN SYSTEM WILL REWRITE THE SYNTAX LIKE:*

```
--CREATE TABLE
CREATE TABLE STUDENT_DTLS
      (
        ROLL_NO NUMBER (5) NOT NULL,
        NAME    VARCHAR2 (30),
        PHONE   NUMBER (10)
      )
      TABLESPACE RAMANA_TABLES
      PCTFREE 10
      INITRANS 1
      MAXTRANS 255;
-- CREATE/RECREATE PRIMARY, UNIQUE AND FOREIGN KEY CONSTRAINTS
      ALTER TABLE STUDENT_DTLS
      ADD CONSTRAINT PK_ROLL PRIMARY KEY (ROLL_NO)
      USING INDEX
      TABLESPACE RAMANA_INDEXES
      PCTFREE 10
      INITRANS 2
     MAXTRANS 255;
```

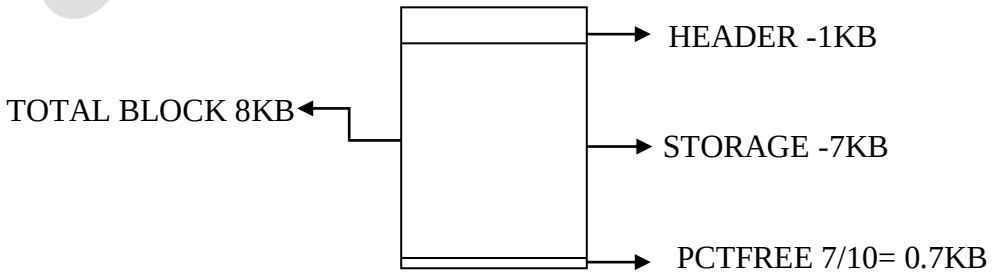
EXPLANATION:

INITRANS, MAXTRANS:

Initial transaction by default 1 for tables and 2 for constraints
Maximum transaction by default 255

Means you able to make maximum 255 transactions without applying commit if you done 256th transaction without commit previous transactions its leads to get system hang.

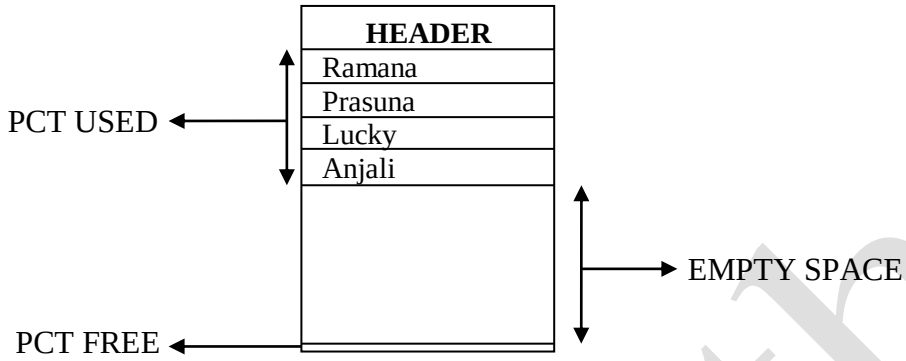
PCTFREE: pctfree means percentage free. It useful for any changes happens in the table in future by using update that space is useful for that extended data. System takes percentage free by default '10' if we not mentioned, means every block contain 0.7kb of pctfree by default.



If you don't want to leave that 0.7kb pctfree then you should write like 'pctfree free'.

PCTUSED**:** pctused means percentage used means what percentage of data you have used in the block. pctfree use for reuse the empty space of block means if 1st block completed then next inserting data stores in 2nd,3rd blocks respectively after that if some data is deleted from 1st block then some empty place forms in 1st block.

When you given pctused are 60 then empty space in the block reaches more than 40% then next inserting data stores in 1st block.



INITIAL 64KB: If create a table then initially one extend will be created. 1 extend contain 8 blocks each block size 8kb.

EXTEND = 8 BLOCKS * 8 KB
= 64KB

Note: when we creating a table it occupies 1MB space.

MIN EXTENTS, MAX EXTENTS:

Min extents always will be 1 because if one table is created then initially 1st one extend only created after that by inserting more data that numbers extents will be extends.

Maximum extents by default unlimited because we don't know that how many extents we needed for our data, if data is increasing then extents also increase. If you want to write maximum extends you can write but how you know that some extends are enough for your data.

GLOBAL TEMPORARY TABLE:

Global temporary table are two types 1 is transaction level and 2nd one is session level.

1. **Transaction Level:** In transaction level global temporary table data will be present in the table until you give the commit.

```
CREATE GLOBAL TEMPORARY TABLE STUDENT_NEW (ROLLNO NUMBER, NAME
VARCHAR2(30), MARKS NUMBER) ON COMMIT DELETE ROWS;
```

2. **Session Level:** In session level global temporary table data will be present in the table until session end (logout).

```
CREATE GLOBAL TEMPORARY TABLE GTT_STUDENT (ROLLNO NUMBER, NAME
VARCHAR2(30),MARKS NUMBER) ON COMMIT PRESERVE ROWS;
```

- **PREDEFINED TABLES:**
  - **PLAN_TABLE;**
  - **PLSQL_PROFILER_DATA;**
  - **V$DATAFILE;**
  - **DUAL;**

To know the table occupied space in database:

```
↳ SELECT * FROM DBA_SEGMENTS WHERE SEGMENT_NAME='EMP';
```

ROWID:

```
ROWID= DB FILE ID + SEGMENT ID +BLOCK ID + ROW NUMBER
```

- ROWID is combination database file_id, segement_id, block_id and row number
- ROWID is a static value means always same row contain same value.
- ROWID is a **hexadecimal** value. Contains 18 numbers.
- It will be store in database. Oracle automatically provides rowid when user inserts the new row into the table.

ROW NUMBER:

- It is numeric value, based on the select statement output row number will generate
- Row number is not a static value based on the output of the select statement it will change dynamically.
- Row number won't store in data base temporarily it will generate for select statement output.

ROWID	ROW NUMBER
• Row id is static value (Always same row contain same rowid).	• Row number is not a static value (Based on the output of select statement row number will change dynamically).
• Row id is a **HEXA DICIMAL** number (18 digit values).	• Row number is numeric value
• Row id is combination of (File ID, Object ID, Block ID, Row Number)	• Row number is numeric value
• Oracle automatically provide row id when user insert new row into the table.	• Based on select statement output row number will generate.
• Row id will be stored in the data base.	• Row number won't store in database (temporarily it will generate for select statement output).

ORACLE means “ Oak Ridge Automatic Computer and Logical Engine ”.

ORACLE IS A BACKEND LANGUAGE

IT CONTAINS: 1. SQL (Structured Query Language);

2. PL/SQL (Procedural Language extension of SQL);

STRUCTURED QUERY LANGUAGE

SQL is used to communicate with a database. It is the standard language for relational database management systems. SQL is standard programming language specifically designed for storing, retrieving, managing or manipulating the data inside a relational database management system (RDBMS).

DDL COMMANDS:

DDL: Data Definition Language

1. CREATE
2. ALTER
   - ADD
   - MODIFY
   - DROP
3. DROP

- **CREATE**: This command is used create objects
  - `CREATE TABLE STUDENT_INFO (ROLL_NO NUMBER(10), NAME VARCHAR2(30),MARKS NUMBER (5,2));`

- **ALTER:** This command is used to make changes in any object
  - `ALTER TABLE STUDENT_INFO ADD ADDRESS VARCHAR2(60);`
  - `ALTER TABLE STUDENT_INFO DROP COLUMN MARKS;`
  - `ALTER TABLE STUDENT_INFO MODIFY ADDRESS VARCHAR2(100)`

- **DROP:** This command is used to delete the object permanently
  - `DROP TABLE STUDENT_INFO;`
  - `DROP TABLE DEPT;`

If we want that table back when by mistake we have dropped that table we should write

- `FLASHBACK TABLE DEPT TO BEFORE DROP;`

All the dropped objects we can see in '`SELECT * FROM RECYCLEBIN`'

If you want to delete that recyclebin then
open SQL command window write like '`SQL> PURGE RECYCLEBIN;`'
If you want to drop one object permanently without storing in recyclebin then
open SQL command window write like '`SQL> DROP TABLE EMP_2 PURGE;`'

- **TRUNCATE:** This command is used to delete table data permanently
  - `TRUNCATE TABLE EMP;`

DML COMMANDS:

DML: Data Manipulation Language

1. INSERT
2. UPDATE
3. DELETE

- **INSERT:** This command is use full for insert data into table/view.
  - `INSERT INTO STUDENT_INFO (ROLL_NO,NAME,MARKS) VALUES (1103,'AR MURTHY', 980);`

Insert with select statement:

  - `INSERT INTO DUM_EMP SELECT * FROM EMP;`
  - `INSERT INTO EMP_INFO(EMPNO, ENAME, PHOTO) VALUES (1103,'RAMANA',BFILENAME('DATA_PUMP_DIR','INST_LOGO_1.JPG'));`

- **UPDATE:** This command used to update the table data/view.
  - `SELECT * FROM STUDENT_INFO SET MARKS=990 WHERE ROLL_NO=1103;`

When one user is updating some records in one table at the same time another user trying to do for update on same table it won't allow because some records are locked in that table. In this kind of situations you want to use for update you should go for skip locked concept.

USER_1: `UPDATE EMP SET COMM=500 WHERE DEPTNO=30;` it will lock all the records from 30th department.

USER_2: `SELECT * FROM EMP FOR UPDATE SKIP LOCKED;` it won't bring any record from 30th department.

- **DELETE:** This command is used delete the data from table/view.
  - `DELETE FROM EMP;`
  - `DELETE FROM STUDENT_INFO WHERE ROLL_NO=1103;`

If we want that data back when by mistake we have deleted and committed that data we should write

  - `INSERT INTO EMP SELECT * FROM EMP AS OF TIMESTAMP SYSDATE-10/24/60;`

DCL COMMANDS:

DCL: Data Control Language

- **GRANT:** To give permissions to another user/schema.
  - `GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO AR_MURTHY;`
  - `GRANT SELECT, INSERT, UPDATE ON SYN_EMP TO AR_MURTHY`
  - `GRANT ALL ON SYN_DEPT TO AR_MURTHY`
  - `GRANT EXECUTE ON SYN_SP_COMM_UPDATE TO AR_MURTHY`
  - `GRANT EXECUTE ON PKG.P1 TO AR_MURTHY`
- **REVOKE:** To cancel the permissions for any user/schema
  - `REVOKE READ, WRITE ON DIRECTORY DATA_PUMP_DIR FROM AR_MURTHY;`
  - `REVOKE EXECUTE ON PKG.P1 FROM AR_MURTHY`

DQL COMMAND:

DQL: Data Query Language.

- **SELECT:** To retrieving the data and if we want to retrieve any filtered data then you can select the data by using where clause.
  - `SELECT * FROM EMP`
  - `SELECT * FROM EMP WHERE JOB='SALESMAN' AND SAL>1000;`
  - `SELECT DEPTNO,SUM(SAL) FROM EMP GROUP BY DEPTNO HAVING SUM(SAL)>10000;`

- **WHERE:** Where clause is used to put a condition on rows data and where clause will filter the rows data
  - `SELECT COUNT(*) FROM EMP WHERE DEPTNO=10;`

TCL COMMANDS:

TCL: Transaction Control Language.

It will control complete action of a transaction by either we give commit or rollback means every transaction completely under control of TCL (commit, rollback)

Note: Transaction means insert or update or delete.

`COMMIT` : To store the transaction (save work done);
`ROLLBACK` : To cancel the transaction (means it will restore database to original since the last commit);

SPECIAL CLAUSES:

- **GROUP BY:** Group by clause is used to group the rows data
  - `SELECT DEPTNO,SUM(SAL) FROM EMP GROUP BY DEPTNO`
  - `SELECT DEPTNO,JOB, COUNT(*) FROM EMP GROUP BY DEPTNO,JOB ORDER BY DEPTNO`

- **HAVING:** Having clause is used to put a condition on grouped data. Having clause always follow by 'Group by' clause. Means having clause filters the grouped data only.
  - `SELECT DEPTNO,COUNT(*) FROM EMP GROUP BY DEPTNO HAVING DEPTNO=20;`

- **ORDER BY:** Order by clause is used bring the data order wise on which column based you want to bring the order.
  - Oder by will support to both `WHERE` and `HAVING` clauses.
  - `ASC` for ascending order.
  - `DESC` for descending order.

  - `SELECT * FROM EMP ORDER BY SAL DESC`
  - `SELECT * FROM EMP WHERE COMM IS NOT NULL ORDER BY SAL`
  - `SELECT DEPTNO,MAX(SAL) FROM EMP GROUP BY DEPTNO ORDER BY MAX(SAL)`

SPECIAL OPERATORS:

- **BETWEEN / NOT BETWEEN:** If you want to put condition on data like values between 10 and 20 for this kind of situations you should go with BETWEEN;
  - Search for the values within the range.
  - Used with numbers & data values only.
  - We can't use this for character data type.

```
SELECT * FROM EMP WHERE SAL BETWEEN 1000 AND 2000;
SELECT * FROM EMP WHERE SAL NOT BETWEEN 1000 AND 2000;
SELECT SAL,JOB,COUNT(*) FROM EMP GROUP BY SAL,JOB HAVING SAL
BETWEEN 1000 AND 2000;
```

- **TRUNC:** To remove dismals

```
SELECT TRUNC(SYSDATE) FROM DUAL;
```

- **DISTINCT:** To avoid duplicate values in the column. Means it is useful to know how many types of values in the column.

```
SELECT DISTINCT JOB FROM EMP;
```

- **LIKE / NOTLIKE:**
  - It is only useful for characters
  - Used to search for character values with a pattern.
  - It uses 2 meta characters
    1) Under Score : _ : represents single character;
    2) Percentage : % : represents zero or more characters;

```
SELECT * FROM EMP WHERE ENAME LIKE 'J%'; (NAME'S START WITH 'J')
SELECT * FROM EMP WHERE ENAME LIKE '%S'; (NAME'S ENDS WITH 'S')
SELECT * FROM EMP WHERE ENAME LIKE '%A%'; (WHICH NAME'S HAVING
'A' IN THEIR SELVES);
```

NOTE: %, _. These two are Meta characters for like.

If we want to bring the names which have '%' in their self.

```
SELECT * FROM EMP WHERE ENAME LIKE '%/%%' ESCAPE '/';
```

If we want to bring the names which have '_' in their self.

```
SELECT * FROM EMP WHERE ENAME LIKE '%/_%' ESCAPE '/';
```

- **IN & NOT IN:** If you want to put two or more records in filter condition then better to go for IN, NOT IN operators.
  - Picks one by one value from list of values.
  - Supports with all data types.
  - We can use this to retrieve/manipulate data fast.
  - NOT IN won't support to NULL.

```
SELECT * FROM EMP WHERE SAL IN (500,1500,2500);
SELECT * FROM EMP WHERE SAL NOT IN (1000,2000,3000);
```

- **IS NULL & IS NOT NULL:**
  - It is an undefined and incomparable value.
  - It is not equal to space or zero.
  - It will not occupy any memory.
  - Any arithmetic operation with null returns null only.
  - It is representing with null keyword shown as space on to screen.
  - Supports all data types.

```
SELECT * FROM EMP WHERE MGR IS NULL;
SELECT * FROM EMP WHERE COMM IS NOT NULL;
```

- **WITH:** It will store one query result and provides one alias name for that.

```
WITH T AS (SELECT MAX(SAL)SAL FROM EMP)
SELECT * FROM EMP E, T WHERE E.SAL=T.SAL
UNION ALL
SELECT * FROM EMP E, T WHERE E.SAL=T.SAL AND E.DEPTNO=10;
```

DUAL TABLE:

It is system defined table. It supports to retrieve general information from select statement. It is useful to do calculations with numbers and dates also useful to bring days by using dates calculations.

We can't make any changes in this table means we are not able to do insert, update or delete any information in this dual table

```
SELECT TO_CHAR(TO_DATE('15/8/1947','DD/MM/YYYY'),'DAY')FROM DUAL;
SELECT TO_CHAR(SYSDATE,'DD')||'TH '||TO_CHAR (SYSDATE,'MONTH') ||'
'||TO_CHAR(SYSDATE,'DAY')||' ' ||TO_CHAR(SYSDATE,'YYYYSP') QUARIE_75
FROM DUAL;
SELECT TRUNC((SYSDATE-TO_DATE('15/8/1947','DD/MM/YYYY'))/365) FROM
DUAL;
```

AGGREGATE FUNCTIONS:

- Oracle provides some predefined aggregate functions like SUM, AVG, MAX/MIN, COUNT etc.., to make calculations easier.
- Aggregate functions always gives output in single row only
- We can't write normal columns along with aggregate functions without using 'GROUP BY' clause;

```
SELECT MAX(SAL) FROM EMP;
SELECT JOB,COUNT(*) FROM EMP GROUP BY JOB;
SELECT DEPTNO FROM EMP GROUP BY DEPTNO HAVING SUM(SAL)>9000;
SELECT JOB,AVG(SAL),COUNT(JOB) FROM EMP GROUP BY JOB,SAL;
```

NOTE: COUNT gives that no of values stored in the normal column or grouped column;

SET OPERATORS:

- **UNION:**
  - ```
    SELECT EMPNO FROM EMP
    UNION
    SELECT DEPTNO FROM DEPT
    ```
    - It is mostly useful in mixing data from two tables.
    - It works like 'OR' condition.
    - 'OR' only useful in one table but 'UNION' useful to mix two tables data.
    - It gives unique values.

- **MINUS:**
  - ```
    SELECT EMPNO FROM EMP WHERE DEPTNO=20
    MINUS
    SELECT EMPNO FROM EMP WHERE SAL>1000
    ```
    - It works like opposite 'AND' condition.
    - It gives not matched data between the results.

- **INTERSECT:**
  - ```
    SELECT EMPNO FROM EMP WHERE DEPTNO=20
    INTERSECT
    SELECT EMPNO FROM EMP WHERE SAL>1000
    ```
    - It works like 'AND' condition
    - It gives matched data between the results.

- **UNION ALL:**
  - ```
    SELECT EMPNO FROM EMP WHERE DEPTNO=20
    UNION ALL
    SELECT EMPNO FROM EMP WHERE SAL>1000
    ```
    - It will merge the two results
    - And it gives duplicate values.

MERGE TABLE:

If you want to merge the data from any table by importing data from another table then you will go for table merge. It will do two actions at a time they are updating and inserting.

- Updating by modifying data which are existing in source table.
- Inserting the data remaining data which are not found in target table.
- ```
  MERGE INTO EMP_NEW T USING EMP S ON (T.EMPNO = S.EMPNO) WHEN MATCHED
  THEN UPDATE SET T.SAL=S.SAL, T.COMM=S.COMM WHEN NOT MATCHED THEN
  INSERT VALUES (S.EMPNO,S.ENAME,S.JOB,S.MGR,S.SAL,S.COMM,S.DEPTNO);
  ```

  Note: For example in employee tables data always there is a chance to get changes in salary, commission, job etc., columns only, mostly id number, name, hire date, always in same position so better to mention only columns which are getting changes in update statement.
  ```

DATE FUNCTIONS:

Oracle provides some predefined date functions like ADD_MONTHS, MONTHS_BETWEEN, NEXT_DAY, LAST_DAY, TO_CHAR etc.., to make date calculations easier.

```
SELECT ADD_MONTHS(SYSDATE,12) FROM DUAL;
SELECT *FROM EMP WHERE TRUNC(MONTHS_BETWEEN(SYSDATE,HIREDATE)/12)>30;
SELECT NEXT_DAY(SYSDATE,'SUN') FROM DUAL;
SELECT LAST_DAY(LAST_DAY(SYSDATE)+1) FROM DUAL;
SELECT TO_CHAR(ROUND(ADD_MONTHS(SYSDATE,-8)),'DAY') FROM DUAL;
SELECT EMPNO,ENAME,SAL,TRUNC(MONTHS_BETWEEN(SYSDATE,HIREDATE)/12)
FROM EMP;
```

- **TO_CHAR:** This function converts any date value to a char value in the specified format.
- **TO_DATE:** It converts string into date value.
- **MONTHS_BETWEEN:** This function is used to find the number of months between the given two dates.
- **ADD_MONTHS:** This function is used to add specific number of months to given date.
- **LAST_DAY:** This function is used to find out the last day of month for the given date.
- **NEXT_DAY:** This function is used to find next day date based on the given date.
- 'D'-day of the week.
- 'DD' -day of the month.
- 'DDD' -day of the year.
- 'DY' -WEN (day name in 1st 3 characters).
- 'DAY' -Wednesday (full name of the day).
- 'MM' - 05 (month).
- 'MON' -APR (month name in 1st 3 characters).
- 'MONTH' -April (full name of the month).
- 'YY' -last two digits of year.
- 'YYY' -2020 (YEAR).
- 'HH' -10 (hour).
- 'HH24' -16 (means 4pm).
- 'HH12' -4.
- 'MI' -minute.
- 'SS' -seconds.
- 'Q' –quarter of the year.
- 'CC' –century of the year.
- 'WW' -45 week of the year.
- 'W' –week of the month.
- 'SP' –spell out.
- 'DDSP' –today's date is 20th.
- 'YYYYSP' –two thousand twenty.

CHAR FUNCTIONS:

Oracle provides some predefined char functions like LENGTH, SUBSTR, INSTR, REPLACE, TRIM, UPPER/LOWER, REVERSE, TRIM etc.,

```
SELECT LENGTH('RAMANA MURTHY') FROM DUAL;
SELECT SUBSTR('RAMANA MURTHY',3) FROM DUAL;
SELECT INSTR('HAI RAMANA MURTHY','A',3,2) FROM DUAL;
SELECT REPLACE('RAMANA MURTHI','I','Y') FROM DUAL;
SELECT UPPER('RAMANA MURTHY'),LOWER('RAMANA MURTHY') FROM DUAL;
SELECT REVERSE('YHTRUM ANAMAR') FROM DUAL;
SELECT CHR(23),ASCII('B') FROM DUAL;
SELECT TRIM('  RAMANA MURTHY  ') FROM DUAL;
SELECT CONCAT(ENAME,EMPNO) FROM EMP;
SELECT  WM_CONCAT(ENAME) FROM EMP;
```

NOTE:

- If you pass the number then `CHR` will give you the string which is related to the number
- If you pass the string then `ASCII` will give you that string number
- Means every string has their individual number

- `TRIM` will remove unnecessary space placed at edges of the sentence
- `LTRIM` will remove unnecessary space placed at left edge of the sentence
- `RTRIM` will remove unnecessary space placed at right edge of the sentence

- `CONCAT` will mix the strings or two columns data
- **`WM_CONCAT`** will mix the entire column data and put in single row each row data separates with ‘,’;

♦ CHAR DATA TYPES:

- Char
- Varchar2
- Long
- Clob

CHAR:

✓ Supports to maximum 2000.
✓ By default it takes size 1, if we want to write the size we can write and it always occupy the space which we have provided for every inserting record it doesn’t care length of the record we are inserting.

VARCHAR2:

✓ 4000 up to 11g and 32767 from 12c.
✓ We should provide size for Varchar.
✓ What size of the record we are inserting that length only it occupy if we inserting the record that record is more than provided size then it won’t allow.

LONG:

- ✓ Maximum size 2GB.
- ✓ One table can allow one long data type column.
- ✓ We should not write size for long.

CLOB:

- ✓ Maximum size 4GB.
- ✓ One table can allow any number of clob data type columns.
- ✓ We should not write size for clob.

ANALYTICAL FUNCTIONS:

Generally we can't write aggregate functions along with normal columns without using group by clause. But without group by clause we want to write aggregate function with normal column then we should go for the 'Analytical Function'.

- ✓ OVER()
- ✓ OVER(ORDER BY)
- ✓ OVER(ORDER BY **SAL** ROWS UNBOUNDED PRECEDING)
- ✓ RANK()
- ✓ **DENSE_RANK**()
- ✓ **ROW_NUMBER**()
- ✓ **LISTAGG**

```
SELECT ENAME,SAL,SUM(SAL)OVER() FROM EMP;
```

```
SELECT ENAME,SAL,SUM(SAL)OVER(ORDER BY SAL) FROM EMP;
```

```
SELECT ENAME,SAL,SUM(SAL)OVER(ORDER BY SAL ROWS UNBOUNDED PRECEDING)
FROM EMP;
```

- ROWS UNBOUNDED PRECEDING will avoid the overlapping values in sum(sal) column;

```
SELECT ENAME,DEPTNO,SAL,SUM(SAL) OVER(PARTITION BY DEPTNO) SAL2 FROM
EMP;
```

- PARTITION BY will dived the data into groups based on the column which you have preferred;

```
SELECT ENAME,DEPTNO,SAL,DENSE_RANK() OVER( PARTITION BY DEPTNO ORDER
BY SAL DESC) FROM EMP;
```

- **DENSE_RANK**() is useful to provide ranks;

```
SELECT ENAME,DEPTNO,SAL,DENSE_RANK() OVER(PARTITION BY DEPTNO ORDER
BY SAL DESC)DRK, RANK() OVER(PARTITION BY DEPTNO ORDER BY SAL
DESC)RNK, ROW_NUMBER() OVER(PARTITION BY DEPTNO ORDER BY SAL DESC)
RNM FROM EMP;
```

- **DENSE_RANK**() if five members got 1st rank **DENSE_RANK** will provides 1st rank to those five members and provides 2nd rank to sixth member;
- RANK() if five members got 1st rank then RANK will provides 1st rank to those five members and provides 6th rank to sixth member;
- **ROW_NUMBER**() will simply provides one rank for each row;

- **LISTAGG (Analytical Function):**
  - `SELECT DEPTNO,LISTAGG(ENAME,'-') WITHIN GROUP (ORDER BY ENAME) FROM EMP GROUP BY DEPTNO;`

    It's works like WM_CONCAT with group by clause.

    Differences:
    - LISTAGG separate the records with different type special characters. WM_CONCAT by default separate the records with comma only.
    - We can use order by clause in LISTAGG for any column in the table. We can use order by clause in WM_CONCAT for only column which is we used to group by.

- **ROLLUP AND CUBE:** These two are always follows with group by clause.
  - 'ROLLUP' will give the sum of group on which column based you grouped the data and also gives the grand total of all groups.
    - `SELECT DEPTNO,JOB,SUM(SAL) FROM EMP GROUP BY ROLLUP (DEPTNO,JOB);`
  - 'CUBE' will give the sum of group on which based your group the data and also gives the grand total of all groups and additionally it will give sub group wise some, and it gives grand total in top row.
    - `SELECT DEPTNO,JOB,SUM(SAL) FROM EMP GROUP BY CUBE (DEPTNO,JOB);`

ARITHMATIC FUNCTIONS:

Oracle provides some predefined arithmetic functions like POWER, ABS/SIGN, GREATEST/LEAST, CEIL/FLOOR, ROUND, EXP, TRUNC to make arithmetic calculations easier.

- `SELECT POWER(3,3),SQRT(625) FROM DUAL;`
- `SELECT ABS(-2000) , ABS(900) FROM DUAL;`
- `SELECT SIGN(-200), SIGN(100) FROM DUAL;`
- `SELECT GREATEST(10,20,-100), LEAST(10,-200,300) FROM DUAL;`
- `SELECT CEIL(10.0001),FLOOR(99.99) FROM DUAL;`
- `SELECT ROUND(1234567.987,2) ,ROUND(1234567.987,2) FROM DUAL;`
- `SELECT ROUND(1234567.487) FROM DUAL;`
- `SELECT EXP(2) FROM DUAL;`
- `SELECT TRUNC(1000.10), TRUNC(1000.4444,2) FROM DUAL;`

NOTE:

- Given value is positive or negative `ABS` will always return positive values only
- If given value is positive then `SIGN` returns positive value else if given values is negative then `SIGN` returns negative value
- `CEIL` always gives next round figure values of the given dismal and `FLOOR` always gives previous round figure values of the given dismal
- `ROUND` this function is used to find the round the number to nearest.
- `EXP` given E power N result (natural algorithm).
- `TRUNC` this function used to delete dismals or delete dismals from some position.

SEQUENCE:

Sequence is used generate the number. It will generate unique numbers that forward wise. When we need to get unique values forward wise and we don't know that what is previous value then we have to call sequence in this kind of situation.

- `CREATE SEQUENCE SEQ_EMPNO START WITH 1500 INCREMENT BY 1 NOCACHE;`
- `CREATE SEQUENCE SEQ_EMPNO START WITH 1000 INCREMENT BY 1 MAXVALUE 2000 NOCACHE;`
- `SELECT SEQ_EMPNO.NEXTVAL FROM DUAL`
- `INSERT INTO EMP(EMPNO) VALUES (SEQ_EMPNO.NEXTVAL)`

NOTE:

1. By default:
   No Cycle
   Cache - 20
   Start with - 1
   Increment by - 1
   Min value - 1
   Max value - 18 number of 9's (999999999999999999);
2. Always min value <= Start with.
3. We can't rollback the sequence (drawback/disadvantage).
4. We can't call CURRVAL without calling NEXTVAL.
5. To know the sequence current value we can see in LAST_NUMBER column in this:
   `SELECT * FROM USER_SEQUENCES;`

KILL THE SESSION:

```
SELECT U.OBJECT_NAME,V.SESSION_ID,S.SERIAL#,S.OSUSER
FROM USER_OBJECTS U, V$LOCKED_OBJECT V, V$SESSION S
WHERE U.OBJECT_ID=V.OBJECT_ID AND V.SESSION_ID=S.SID;
```

1st way:

- Tools
- Sessions
- Select the session id and right click on that session id then kill that session;
  or
  Select the session id and click on the kill button.

2nd way:

- `ALTER SYSTEM KILL SESSION '25,3563' IMMEDIATE`

  Here: 25 - Session id
  3563 - Serial number.

INDEX

Index will point out the physical address of the data. Index completely designed based on rowid. rowid nothing but address of the data that's why when we are using index column in where condition for update, delete or select output will come very fast. Means index is useful to retrieving or manipulating the data when we use that indexed column in where condition.

NOTE: Index won't store the entire data of the table. It stores only column data with their rowid's on which column you have created the index in leaf blocks. Once search the data using where condition it catches the rowid's of the conditioned data and then brings the data from table by using that rowid's.

To search indexes list of the schema:

↳ `SELECT * FROM USER_INDEXES;`

To search which column having indexes:

↳ `SELECT * FROM USER_IND_COLUMNS;`

Q: On which column you have to create index..?
A: Mostly on which column based we have searching the data on that column only you have to create index.

- **TYPES OF INDEXES:**
  - B-tree index
  - Unique index
  - Bitmap index
  - Function based index
  - Reverse key index

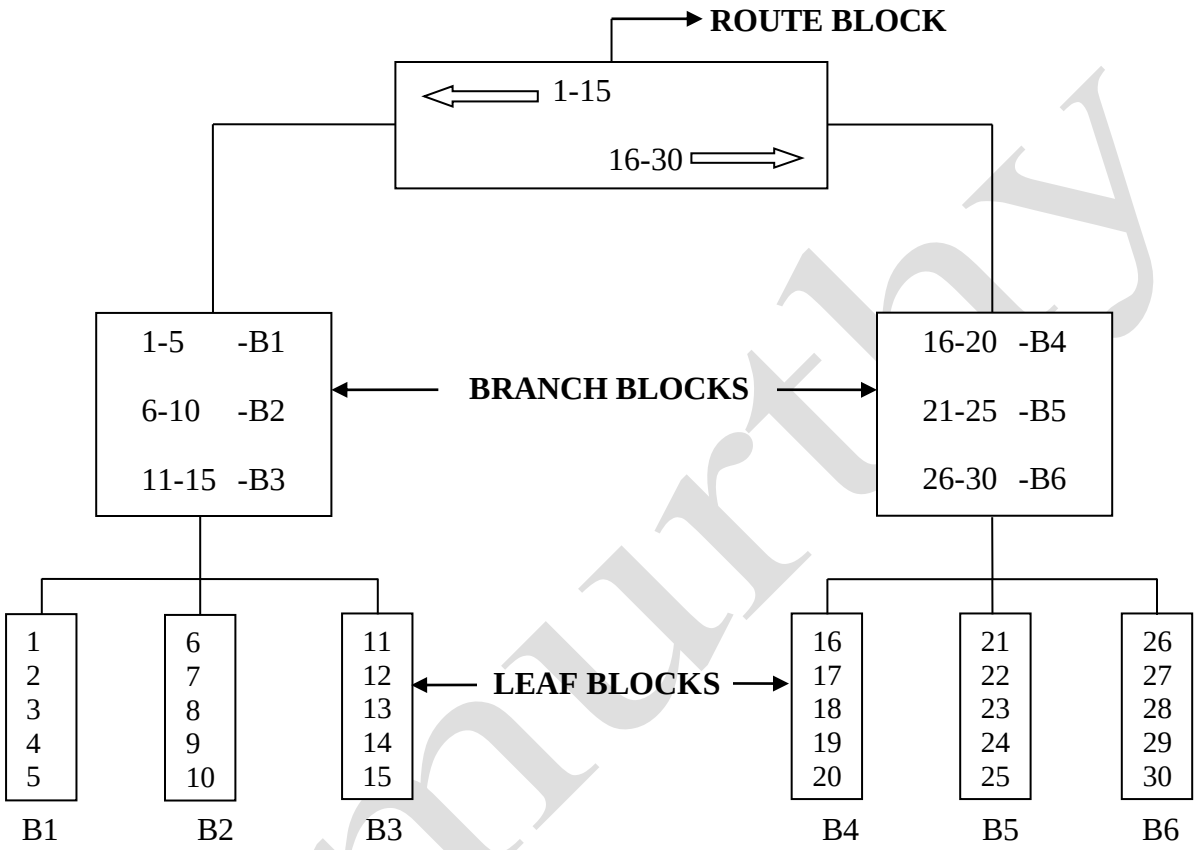
$$\text{CORDINALITY} = \frac{\text{TOTAL NUMBER OF RECORDS} - \text{NULL VALUES}}{\text{DISTINCT VALUES}}$$

WHEN CARDINALITY = 1	THEN UNIQUE INDEX
WHEN CARDINALITY BETWEEN 1 AND 20	THEN B-TREE INDEX
WHEN CARDINALITY >20 THEN	THEN BITMAP INDEX

♦ **B-TREE INDEX:**

➢ `CREATE INDEX IDX_EMPNO ON EMP(EMPNO) TABLESPACE RAMANA_INDEXES;`

B-tree index will separate the data into leaf blocks to make data searching easier. B-tree index contain root, branch, leaf blocks, root blocks, branch blocks contain directions and leaf blocks contain the data.



♦ **UNIQUE INDEX:**

➢ `CREATE UNIQUE INDEX IDX_EMPNO ON EMP(EMPNO) TABLESPACE RAMANA_INDEXES;`

Unique index also works like b-tree index but small differences is there, in b-tree index even searching record founded then also it verifies next row for confirmation of is same record exists in next row if not found then it will bring data from table by using founded rowid's.

But in unique index if searching record is found then immediately its go and bring the data from the table by using the only one rowid. There is no chance to go and search in next row for same record because unique index blindly follows that column does not contain any duplicate values.

When we are creating unique or primary key constraints on any column automatically unique index will be created.

➢ `ALTER TABLE STUDENT_DTLS ADD CONSTRAINT PK_ROLL PRIMARY KEY (ROLL_NO)USING INDEX TABLESPACE RAMANA_INDEXES;`

- **BITMAP INDEX:**
  - `CREATE BITMAP INDEX IDX_DEPTNO ON EMP(DEPTNO) TABLESPACE RAMANA_INDEXES;`

If column having repeating data then better to go for bitmap index, mostly for gender, branch id columns we will create bitmap index.

CIVIL	RAMANA (rowid)	0	0	0	LUCKY (rowid)	0
MECH	0	0	ARYAN (rowid)	0	0	0
EEE	0	STARK (rowid)	0	0	0	0
ECE	0	0	0	0	0	KALEESI (rowid)
CSE	0	0	0	ROBBERT (rowid)	0	0

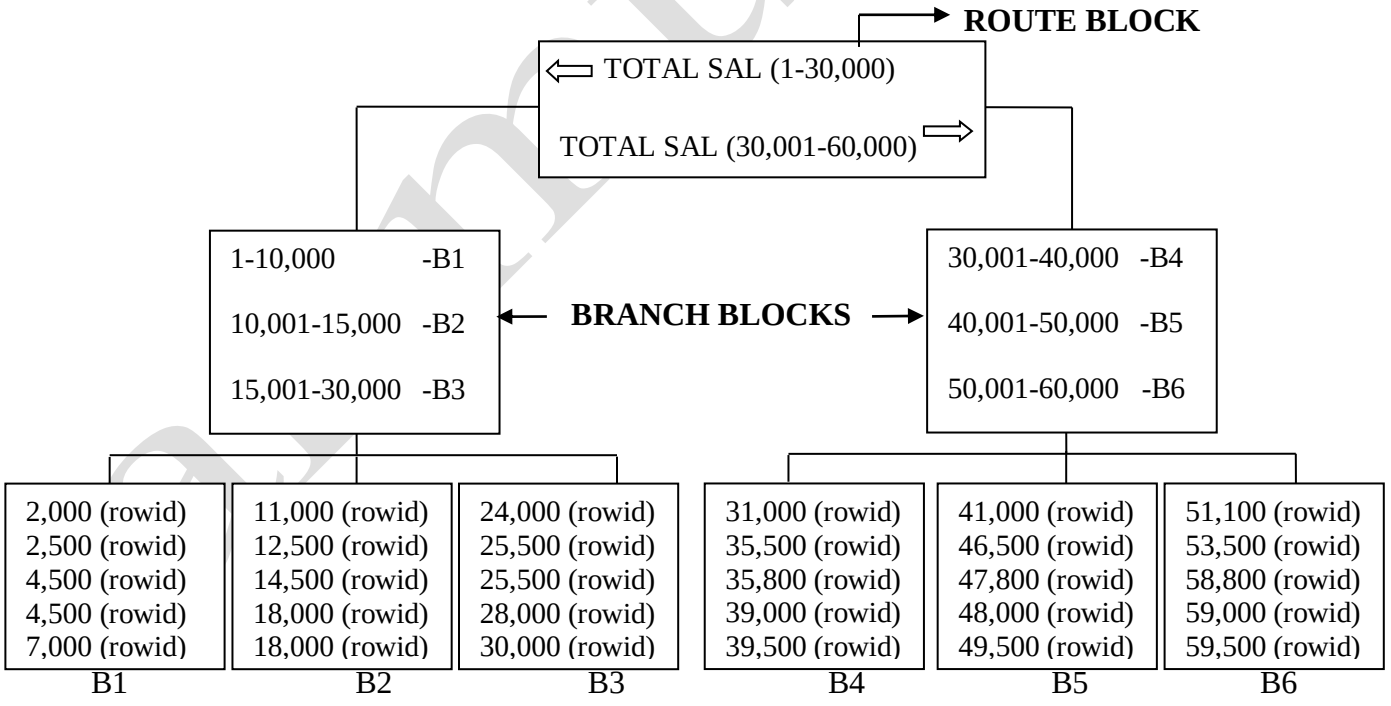
When we create a bitmap index then immediately this table will be created internally.

- **FUNCTION BASED INDEX:**
  - `CREATE INDEX IDX_SF_TOTAL ON EMP(SF_TOTAL_SAL(SAL,COMM,TAX)) TABLESPACE RAMANA_INDEXES;`

Here we are creating a index based on function.

Ex: We have function for total salary SF_TOTAL_SAL (SAL+COMM+TAX).

If we create index based on this total salary function that index will stores like below.



Then we have to search data like

'`Select * from emp where sf_total_sal(sal,comm,tax)=5000`'

Then system directly goes to B2 block and collect that total salary rowid and bring the data from emp table by using that rowid's.

❖ **REBUILD INDEX:**

If you are performing more DML operations on the index may be it will go for invalid status. When you run one query if it is gives output very slowly then you should go for execution plan, if you are going for the execution plan in execution plan you are not see the index scan then you will realize that index is not working using oracle optimization so then immediately we have to do rebuild the index (active the index).

↳ `ALTER INDEX IDX_EMPNO REBUILD;`

❖ **ONLINE KEY:**

If you are trying to rebuild the index at the same time somebody is writing DML operation or writing select statement on that table there is chance to system struck or hang. If you write online keyword at the time of creation of any kind of index then there is no chance to dead lock or hang errors at the time of rebuild the index then we can rebuild the index at any time.

↳ `CREATE INDEX IDX_EMP ON EMP(EMPNO) ONLINE;`

↳ `ALTER INDEX IDX_EMP REBUILD ONLINE;`

↳ `DROP INDEX IDX_EMP;`

♦ **REVERSE KEY INDEX:**

➢ `CREATE INDEX IDX_EMPNO ON EMP(EMPNO) REVERSE;`

It will store the values in reverse order in their self means if you insert 304 it will store like 403. It won't modify any data in original blocks it just store data as reverse order in index only. In main table data will be inserted like what you have inserted. When you are inserting the data in table like EMPNO=304 then immediately index will bring the rowid and store it in index as EMPNO=403 with the same rowid.

Then if write "`SELECT * FROM EMP WHERE EMPNO=304`" index will go and bring the rowid of 403 give the output by using that rowid, actually that rowid is belongs to 304 but index has stored that in reverse order means that rowid and remaining blocks data never change.

It helps index by avoiding invalid status when we are performing more DML operations on the table. Because if table having 500 records then index store like 1-250 at left side and 251-500 right side like B-tree index. Then if you write "`DELETE FROM EMP WHERE EMPNO BETWEEN 100 AND 200`" then index records delete from both side because

102, 121, 302, 342 etc.,→ 201, 121, 203, 243 are store in right side.

103, 113, 204, 263 etc.,→ 301, 311, 402, 362 are store in right side.

If you are the records between 100 and 200 from the table then index records deletes from left side and right side, then no chance to get so much difference between left side blocks and right side blocks so there is no chance for index to get invalid status.

INDEX SCANS

These are avoid the table access full and collect the data from table within short time then table access full.

TABLE ACCESS FULL:

Table access full is nothing but system blindly goes and searches all the rows for a particular object without knowing the address.

When the column which you have used to filter the data by where clause does not having any index or the index is not a selective index then system goes for table access full.

- **INDEX SCAN TYPES:**
  - INDEX RANGE SCAN
  - UNIQUE INDEX SCAN
  - INDEX FULL SCAN
  - INDEX FAST FULL SCAN
  - INDEX SKIP SCAN

$$\text{SELECTIVITY} = \frac{\text{TOTAL DISTINCT VALUES}}{\text{TOTAL VALUES}}$$

If selectivity is 0 to 0.5	then Bitmap Index
If selectivity = 1	then B-tree/ Unique index
If selectivity is 0.5 to 1	then B-tree index

NOTE: Cardinality is useful for users to select which index is better for the column.
Selectivity is useful for system to select on which index scan have to search.
If it is not a selective index then system will skip index scan and go for table access full.

- **INDEX RANGE SCAN:**

Either we are searching left or right side then called index range scan. We are know that the range will mentioned in root block (Ex. 1-10 left side 11-20 right side) by using this range in root block index range scan will go search the data.

- **UNIQUE INDEX SCAN:**

It is also works like index range scan but there is a small difference in index scan even record is found then also it will and search for same record in next row. But unique index scan if searching record is found then immediately it catches the rowid and stops searching then go and bring the data from table by using the rowid. Because it is blindly follows that column does not contain any duplicate values. It is mostly works for unique indexes.

- **INDEX FAST FULL SCAN:**

  When we create index on a column and we need to collect all the data from that column only without any where clause or order by clause it is definitely goes for index fast full scan.

  Ex: EMPNO column has index in EMP table. Then you write query like

  "`select empno from emp`". In this condition system goes for index fast full scan.

- **INDEX FULL SCAN:**

  When we create index on a column and we need to collect all the data from that column only with using order by clause then it's go for index full scan.

  Ex: EMPNO column having has index in EMP table. Then you write query like

  "`select empno from emp order by empno`".

  In this condition system go for index full scan because we are asking all the data from the column which is having index only but we have written order by clause that's why it won't go for fast full scan.

- **INDEX SKIP SCAN:**

  Whatever the data we are searching and which column based we are searching data even that column having index but still there is chance to table access full because may be that index not a selective index.

  Here even that column have index but system will skip the index scan and go for table access full that's why this is called "index skip scan".

INDEX ORGANIZED TABLE:

If we confidently know that every time you have to search/filter the data based on one specific column only. Then better to go for index organized table but the column should have primary key constraint. In index organized table entire table data will stores in leaf blocks there is no need to store data separately. Here entire table data separates in leaf blocks. In index organized table leaf blocks contains all column data with rowid's in their self. But the disadvantages are when we are searched the data by using another column it's completely failure or go to table access full (takes lot of time).

```
CREATE TABLE STUDENT_DTLS( ROLL_NO NUMBER(5),NAME VARCHAR2(30),PHONE
NUMBER(10),CONSTRAINT PK_ROLL PRIMARY KEY(ROLL_NO)) ORGANIZATION
INDEX TABLESPACE RAMANA_TABLES;
```

PARTITION

Partition will divide larger table data into smaller more manageable pieces. Partition allows tables, indexes and index organized table to be sub divide into smaller pieces.

- If table having more than 2GB data then we should go for partitioning.
- When the contents of a table need to be distributed across different types of storage devices.

♦ RANGE PARTITION:

A table that is partitioned by range is partitioned in such a way that each partition contains rows for which the partitioning expression value lies within a given range.

```
CREATE TABLE EMPLOYEE(EMP_NO NUMBER(2),EMP_NAME VARCHAR(2))
PARTITION BY RANGE(EMP_NO) (PARTITION P1 VALUES LESS THAN(10),
PARTITION P2 VALUES LESS THAN(20), PARTITION P3 VALUES LESS
THAN(30),PARTITION P4 VALUES LESS THAN(MAXVALUE));
```

Inserting records into partitioned table:

```
INSERT INTO EMPLOYEE VALUES(101,'A');   ------THIS WILL GO TO P1
INSERT INTO EMPLOYEE VALUES(201,'B');   -------THIS WILL GO TO P2
INSERT INTO EMPLOYEE VALUES(301,'C');   ------THIS WILL GO TO P3
INSERT INTO EMPLOYEE VALUES(401,'D');   ------THIS WILL GO TO P4
```

Select records from partitioned table:

```
SELECT *FROM EMPLOYEE;
SELECT *FROM EMPLOYEE PARTITION(P1);
SELECT *FROM EMPLOYEE WHERE EMP_NO IN (101,201);
```

Adding a partition:

```
ALTER TABLE EMPLOYEE ADD PARTITION P5 VALUES LESS THAN(40);
```

Dropping a partition:

```
ALTER TABLE EMPLOYEE DROP PARTITION P1;
```

Renaming a partition:

```
ALTER TABLE EMPLOYEE RENAME PARTITION P3 TO P6;
```

Truncate a partition:

```
ALTER TABLE EMPLOYEE TRUNCATE PARTITION P5;
```

Splitting a partition:

```
ALTER TABLE EMPLOYEE SPLIT PARTITION P2 AT(12)INTO(PARTITION
P21,PARTITION P22);
```

Exchanging a partition:

```
ALTER TABLE EMPLOYEE EXCHANGE PARTITION P2 WITH TABLE EMPLOYEE_X;
```

Moving a partition:

```
ALTER TABLE EMPLOYEE MOVE PARTITION P21 TABLESPACE ABC_TBS;
```

♦ **LIST PARTITION:**

List partitioning enables you to explicitly control how rows map to partitions by specifying a list of discrete values for the partitioning key in the description for each partition.

```
CREATE TABLE EMPLOYEE (EMP_NO NUMBER(2),EMP_NAME VARCHAR(2),JOB
VARCHAR2(10)) PARTITION BY LIST(JOB)(PARTITION P1 VALUES('BOSS'),
PARTITION P2 VALUES('MANAGER','HR'),PARTITION P3 VALUES
('ACCOUNTANT'),PARTITION P4 VALUES('CLERK','SUPERVISOR'));
```

Inserting records into partitioned table:

```
INSERT INTO EMPLOYEE VALUES(10,'RAMANA','BOSS'); -- GO TO P1
INSERT INTO EMPLOYEE VALUES(11,'KITTU','SUPERVISOR' -- GO TO P4
INSERT INTO EMPLOYEE VALUES(12,'MURTHY','MANAGER');  -- GO TO P2
INSERT INTO EMPLOYEE VALUES(13,'PRASUNA','MANAGER');  -- GO TO P2
INSERT INTO EMPLOYEE VALUES(14,'GOVIND','MANAGER');  -- GO TO P2
INSERT INTO EMPLOYEE VALUES(15,'MADHAVI','CLERK');  -- GO TO P4
INSERT INTO EMPLOYEE VALUES(16,'GANESH','HR');  -- GO TO P2
```

Select records from partitioned table:

```
SELECT *FROM EMPLOYEE;
SELECT *FROM EMPLOYEE PARTITION(P1);
SELECT *FROM EMPLOYEE WHERE JOB='MANAGER';
```

Adding a partition:

```
ALTER TABLE EMPLOYEE ADD PARTITION P5 VALUES('ENGINEER','SUPPORT')
```

Dropping a partition:

```
ALTER TABLE EMPLOYEE DROP PARTITION P5;
```

Renaming a partition:

```
ALTER TABLE EMPLOYEE RENAME PARTITION P5 TO P1;
```

Truncate a partition:

```
ALTER TABLE EMPLOYEE TRUNCATE PARTITION P5;
```

Exchanging a partition:

```
ALTER TABLE EMPLOYEE EXCHANGE PARTITION P1 WITH TABLE EMPLOYEE_X;
```

Moving a partition:

```
ALTER TABLE EMPLOYEE MOVE PARTITION P2 TABLESPACE ABC_TBS;
```

PARTITIONING KEY:

Each row in a partitioned table is unambiguously assigned to a single partition. The partitioning key is comprised of one or more columns that determine the partition where each row will be stored.

Q: What is the advantage of partitions, by storing them in different Tablespace's..?

1 Reduces the possibility of data corruption in multiple partitions.
2 Backup and recovery of each partition can be done independently.

♦ **HASH PARTITIONS:**

Hash partitioning maps data to partitions based on a hashing algorithm that Oracle applies to the partitioning key that you identify.

```
CREATE TABLE EMPLOYEE(EMP_NO NUMBER(2),EMP_NAME VARCHAR(2))
PARTITION BY HASH(EMP_NO) PARTITIONS 5;
```

Here oracle automatically gives partition names like

SYS_P1
SYS_P2
SYS_P3
SYS_P4
SYS_P5

Inserting records into hash partitioned table based on hash function:

```
INSERT INTO EMPLOYEE VALUES (5,'RAMANA');
INSERT INTO EMPLOYEE VALUES (8,'GOVIND');
INSERT INTO EMPLOYEE VALUES (14,'GANESH');
INSERT INTO EMPLOYEE VALUES (19,'VISWA');
```

Inserting records into hash partitioned table:

```
SELECT *FROM EMPLOYEE;
SELECT *FROM EMPLOYEE PARTITION (SYS_P2);
```

Adding a partition:

```
ALTER TABLE EMPLOYEE ADD PARTITION P9;
```

Renaming a partition:

```
ALTER TABLE EMPLOYEE RENAME PARTITION P9 TO P10;
```

Truncate a partition:

```
ALTER TABLE EMPLOYEE TRUNCATE PARTITION P9;
```

Exchanging a partition:

```
ALTER TABLE EMPLOYEE EXCHANGE PARTITION SYS_P1 WITH TABLE
EMPLOYEE_X;
```

Moving a partition:

```
ALTER TABLE EMPLOYEE MOVE PARTITION SYS_P1 TABLESPACE RAMANA_TAB;
```

ADVANTAGES OF PARTITIONS:

- Reducing downtime for scheduled maintenance, which allows maintenance operations to be carried out on selected partitions while other partitions are available to users.
- Reducing downtime due to data failure, failure of a particular partition will no way affect other partitions.
- Partition independence allows for concurrent use of the various partitions for various purposes.

CLUSTERS

Cluster we have to create before creating the tables cluster holds common column shared by two tables cluster will be store the matching of two tables data separately in cluster area separately when we are using that column in join conditions no need to go and check the for each record because cluster is already storing mapping data in cluster area. It will improve performance while retrieving or manipulating data from master child tables.

If you create index on cluster it will give better performance because it again that will be divide into two parts.

```
CREATE CLUSTER CLSTR_1(DEPTNO NUMBER(4);
CREATE TABLE DEPT (DEPTNO NUMBER(4),DNAME VARCHAR2(30), LOC
VARCHAR2(15)) CLUSTER CLSTR_1(DEPTNO);
CREATE TABLE EMP (EMPNO NUMBER, ENAME VARCHAR2(30),SAL NUMBER, DEPTNO
NUMBER(4)) CLUSTER CLSTR_1(DEPTNO);
```

DEPT		EMP	
DEPTNO	**ROWID**	**DEPTNO**	**ROWID**
10	ROWID	10	ROWID
10	ROWID	10	ROWID
20	ROWID	20	ROWID
20	ROWID	20	ROWID
20	ROWID	20	ROWID
30	ROWID	30	ROWID
30	ROWID	30	ROWID

COMPOSITE INDEX: If we create one index on two in a table that is called composite index.

```
CREATE INDEX IDX_SAL_COMM ON EMP(SAL,COMM);
```

NOTE: If table having two composite indexes for (X, Y) index_1, for (X, Z) index_2 then if you are writing where condition based on X column then index will work based on which index having less branch blocks, if both indexes having same number of branch blocks then it works based on the number of leaf of blocks.

```
SELECT * FROM DBA_INDEXES WHERE INDEX_NAME IN
('IDX_LOAN_ID','IDX_CUST_NO');
```

- In this it will consider 1st B_LEVEL column, next LEAF_BLOCKS column.
- If one index has more null values it go for another index even it having less branch, leaf blocks. Because for null values index not powerful.

JOINS

Joins are used to get data from different columns and various tables by linking the columns of various tables by using proper join condition. Joins are use full to avoid the cartesian product.

- **SELF JOIN:**
  - `SELECT W.ENAME,M.ENAME FROM EMP W, EMP M WHERE W.MGR=M.EMPNO;`

- **EQUILENT JOIN:**
  - `SELECT E.ENAME,E.JOB,D.DNAME,E.SAL FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO AND D.DNAME IN ('SALES','ACCOUNTING');`

- **NON EQUILENT JOIN:**
  - `SELECT E.ENAME,E.SAL,G.GRADE FROM EMP E, SALGRADE G WHERE E.SAL BETWEEN G.LOSAL AND G.HISAL;`

- **LEFT OUTER JOINS:**
  - `SELECT E.ENAME,E.JOB,D.DNAME FROM EMP E, DEPT D WHERE E.DEPTNO(+)=D.DEPTNO;`
    - It gives all the dnames available in the dept table even that deptno not allotted for any employee.

- **RIGHT OUTER JOINS:**
  - `SELECT E.ENAME,E.JOB,D.DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO(+);`
    - It gives all the ename, jobs available in the emp table even that employee not allotted for any department.
    - **NOTE:** Whatever the join you have used even `LEFT OUTER JOIN (or) RIGHT OUETR JOIN` that brings on which table column you have attached (+) symbol that opposite table column entire data
      Means that statement brings all join condition matched values and also not matched distinct values;

- **FULL OUTER JOINS:**
  - `SELECT E.ENAME,E.JOB,D.DNAME FROM EMP E FULL OUTER JOIN DEPT D ON( E.DEPTNO=D.DEPTNO);`
    - ✓ `FULL OUTER JOIN` will brings join conditions matched data from the tables and also condition not matched distinct values from the both tables;

JOIN METHODS

Oracle internally perform join if you are executing a query. If you are executing some query with join then oracle internal fetch the data with these join methods.

♦ NESTED LOOP JOIN:

IN JOINS TABLES CONSIDER LIKE
LESS DATA TABLE = LEADING TABLE (Ex: DEPT)
MORE DATA TABLE = PROBING TABLE (Ex. EMP)

If you joining two tables and that probing table column having more data than leading table data and probing table column has selective index then it goes for nested loop join.

```
SELECT * FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO
```

♦ SORT MERGE JOIN:

When joining two tables and both the join columns have indexes in both tables and the join condition is comparison (>, <) then it is go for sort merge join.

NOTE: In this kind situations sort merge join always follows by comparison (>, <) only. It does not work in equijoin.

```
SELECT * FROM EMP1 A, EMP2 B WHERE A.SAL>B.SAL;
```

♦ HASH JOIN:

When you're joining two tables and both the joining columns has nearly equal quantity of data. Then it goes for hash join. Even both the join conditioned column having index/ doesn't have index its go for hash join only.

If you want to decrease the execution time then you should go for filter the data by using any column and that filter condition column should have index.

```
SELECT * FROM EMP1 A, EMP2 B WHERE A.EMPNO=B.EMPNO AND
A.SAL>100
```

NORMALIZATION:

Normalization is the process of combining various individual modules into a single process by using data integrity, main advantage of normalization avoiding data redundancy.

EX: Employee data, employee previous experience data. When one employee having previous experience in five companies, if we want to write both the information's in one table then we should write that employee information with each previous experience, to avoid that we create another table for previous experiences and put some link between those tables using data integrity that process is called normalization.

Note: Data integrity nothing but integrity constraint (foreign key);

OPTIMIZATION:

Optimization nothing but execution of the query and it has two methods.

1. Rule based
2. Cost based

♦ RULE BASED:

If we write a query for searching the data which column you have used to put filter condition that column has index. Then rule based optimization blindly go and use the index for statement execution. If that index not a selective index and that is not giving good performance then also it's go and use that index for query execution. Because rule based optimization follows rule only.

♦ COST BASED:

In this condition before execution system will check cost, cardinality and the column having selective index or not then it's execute the plan. If system having selective index then its go and use the index, if the index is not a selective index go for index skip scan means it will go to table access full.

Optimization will plan the execution of a query like which database file, which extend, which join method, which back and which index scan we have to use for execution. Once you run any query it's automatically generates a 'plan id'. Next you run the same query it's brings the output lesser time then previous time by using that 'plan id'.

HINT:

Hint will pass a message how the query should be execute.

HINT: `/*+IDX_DEPTNO(DEPTNO)*/`

```
SELECT /*+IDX_DEPTNO(DEPTNO)*/ * FROM EMP WHERE DEPTNO=10;
```

PARLELL HINT:

When you are searching the data if you are going for the parallel hint then it will divide the data into pieces based on what are degrees you have given. Then it will search the data output we can get in fast.

```
SELECT EMPNO /*+(PARALLEL 40)*/ , ENAME, SAL FROM EMP WHERE
EMPNO=7369;
```

In this we are given the degree is 40

Then system will separate blocks into 40% of blocks for one processor, 40% of blocks for one processor and 20% of blocks for another processor and then system go and search the data and output we can get in fast.

QUERY TUNING/EXECUTION PLAN:

There are 3 methods to get this plan

1. Select the statement and press (F5).
2. Process is
   - ↳ Select the statement
   - ↳ Tools
   - ↳ Explain plan.
3. Process is
   - ↳
```
EXPLAIN PLAN SET STATEMENT_ID='R' FOR
SELECT E.ENAME,E.HIREDATE,D.DNAME FROM EMP E, DEPT D WHERE
E.DEPTNO=D.DEPTNO;
```
   - ↳
```
SELECT * FROM PLAN_TABLE WHERE STATEMENT_ID='R';
```

Cost: No. of hits to the database (means how many times it goes to database to bring the data).
Cardinality: How many records it's searched.
Bytes: How much of area it's searching for the data.

JULIET FORMAT:

If you want to write a number in letters then you should go for Juliet format.

- ➢
```
SELECT TO_CHAR(TO_DATE(100000,'J'),'JSP') FROM DUAL;
```

A) ----ONE HUNDRED THOUSAND.

CONSTRAINTS

If you want to put any rule or restriction on a column you should go for constraint. We can't see and modify constraints, those are inbuilt and encrypted. Constraints we can write on columns only.

Constraints are 4 types:

- Unique key constraint
- Primary key constraint
- Check constraint
- Foreign key / Reference constraint
  - ✓ Normal foreign key
  - ✓ On delete cascade
  - ✓ On delete set null

♦ UNIQUE KEY CONSTRAINT:

➢ `ALTER TABLE EMP ADD CONSTRAINT UK_EMPNO UNIQUE (EMPNO)`

- It provides restriction on inserting duplicates values into the corresponding column.
- We can create any number of unique key constraints on a table
- Unique key allows any number of null values into the corresponding column
- At the time of creation of unique key constraint automatically unique index will be create.

♦ PRIMARY KEY CONSTRAINT:

➢ `ALTER TABLE EMP ADD CONSTRAINT PK_EMPNO PRIMARY KEY (EMPNO)`

- It provides restriction on inserting duplicates and null values into the corresponding column
- We can able to create one number of primary key constraint on a table.
- At the time of creation of primary key constraint automatically unique index will be create.

♦ CHECK CONSTRAINT:

➢ `ALTER TABLE EMP ADD CONSTRAINT CK_DEPTNO CHECK (DEPTNO IN (10,20,30));`

- It is user friendly constraint
- It is works under the condition what we give
- It is accepts null values

♦ FOREIGN KEY / REFERENCE CONSTRAINT:

➢ CREATION STEPS:

↳ `ALTER TABLE DEPT ADD CONSTRAINT PK_DEPTNO PRIMARY KEY (DEPTNO);`

↳ `ALTER TABLE EMP ADD CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO);`

- Foreign key will accept null, duplicate values
- If you want to create a foreign key then reference table column should be primary key.
- Foreign key won't create any index automatically.

♦ FOREIGN KEY / REFERENCE KEY TYPES:

✓ Normal foreign key:

- It does not allows to delete data from the parent table if the corresponding child records existing in the child table.

```
ALTER TABLE EMP ADD CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO)
REFERENCES DEPT(DEPTNO);
```

```
DROP TABLE DEPT CASCADE CONSTRAINT;
```

✓ Foreign key on delete cascade;

- It allows delete data in the parent table.
- But if we delete records in the parent table then automatically corresponding child records data will be deleted in child table.

```
ALTER TABLE EMP ADD CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO)
REFERENCES DEPT(DEPTNO)ON DELETE CASCADE;
```

✓ Foreign key on delete set null:

- It is allows delete data in the parent table.
- But if you delete the data in the parent table then all the corresponding child records will become null.

```
ALTER TABLE EMP ADD CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO)
REFERENCES DEPT(DEPTNO)ON DELETE SET NULL;
```

♦ COMPOSITE CONSTRAINT:

```
ALTER TABLE EMP ADD CONSTRAINT UK_EMP UNIQUE (EMPNO,ENAME);
```

When we create constraint on more than one column is called composite constraint. If you create composite constraint on two columns it compares the values from both columns. It combines both columns into single column and works on it.

- Composite unique key
- Composite primary key
- Composite check
- Composite foreign

Composite unique key:

ALLOWS		DOESN'T ALLOW	
EMPNO	ENAME	EMPNO	ENAME
103	RAMANA	103	RAMANA
104	PRASUNA	104	PRASUNA
104	LUCKY	104	LUCKY
103	MURTHY	103	RAMANA
105	GOVINDH	104	PRASUNA
105	NULL	105	LUCKY

Means it is convert that two columns into a single column and won't allow duplicate values on it. Composite primary key also works like composite unique key but won't allow null values.

TO DISABLE A CONSTRAINT:

Method_1:

- Right click on table name
- Go to edit
- Go for key window
- Click on the enable button.

Method_2:

```
ALTER TABLE EMP DISABLE CONSTRAINT PK_EMPNO;
ALTER TABLE EMP ENABLE CONSTRAINT PK_EMPNO;
```

- `SELECT * FROM USER_CONSTRAINTS;`
  - To know the list of constraints in the schema
- `SELECT * FROM USER_CONS_COLUMNS`
  - To know the list of tables, columns which are having constraints
- `SELECT * FROM USER_CONS_COLUMNS T, USER_CONSTRAINTS C WHERE C.CONSTRAINT_NAME=T.CONSTRAINT_NAME AND C.CONSTRAINT_NAME NOT LIKE 'BIN$%'`
  - To constraint type, column and table name also then you should join both the tables (**USER_CONSTRAINTS and USER_CONS_COLUMNS**);

CASE:

- Case is statement and it is used to provide if, then, else type of logic
- Case we can use inside the select statement and with select statement also we can use
- It supports operators like (>, < ,>=, =<, =, in, between, like, not like)
- By using case we can directly assign a value to variable in PL/SQL block
  - `SELECT ENAME, SAL, CASE WHEN DEPTNO=10 THEN 'SOFTWARE' WHEN DEPTNO=20 THEN 'ADMIN' ELSE 'DBA' END JOB FROM EMP;`

DECODE:

- Decode is function and it is used to provide if, then, else type of logic
- Inside select statement only we can use decode. Without select statement we cannot use decode
- It does not supports operators like (>, < ,>=, =<, =, in, between, like, not like)
  - `SELECT ENAME,SAL,DECODE(DEPTNO,10,'SOFWARE',20,'ADMIN','DBA') JOB FROM EMP;`

VIEWS

When we use same select statement repeatedly then better to create a view on the query. And also it helps to hide the complicity of the query.

- ✓ View is a stored select statement. It won't store the data. It stores only the query of the statement only, but it allows changes like update, delete.
- ✓ If we make any changes in the view automatically it updates in the main table.
- ✓ It won't occupy any space. Means it won't store the data physically it will stores only select statement query permanently in the database.
- ✓ If the corresponding tables dropped then view become invalid.
- ✓ `SELECT * FROM USER_VIEWS`

❖ **PREDEFINED VIEWS:**

- ALL_SOURCE;
- ALL_WM_CONSTRAINTS;
- ALL_IND_COLUMNS;
- ALL_IND_PARTITIONS;

♦ **SIMPLE VIEW:**

➢ `CREATE VIEW VW_EMP AS SELECT EMPNO,ENAME,JOB,SAL FROM EMP;`

- If you create a view on only one table is called simple view
- It allow DML commands

♦ **COMPLEX VIEW:**

➢ `CREATE VIEW VW_EMP_DTLS AS SELECT E.ENAME,E.SAL,E.JOB,D.DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;`

- If you create a view on more than one table is called complex view.
- It does not allow DML operations.

NOTE: if you want to do DML operations on complex view then you should go for instead of trigger.

♦ **READ ONLY VIEW:**

- If we create a view by using arithmetic, aggregate or char functions these views are called read only view
- It won't allow DML commands
- Those views you cannot modify. If we modifying the view data where it will be stored..?

NOTE: If you add read only command for simple at the time of creating, it becomes read only view.

♦ **MATERIALIZED VIEW:**

It is a stored select statement as well as it will store the data of the select statement. Means it will store both select statement and data. It is a data base object and it contains result of the query. Whatever changes you have done on this view should not impact on the table but whatever changes you have done on the table it may or may not impact on the materialized view it depends on method of materialized view. Materialized view has four methods.

✓ **BASIC MATERIALIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_NAME AS SELECT ENAME, DNAME FROM EMP E,DEPT D WHERE E.DEPTNO=D.DEPTNO;`

In basic materialized view at the time of creation that view stores the data permanently after creation whatever the changes you have done in the table that's not effects on the view.

That view data individual and you able to do DML operations on the view also.

✓ **REFRESH FAST ON COMMIT MATERIALIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_EMP REFRESH FAST ON COMMIT AS SELECT * FROM EMP`

This materialized view needs

- Primary key and
- Log (`create materialized view log on emp`)

Whatever the changes you have done in the table won't effect on the view until you give the commit. If you apply the commit after making some changes in the table then immediately the view data also modify automatically.

- The log store the script of changes what you have apply commit after until you apply the commit after the commit the log will apply the same changes on the view.
- That primary key helps to store the changes order wise in log .That's why both log and primary key are compulsory for refresh fast on commit method of materialized view.
- That select statement should be containing primary key column. It won't allow join we can able to create this view on single table only.

✓ **FORCE MATERIALIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_REFRESH BUILD IMMEDIATE AS SELECT ENAME, DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;`

In this materialized view changes you have done in the table not effects on the view. Until you have done a separate action for that. If you want to do same changes on view also then you should go for this.

```
BEGIN
 DBMS_MVIEW.refresh('MV_GRADE','FORCE');
END;
```

After you have done this forcefully view data also updates exactly like table data, until that view data in same position.

✓ **INTERVAL MATERAILIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_GRADE REFRESH START WITH SYSDATE NEXT (SYSDATE+1) AS SELECT EMPNO,ENAME,DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;`

In this view data will refresh with uniform interval which you have given. Means in this view what you have done the changes in the table automatically updates in their selves for every interval. There is no need of any separate action for data refreshing view. That's why this is called interval materialized view.

To see all materialized views: `SELECT * FROM USER_MVIEWS;`

SYNONYMS

The main advantage of synonym is give one object permission to another user. But when you are giving the permission you have to write what kind of permissions you want give for user you should mention.

- **WITHOUT PUBLIC SYNONYM:**

USER(**ARMURTHY**):

```
↳ GRANT SELECT ON BRANCH_INFO TO RAMANA143;
↳ GRANT ALL ON STUDENT_INFO TO RAMANA143;
```

  - Then if user 'RAMANA143' want to use the table then he should ask permission by mentioning user 'ARMURTHY' name before the object name in the query every time. Because it is not permission he given permission the table.

USER(**RAMANA143**):

```
↳ SELECT * FROM ARMURTHY.BRANCH_INFO;
↳ INSERT INTO ARMURTHY.STUDENT_INFO;
```

- **WITH PUBLIC SYNONYM:**

USER(**ARMURTHY**):

```
↳ CREATE PUBLIC SYNONYM SYN_STUDENT_DATA FOR STUDENT_DATA;
↳ CREATE PUBLIC SYNONYM SYN_SYN_SP_COMM_CAL;
↳ GRANT INSERT, UPDATE ON SYN_STUDENT_DATA TO RAMANA143
↳ GRANT ALL ON SYN_ BRANCH_INFO TO RAMANA143;
↳ GRANT EXECUTE ON SYN_SP_COMM_CAL;
```

  - If you create a public synonym on the table there is no need of mention user name every time and that will stored as synonym.

USER(**RAMANA143**):

```
↳ SELECT * FROM SYN_STUDENT_DATA;
```

Synonym is nothing but it is alias name of the original object whatever changes you done at original object it will be impact on the synonym whatever changes your doing in synonym it will impact on the original object.

- **PRIVATE SYNONYM:**

```
↳ CREATE SYNONYM SYN_EMP FOR EMP;
```

  - This synonym only useful same user. The only use of private synonym is data security. Because it gives only insert, select permission for the table.

NOTE: If you want to revoke the permissions you had given to the users.

```
↳ REVOKE ALL ON SYN_STUDENT_DATA FROM RAMANA143;
↳ REVOKE INSERT,UPDATE ON SYN_BRANCH_INFO FROM RAMANA143;
```

↳ If you drop the table synonym will be invalid, but if you recreate the table automatically synonym will be valid there is no need to compile the query again.

↳ Private Synonyms: `SELECT * FROM USER_SYNONYMS;`

↳ Public Synonyms : `SELECT * FROM ALL_SYNONYMS;`

TYPES OF QUERIES

THERE ARE FOUR TYPES OF QUERIES:

- Sub query
- Co-related query
- In line query
- Scalar query

♦ **SUB QUERY:**

After where clause if we write any query as filter condition is called "sub query".

```
SELECT * FROM EMP WHERE SAL = (SELECT MAX(SAL) FROM EMP);
SELECT * FROM EMP WHERE (DEPTNO,SAL) IN (SELECT DEPTNO,MAX(SAL)
FROM EMP GROUP BY DEPTNO);
```

In sub query 1st inner query run and bring some result by using that result outer query will run and gives the output.

♦ **CO-RELATED SUB QUERY:**

If you write outer and inner queries co-related (means inner query result based on outer query result and outer query output based on inner query result). And the total query output is comparison of both inner and outer queries called "co-related sub queries".

```
SELECT * FROM EMP E WHERE E.SAL=
(SELECT MAX(SAL) FROM EMP D WHERE E.DEPTNO=D.DEPTNO AND
E.DEPTNO IN (10,30));
```

↳ This query give's department wise highest salaries.

Note: In sub query 1st inner query runs and gives some data based on the result outer query run and give the output.

But in co-related sub query 1st outer query run and give some result by using that result inner query run and gives some result then using that result outer query will give the output.

Here one query depends on another query that's why this called co-related sub query.

Sub Query:

```
SELECT * FROM EMP E WHERE (E.DEPTNO,E.SAL) IN (SELECT
DEPTNO, MAX(SAL) FROM EMP GROUP BY DEPTNO)
```

Co –related sub query:

```
SELECT * FROM EMP E WHERE E.SAL=(SELECT MAX(SAL) FROM EMP D
WHERE E.DEPTNO=D.DEPTNO);
```

These both queries will give same results department wise highest salaries.

- **IN LINE QUERY:**

  After from clause instead of table name if you write a query and gives alias name for that is called "In line query".

  - ```
    SELECT * FROM EMP E, (SELECT MAX(SAL) SAL FROM EMP UNION SELECT
    MIN(SAL) FROM EMP)Z WHERE E.SAL=Z.SAL;
    ```
    - ↳ Give's highest, least salary employees.

  - ```
    SELECT * FROM (SELECT X.*,ROWNUM R FROM (SELECT E.* FROM EMP E
    ORDER BY SAL DESC) X) Z WHERE Z.R=2;
    ```
    - ↳ Give's 2nd highest salary employee.

- **SCALAR QUERY:**

  Generally in between select and from clause always we have to write the column name instead of column name if we write query to find any column that is called scalar query.

  - ```
    SELECT EMPNO, ENAME, SAL, (SELECT DNAME FROM DEPT D WHERE
    D.DEPTNO=E.DEPTNO) DNAME FROM EMP E;
    ```

LEVEL:

↳ To print 1-10 number:

```
SELECT LEVEL FROM DUAL CONNECT BY LEVEL<=10;
```

↳ To print each letter in a separate row from the given string:

```
SELECT SUBSTR('PINEAPPLE',LEVEL,1) FROM DUAL CONNECT BY
LEVEL<=(SELECT LENGTH('PINEAPPLE') FROM DUAL);
```

↳ To print every month first day, every month last day:

```
SELECT ADD_MONTHS(TRUNC(SYSDATE,'YYYY'),(LEVEL-1)) MONTH_FIRST,
LAST_DAY(ADD_MONTHS(TRUNC(SYSDATE,'YYYY'),(LEVEL-1))) MONTH_LAST
FROM DUAL CONNECT BY LEVEL<=12;
```

↳ To print all Saturdays, Sundays in Year:

```
SELECT (NEXT_DAY(TRUNC(SYSDATE,'YYYY'),'SAT')-7)+LEVEL*7
SATUR_DAY_2020,(NEXT_DAY(TRUNC(SYSDATE,'YYYY'),'SUN')-7)+LEVEL*7
SUNDAY_2020 FROM DUAL CONNECT BY LEVEL<=52;
```
```

REGULAR EXPRESSIONS

- **REGEXP_COUNT**
  - ```
    SELECT REGEXP_COUNT('GOOD MORNING','O') FROM DUAL;
    ```
    ----3 -(gives count of 'o' in 'good morning');
  - ```
    SELECT REGEXP_COUNT(ENAME,'A') FROM EMP;
    ```
    -----(gives count of 'a' in each ename (each row));

- **REGEXP_REPLACE**
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA MURTHY@143','\D') FROM DUAL;
    ```
    ---'143' -(only for digits);
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA MURTHY@143','\d') FROM DUAL;
    ```
    --'$RAMANA MURTHY@' -(only for characters and special characters);
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA  MURTHY@143','\S') FROM DUAL;
    ```
    ---' '-(only for space);
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA MURTHY@143','\s') FROM DUAL;
    ```
    ---'$RAMANAMURTHY@' -(for without space / remove space);
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA MURTHY@143','\W') FROM DUAL;
    ```
    ---'RAMANAMURTHY143' -(for remove special characters);
  - ```
    SELECT  REGEXP_REPLACE('$RAMANA  MURTHY@143','\w') FROM DUAL;
    ```
    ---'$ @' -(only for special character);

- **REGEXP_LIKE**
  - To print all enames which as 'ad' in their self:
    ```
    SELECT * FROM EMP WHERE REGEXP_LIKE(ENAME,'AD');
    ```
  - To print all enames which has 'KI' in their self and enames which has 'ES' in their self;
    ```
    SELECT * FROM EMP WHERE REGEXP_LIKE(ENAME,'KI|ES');
    ```

- **REGEXP_SUBSTR**
  - ```
    SELECT REGEXP_SUBSTR(P_DELIVERY_IDS,'[^,]+',1,ROWNUM) BULK COLLECT
    INTO TYP_ID FROM DUAL CONNECT BY ROWNUM<=((LENGTH(P_DELIVERY_IDS)-
    LENGTH (REPLACE(P_DELIVERY_IDS,',','')))+1);
    ```
    - When client send the records as comma separated values then you want covert into column rows data then it is use full.

Ex:
```
SELECT REGEXP_SUBSTR('1100,1101,1103,1104,1105,1106','[^,]+',1,ROWNUM)
FROM DUAL CONNECT BY ROWNUM<=((LENGTH('1100,1101,1103,1104,1105,1106')
- LENGTH(REPLACE('1100,1101,1103,1104,1105,1106',',','')))+1);
```

- It's works like exactly opposite direction of WM_CONCAT.
- WM_CONCAT covert a column rows into single row as comma separated values. REGEXP_SUBSTR convert comma separated records into column rows.
```

SQL LOADER

It is used to export OS data into database. It will be export the data from OS in four steps. One control file required for this, control file is nothing but notepad text document that file should save as all files type only. And that OS file should save as CSV format only.

- **Create table in SQL.**
  Ex:
```
CREATE TABLE STUDENT_DATA (ROLL_NO NUMBER,STUDENT_NAME
VARCHAR2(30),MARKS NUMBER);
```

- **Save data in excel sheet as CSV format only.**
  Ex: File name : STUDENT_DTLS.
  Save as type: CSV (Comma delimited).

- **Write your aim in control file (note pad) and save it in all files format only.**
  Ex: LOAD DATA INFILE 'D:\STUDENT _DTLS.CSV' INTO TABLE STUDENT_DATA FILEDS TERMINATED BY "," (ROLL_NO, STUDENT_NAME, MARKS).

  File name : STUDENT.CTL
  Save as type: All files.

- **Open command prompt and write concept here.**
  Ex: SQLLDR armurthy/acr143@ORCL CONTROLL=D:\ STUDENT.CTL;
  (Here : armurthy - username , acr143 - password)

  If you want import data from OS then your data table should be empty but if want add another table data same table you must use 'APPEND' command.

  Ex: LOAD DATA INFILE 'D:\STUDENT _DTLS2.CSV' APPEND INTO TABLE STUDENT_DATA FILEDS TERMINATED BY "," (ROLL_NO, STUDENT_NAME, MARKS).

  If you want to store new data in same table by removing old data then you should write 'REPLACE' in 'APPEND' place. If you want to store data from 3rd or 4th row that means you want to skip 1st three rows then you should go for 'SKIP' key word.

  Ex: OPTIONS (SKIP=3) LOAD DATA INFILE 'D:\STUDENT _DTLS2.CSV' REPLACE INTO TABLE STUDENT_DATA FILEDS TERMINATED BY "," (ROLL_NO, STUDENT_NAME, MARKS).

ORACLE LOADER

This is used to export data from OS to database by using external table. After loading the data in external table we can move that data into database table easily.

```
CREATE TABLE EXT_STUDENT_DATA(ROLL_NO NUMBER,
  STUDENT_NAME VARCHAR2 (30), MARKS NUMBER)
ORGANIZATION EXTERNAL
(
TYPE ORACLE_LOADER
DEFAULT DIRECTORY DATA_PUMP_DIR
ACCESS PARAMETERS
(   RECORDS DELIMITED BY NEWLINE SKIP 0
  NOBADFILE NODISCARDFILE NOLOGFILE
      FIELDS TERMINATED BY ','
 MISSING FIELD VALUES ARE NULL
    )  LOCATION
('STUDENT_DTLS.CSV') )
 REJECT LIMIT UNLIMITED;
```

Note: That the OS file should be placed in the DATA_PUMP_DIR location. That DATA_PUMP_DIR is the location where you are installed the oracle.

After loading the data the data will be at external table we are not able to do any DML operations on that. Then if you want to do any changes in that data should do it in that OS file, if you are done change in that OS file it's automatically updates in the external table. Or you can create table like this external table in the database.

Ex: `CREATE TABLE STUDENT_DATA AS SELECT * FROM EXT_STUDENT_DATA;`

If you want to write any conditions for loading you should write in before

```
NOBADFILE NODISCARDFILE NOLOGFILE
```

Ex: `LOAD WHEN MARKS >450;`

```
BADFILE 'STUDENT_1.BAD'
DISCARDDILE 'STUDENT_2.DISC'
LOGFILE 'STUDENT_3.LOG'
```

BADFILE: Condition satisfied but error occurred by data type, data size this kind of data stored in this.
DISCARDDILE: Condition not satisfied data stored.
LOGFILE : This is the total script of oracle loader running base.

IMPORT and EXPORT:

This import and export concept is used transfer the data from one data base to another data base.

To export data

- Open command prompt.
- EXP armurthy/acr143@ORCl file=D:\RAMANA_EXP.DMP OWNER=armurthy statistics=none.

Statistics means if anyone doing any action in that database not consider that one.

To import data

- Open command prompt.
- IMP ramana143/ramana143@ORCL file=D:\RAMANA_EXP.DMP FROMUSER=ARMURTHY TOUSER=RAMANA143.

Whatever the file you have exported that the file can be exported in binary file format for data security purpose you can read this data after importing in a another schema you want.

In real time mostly FTP (file transfer protocol) services can use for this import/export for data security purpose because Gmail is not secured.

To export all / selected objects in the schema:

- Tools
- Export user objects
- Then select the objects in displayed objects which you want to export and then select the location and type a name for your file @Output file
- Click on export.

- Then that user can open the text document and select entire scripts paste in SQL command window then automatically all the objects will be created in that schema also.

(OR)

Open the SQL command window and write the text document name

```
SQL> @'D:\oracle\murthy_exp.sql';
```

Then automatically all the objects will be created. Every time you want to run the script you should do like this.

Note: If you want to remove the schema name everywhere from the script you can replace with null by using (CTRL+F).

↳ But in projects all the scripts we are already maintain script in SVN portal, So we can send that script to client like:

```
SET DEFINE OFF
SPOOL DEC_2018_SCRIPT.LOG
CREATE TABLE BULK_EXP_ERRORS
(
MODULE_NAME VARCHAR2 (15),
ERROR_DATE  DATE,
ERROR_NO    VARCHAR2 (10),
ERROR_MSG   VARCHAR2 (512),
ROW_NUMBER  NUMBER
)
TABLESPACE TAB_RAMANA;

CREATE OR REPLACE PROCEDURE SP_BULK_ERROR (P_MODULE VARCHAR2,
P_ERROR_NO VARCHAR2,P_ERROR_MSG VARCHAR2, P_ROW_NO NUMBER)
AS
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
INSERT INTO BULK_EXP_ERRORS VALUES (P_MODULE, SYSDATE,
P_ERROR_NO, P_ERROR_MSG, P_ROW_NO);
COMMIT;
END;
/
CREATE OR REPLACE PROCEDURE SP_FOR_UPDATE
AS
CURSOR C1 IS SELECT * FROM EMP FOR UPDATE;
VEC EMP%ROWTYPE;
BEGIN
OPEN C1;
     LOOP
          FETCH C1 INTO VEC;
     EXIT WHEN C1%NOTFOUND;
UPDATE EMP SET COMM=SAL*10/100 WHERE CURRENT OF C1;
END LOOP;
CLOSE C1;
END;
/
SPOOL OFF
```

We should save this file in SQL format and send to client.

Here:

1. '`SET DEFINE OFF`' means while we running this script is there any others going put them in hold.
2. '**`SPOOL DEC_2018_SCRIPT.LOG`**' will create one log file for execution of the script if any object not created that will be exists in this log file with error number message.

TO CAMPARE TWO SCHEMAS:

- Tools
- Compare user objects
- Target session (user: armurthy, password: acr143)
- Compare (select some objects or process all objects)
- If no difference between both the schemas no issue.
  If any differences are there and we want to see the differences in detail for that we have slides button at right bottom corner. Click on the slides open both schema in separate slides and the differences also visible as highlight.

TO SEND ONE TABLE DATA IN TEXT DOCUMENT:

- Open table data like '`SELECT * FROM EMP`'.
- Now click on the left top corner then rows and columns will be selected.
- Then right click on the table choose export results option in the menu again select CSV option in the opened menu.
- Then provide name and location for the document then automatically entire table data will be store in text document.
- If you want all the rows with insert statements means SQL script of all rows then should select SQL option in export results menu. Then entire table data script will be save text document. Now you can select script and open SQL command wind in which schema you want to insert the data paste the script in SQL window automatically all the data will be inserted in the table.
  Note: That same table should be present in that schema also.

DB LINK

When two users in same database want share data from one user to another user we have public synonyms. But one user want to share for another user in different database then only option we have that is DB link.

Ex:

```
CREATE DATABASE LINK DLK CONNECT TO armurthy IDENTIFIED BY acr143
USING '(DESCRIPTION =(ADDRESS = (PROTOCOL = TCP)(HOST =
localhost)(PORT =1521))(CONNECT_DATA =(SERVER = DEDICATED)
(SERVICE_NAME = ACR)))';
```

```
SELECT * FROM EMP@DLK;
```

FOR DATABASE ADDRESS:

- Open my computer.
- Go for which drive you installed oracle.
- Type "tnsnames.ora" in search.
- "tnsnames.ora" folder will be open.
- Database address placed in it by default installing oracle in system.

SQL - QUERIES

1. DISPLAY INFORMATION FROM DEPT TABLE:
   SELECT * FROM **DEPT**

2. DISPLAY THE DETAILS OF ALL EMPLOYEES:
   SELECT * FROM **EMP**

3. DISPLAY NAME, JOB FOR ALL EMPLOYEES:
   SELECT **ENAME**, **JOB** FROM **EMP**

4. DISPLAY NAME,SAL FOR ALL EMPLOYEES:
   SELECT **ENAME**, **SAL** FROM **EMP**

5. DISPLAY EMPNO, TOTAL SAL OF ALL EMPLOYEES:
   SELECT **EMPNO**, **SAL**+NVL (**COMM**, 0) **TOTAL_SAL** FROM **EMP**

6. DISPLAY NAME, ANNUAL SALARY OF ALL EMPLOYEES:
   SELECT **ENAME**, **SAL***12 **ANNUAL_SAL** FROM **EMP**

7. DISPLAY THE NAMES OF ALL EMPLOYEES WHO ARE WORKING IN DEPARTMENT NO-10:
   SELECT **E.ENAME** FROM **EMP E**, **DEPT D** WHERE **E.DEPTNO**=10
   AND **E.DEPTNO**=**D.DEPTNO**

8. DISPLAY NAMES OF ALL EMPLOYEE WORKINGS AS CLERK AND DRAWING AND SALARY MORE THAN 3000:
   SELECT **ENAME** FROM **EMP** WHERE **JOB** IN ('CLERK', 'DRAWING') AND **SAL**>1000

9. DISPLAY EMPNO, NAMES OF EMPLOYEES WHO EARNS COMMISION:
   SELECT **ENAME**, **COMM** FROM **EMP** WHERE **COMM** IS NOT NULL
   AND **COMM** NOT IN (0)

10. DISPLAY NAME OF EMPLOYEE WHO DO NOT EARN ANY COMMISION:
    SELECT **ENAME** FROM **EMP** WHERE **COMM** IS NULL OR **COMM** IN (0)

11. DISPLAY THE NAMES OF EMPLOYEES WHO ARE WORKING AS CLERK, SALESMAN, ANYLIST, DRAWING AND SALARY MORE THAN 300:
    SELECT **ENAME** FROM **EMP** WHERE **JOB** IN
    ('CLERK','SALESMAN','ANALYST','DRAWING') AND **SAL**>1000;

12. DISPLAY THE NAMES OF EMPLOYEE WHO ARE WORKING IN THE COMPANY FROM FAST FIVE YEARS:
    SELECT **ENAME**, **HIREDATE** FROM **EMP** WHERE ((SYSDATE-**HIREDATE**)/365)>5

13. DISPLAY CURRENT DATE:
    SELECT SYSDATE FROM **DUAL**

14. DISPLAY THE LIST OF EMPLOYEES WHO HAVE JOINED COMPANY BEFORE 30-JUN-90 OR AFTER 31-DEC-90:
SELECT **ENAME**, **HIREDATE** FROM **EMP** WHERE ((SYSDATE-**HIREDATE**)/365)<5

15. DISPLAY THE LIST OF USERS IN YOUR DATABASE:
SELECT * FROM **ALL_USERS**

16. DISPLAY NAMES OF ALL TABLES FROM CURRENT USER:
SELECT * FROM **TABS**

17. DISPLAY THE NAME OF CURRENT USER:
SELECT USER FROM **DUAL**

18. DISPLAY THE NAMES OF EMPLOYEES WORKING IN THE DEPARTMENT N0.10,20,40 EMPLOYEES WORKING AS CLERKS, SALESMAN OR ANALYST:
SELECT **ENAME**, **JOB**, **DEPTNO** FROM **EMP** WHERE **JOB** IN ('ANALYST', 'SALESMAN', 'CLERK') AND **DEPTNO** IN (10, 30, 40)-18

19. DISPLAY EMPLOYEE NAMES FOR WHOSE NAME ENDS WITH ALPHABET S:
SELECT **ENAME** FROM **EMP** WHERE **ENAME** LIKE '%S'

20. DISPLAY THE NAMES FOR EMPLOYEES WHO'S NAMES HAVE SECOND ALPHABET A IN THEIR NAMES:
SELECT **ENAME** FROM **EMP** WHERE **ENAME** LIKE '_A%'

21. DISPLAY THE NAMES OF EMPLOYEES WHOSE NAMES IS EXACTLY FIVE CHARACTER IN LENGTH:
SELECT **ENAME** FROM **EMP** WHERE LENGTH (**ENAME**)=5

22. DISPLAY THE NAMES OF EMPLOYEES WHO ARE NOT WORKING AS MANAGER:
SELECT **ENAME**, **JOB** FROM **EMP** WHERE **JOB** NOT IN ('MANAGER')
------OR-----
SELECT **ENAME**, **JOB** FROM **EMP** WHERE **EMPNO** NOT IN (**MGR**)

23. DISPLAY THE NAME OF EMPLOYEES WHO ARE NOT WORKING AS SALESMAN OR CLERK OR ANALYST:
SELECT **ENAME**, **JOB** FROM **EMP** WHERE **JOB**
NOT IN ('ANALYST','CLERK','SALESMAN')

24. DISPLAY ALL ROWS FROM EMP TABLE. THE SYSTEM SHOULD WAIT AFTER EVERY SCREEN FULL OF INFORMATION:
SELECT * FROM **EMP** (**REMAING FRONT_END** USER **JOB**)

25. DISPLAY THE TOTAL NUMBER OF EMPLOYEES WORKING IN THE COMPANY:
SELECT COUNT(*) FROM **EMP**

26. DISPLAY THE TOTAL SALARY BEING PAID TO ALL EMPLOYEES:
SELECT SUM (**SAL**+NVL (**COMM**,0))FROM **EMP**

27. DISPLAY THE MAXIMUM SALARY FROM EMP TABLE:
SELECT MAX (**SAL**) FROM **EMP**

28. DISPLAY THE MINIMUM SALARY FROM EMP TABLE:
SELECT MIN (**SAL**) FROM **EMP**

29. DISPLAY THE TOTAL SALARY BEING PAID TO CLERK:
SELECT MAX(**SAL**) FROM **EMP** WHERE **JOB**='CLERK'

31. DISPLAY THE MAXIMUM SALARY BEING PAID IN THE DEPARTMENT NO.20:
SELECT MAX (**SAL**) FROM **EMP** WHERE **DEPTNO**=20

32. DISPLAY THE MIN SAL BEING PAID TO ANY SALESMAN:
SELECT MIN (**SAL**) FROM **EMP** WHERE **JOB**='SALESMAN'

33. DISPLAY THE AVG SALARY DRAWN BY MANAGERS:
SELECT TRUNC (AVG(**SAL**)) FROM **EMP** WHERE **JOB**='MANAGER'

34. DISPLAY THE TOTAL SALARY DRAWN BY ANALYST WORKING IN DEPARTMENT NO.20:
SELECT **SAL**+NVL (**COMM**,0) **TOTAL_SAL** FROM **EMP** WHERE **DEPTNO**=20 AND **JOB**='ANALYST'

35. DISPLAY THE NAMES OF THE EMPLOYEES IN ORDER OF SALARY i.e THE NAME OF THE EMPLOYEE EARNING LOWEST SALARY SHOULD APPEAR FIRST:
SELECT **ENAME**, **SAL** FROM **EMP** ORDER BY **SAL**

36. DISPLAY THE NAMES OF EMPLOYEES IN DESCENDING ORDER OF SALARY:
SELECT **ENAME**,**DEPTNO**, **SAL** FROM **EMP** ORDER BY **SAL** DESC

37. DISPLAY THE DETAILS FROM EMP TABLE IN ORDER OF EMPLOYEE NAME:
SELECT **ENAME**, **SAL** FROM **EMP** ORDER BY **ENAME**

38. DISPLAY THE EMPNO, ENAME, DEPTNO AND SAL. SORT THE OUTPUT FIRST BASED ON NAME AND WITHIN NAME BY DEPTNO AND WITHIN DEPTNO BY SALARY:
SELECT **EMPNO**, **ENAME**, **DEPTNO**, **SAL** FROM **EMP** ORDER BY **ENAME**, **DEPTNO**, **SAL**

39. DISPLAY THE NAME OF THE EMPLOYEE ALONG WITH ANNUAL SALARY. THE NAME OF THE EMPLOYEE EARNING HIGHEST ANNUAL SALARY SHOULD APPEAR FIRST:
SELECT **ENAME**, **SAL***12 **ANUAL_SAL** FROM **EMP** ORDER BY (**SAL***12) DESC

40. DISPLAY NAME, SALARY, HRA, PF, DA, TOTAL SALARY FOR EACH EMPLOYEE. THE OUTPUT SHOULD BE IN THE OREDR OF TOTAL SALARY, (HRA 15%, DA 10%, PF 5%) OF SAL AND TOTAL SALARY WILL BE (SAL*HRA*DA)-PF:
SELECT **ENAME**, **SAL**, **SAL***15/100 **HRA**, **SAL***10/100 **DA**, **SAL***5/100 **PF** FROM **EMP** ORDER BY (**SAL*****HRA*****DA**)-**PF**

41. DISPLAY THE DEPTNO, TOTAL NUMBER OF EMPLOYEES WITH EACH JOB GROUP:
SELECT **DEPTNO**, COUNT(*) FROM **EMP** GROUP BY **DEPTNO** ORDER BY **DEPTNO**

42. DISPLAY THE VARIOUS JOBS AND TOTAL NUMBER OF EMPLOYEES IN EACH JOB GROUP:
SELECT **JOB**, COUNT(*) FROM **EMP** GROUP BY **JOB**

43. DISPLAY DEPARTMENT NUMBERS AND TOTAL SALARY FOR EACH DEPARTMENT:
SELECT **DEPTNO**, SUM(**SAL**) FROM **EMP** GROUP BY **DEPTNO**

44. DISPLAY DEPARTMENT NUMBERS AND MAXIMUM SLARY FOR EACH DEPARTMENT:
SELECT **DEPTNO**, MAX(**SAL**) FROM **EMP** GROUP BY **DEPTNO**

45. DISPLAY THE VARIOUS JOBS AND TOTAL SALARY FOR EACH JOB:
SELECT **JOB**, SUM(**SAL**) FROM **EMP** GROUP BY **JOB**

46. DISPLAY EACH JOB ALONG WITH MINIMUM SALARY BEING PAID IN EACH JOB GROUP:
SELECT **JOB**, MAX(**SAL**) FROM **EMP** GROUP BY **JOB**

47. DISPLAY THE DEPARTMENT NUMBERS WITH MORE THAN THREE EMPLOYEES IN EACH DEPARTMENT:
SELECT **DEPTNO**, COUNT(*) FROM **EMP** WHEN HAVING COUNT(*)>3 GROUP BY **DEPTNO**

48. DISPLAY THE VARIOUS JOBS ALONG WITH TOTAL SALARY FOR EACH OF THE JOBS WHERE TOTAL SALARY IS GREATER THAN 4000:
SELECT **JOB**, SUM(**SAL**) FROM **EMP** WHEN HAVING SUM(**SAL**)>4000 GROUP BY **JOB**

49. DISPLAY THE VARIOUS JOBS ALONG WITH TOTAL NUMBER OF EMPLOYEES IN EACH JOB THE OUTPUT SHOULD CONTAIN ONLY THOSE JOBS WITH MORE THAN THREE EMPLOYEE:
SELECT **JOB**, COUNT(*) FROM **EMP** WHEN HAVING COUNT(*)>3 GROUP BY **JOB**

50. DISPLAY THE NAME OF THE EMPLOYEE WHO EARNS HIGHEST SALARY:
SELECT **ENAME** FROM **EMP** WHERE **SAL** = (SELECT MAX (**SAL**) FROM **EMP**)

51. DISPLAY THE EMPLOYEE NUMBER AND NAME OF THE WORKING AS CLERK AND EARNING HIGHEST SALARY AMONG CLERK:
SELECT **EMPNO**, **ENAME**, **SAL** FROM **EMP** WHERE **JOB** ='CLERK' AND **SAL** IN (SELECT MAX (**SAL**) FROM **EMP** GROUP BY **JOB**)

52. DISPLAY THE NAMES OF THE SALESMAN WHO EARNS A SALARY MORE THAN THE HIGHEST SALARY OF ANY CLERK:
SELECT **ENAME** FROM **EMP** WHERE **JOB**='SALESMAN' AND **SAL**>
(SELECT MAX (**SAL**) FROM **EMP** WHERE **JOB**='CLERK')

53. DISPLAY THE NAMES OF THE CLERKS WHO EARN A SALARY MORE THAN THAT OF JAMES OF THAT OF SALARY LESSER THAN THAT OF SCOTT:
SELECT **ENAME** FROM **EMP** WHERE **JOB**='CLERK' AND **SAL** >
(SELECT **SAL** FROM **EMP** WHERE **ENAME**='JAMES') AND **SAL** <
(SELECT **SAL** FROM **EMP** WHERE **ENAME**='SCOTT')

54. DISPLAY THE NAMES OF THE EMPLOYEES WHO EARN A SALARY MORE THAN THAT OF JAMES OF THAT OF SALARY LESSER THAN THAT OF SCOTT:
SELECT **ENAME** FROM **EMP** WHERE **SAL**>(SELECT **SAL** FROM **EMP** WHERE **ENAME**='JAMES') AND **SAL**<(SELECT **SAL** FROM **EMP** WHERE **ENAME**='SCOTT')

55. FIND OUT THE LENGTH OF YOUR NAME BY USING APPROPIATE FUNCTION:
SELECT LENGTH ('RAMANA MURTHY') FROM **DUAL**

56. DISPLAY THE NAMES OF THE EMPLOYEES WHO EARNS HIGHEST SALARY IN THEIR RESPECTIVE DEPARTMENTS:
SELECT **ENAME** FROM **EMP** WHERE **SAL** IN (SELECT MAX (**SAL**)FROM **EMP** GROUP BY **DEPTNO**)

57. DISPLAY THE NAMES OF EMPLOYEES WHO EARNS HIGHEST SALARY IN THEIR RESPECTIVE JOB GROUPS:
SELECT **ENAME** FROM **EMP** WHERE **SAL** IN(SELECT MAX(**SAL**) FROM **EMP** GROUP BY **JOB**)

58. DISPLAY THE EMPLOYEES NAME WHO ARE WORKING IN THE ACCOUNTING DEPARTMENT:
SELECT **E.ENAME** FROM **EMP E**, **DEPT D** WHERE **D.DNAME**= 'ACCOUNTING' AND **E.DEPTNO**=**D.DEPTNO**

59. DISPLAY THE EMPLOYEES NAMES WHO ARE WORKING IN THE CHICAGO:
SELECT **E.ENAME** FROM **EMP E**, **DEPT D** WHERE **D.LOC**='CHICAGO' AND **E.DEPTNO**=**D.DEPTNO**

60. DISPLAY THE JOB GROUPS HAVING TOTAL SALARY GREATER THAN THE MAXIMUM SALARY FOR MANAGERS:
SELECT **JOB**, SUM (**SAL**) FROM **EMP** WHEN HAVING SUM(**SAL**) > (SELECT MAX (**SAL**)FROM **EMP** WHERE **JOB**= 'MANAGER')GROUP BY **JOB**

61. DISPLAY THE NAMES OF EMPLOYEES FROM DEPARTMENT NUMBER 10 AND SALARY GREATER THAN THAT OF ANY EMPLOYEE WORKING IN OTHER DEPARTMENTS:
SELECT **ENAME** FROM **EMP** WHERE **SAL**> (SELECT MAX(**SAL**) FROM **EMP** WHERE **DEPTNO** IN (20,30))

62. DISPLAY THE NAMES OF THE EMPLOYEES FROM DEPARTMENT 10 WITH SALARY GREATER THAN THAT OF ALL EMPLOYEES WORKING IN OTHER DEPARTMENTS:
SELECT **ENAME** FROM **EMP** WHERE **SAL**> (SELECT MAX(**SAL**) FROM **EMP** WHERE **DEPTNO** IN (20, 30))

63. DISPLAY NAMES OF EMPLOYEE IN UPPER CASE:
SELECT UPPER (**ENAME**) FROM **EMP**

64. DISPLAY NAMES OF EMPLOYEE IN LOWER CASE:
SELECT LOWER (**ENAME**) FROM **EMP**

65. DISPLAY NAMES OF EMPLOYEE IN PROPER CASE:
SELECT * FROM **EMP** WHERE UPPER (**ENAME**)= UPPER('smith')

66. DISPLAY THE LENGTH OF ALL EMPLOYEE NAMES:
SELECT **ENAME**, LENGTH (**ENAME**) FROM **EMP**-66

67. DISPLAY THE NAME OF THE CONCATENATE WITH EMPLOYEE NUMBER:
SELECT **ENAME**||'-'||**EMPNO** FROM **EMP**

68. USE APPROPIATE FUNCTION AND EXTRACT 3 CHARECTORS STARTING FROM 2 CHARACTER FROM THE FOLLOWING STRING 'ORACLE' i.e THE OUT PUT SHOULD BE 'RAC':
SELECT SUBSTR('ORACLE',2,3)FROM **DUAL**

69. FIND OUT FIRST OCCURANCE OF CHARACTER 'A' FROM THE FOLLOWING THE STRING 'COMPUTER MAINTANANCE CORPORATION':
SELECT INSTR ('COMPUTER MAINTENANCE CORPORATION', 'A',1) FROM **DUAL**

70. REPLACE EVERY OCCURANCE OF ALPHABET A WITH B IN THE STRING ALLEN'S (USER TRANSLATE FUNCTION):
SELECT REPLACE ('ALLENS','A','B') FROM **DUAL**

71. DISPLAY THE INFORMATION FROM EMP TABLE WHEREVER JOB 'MANAGER' IS FOUND IT SHOULD BE DISPLAYED AS BOSS (REPLACE FUNCTION):
SELECT **ENAME**, **JOB**, REPLACE (**JOB**,'MANAGER', 'BOSS') FROM **EMP** WHERE **JOB**='MANAGER'

72. DISPLAY EMPNO, ENAME, DEPTNO FROM TABLE INSTEAD OF DISPLAY DEPATMENT NUMBERS DISPLAY THE REALATED DEPARTAMENT NAME (DECODE FUNCTION):
SELECT **E.ENAME**, **E.DEPTNO**, DECODE (**D.DEPTNO**, **E.DEPTNO**, **D.DNAME**) FROM **EMP E**, **DEPT D** WHERE **E.DEPTNO**=**D.DEPTNO** ORDER BY **E.DEPTNO**;

73. DISPLAY YOUR AGE IN DAYS:
SELECT TRUNC (SYSDATE **TO_DATE**('25/07/1996', 'DD/MM/YYYY'))FROM **DUAL**

74. DISPLAY YOUR AGE IN MONTHS:
SELECT TRUNC ((SYSDATE **TO_DATE**('25/07/1996', 'DD/MM/YYYY'))/12)FROM **DUAL**

75. DISPLAY CURRENT DATE 15TH AUGEST FRIDAY NINETEEN FORTY SEVEN:
SELECT **TO_CHAR**(SYSDATE,'DD')||'TH '||**TO_CHAR** (SYSDATE, 'MONTH')||' '||**TO_CHAR**(SYSDATE,'DAY')||' ' ||**TO_CHAR**(SYSDATE,'YYYYSP') **QUARIE_75** FROM **DUAL**

76. DISPLAY THE FOLLOWING OUTPUT FOR EACH ROW FROM EMP TABLE AS 'SCOTT HAS JOINED THE COMPANY ON WEDNESDAY 13TH AUGEST NINTEEN NINETY':
SELECT **ENAME** ||' HAS JOINED'||**TO_CHAR**(**HIREDATE**, 'DD')||'TH '||**TO_CHAR** (**HIREDATE**, 'MONTH')||' '||**TO_CHAR** (**HIREDATE**,'DAY')||' '||**TO_CHAR** (**HIREDATE**,'YYYYSP') **JOIN_DATE_QUARIE_76** FROM **EMP**

77. FIND THE DATE OF NAEREST SATURDAY AFTER CURRENT DAY:
SELECT **NEXT_DAY** (SYSDATE,'SAT') FROM **DUAL**

78. DISPLAY CURRENT TIME:
SELECT SYSTIMESTAMP FROM **DUAL**

79. DISPLAY THE DATE THREE MONTHS BEFORE THE CURRENT DATE:
SELECT **ADD_MONTHS** (SYSDATE,-3) FROM **DUAL**

80. DISPLAY THE COMMON JOBS FROM DEPARTMENT NUMBER 10,20:
SELECT **JOB** FROM **EMP** WHERE **DEPTNO**=10 INTERSECT SELECT **JOB** FROM **EMP** WHERE **DEPTNO**=20

81. DISPLAY THE DETAILS OF THE THOSE WHO DO NOT HAVE ANY PERSON WORKING UNDER THEM:
SELECT DISTINCT **ENAME**, **JOB**, **MGR** FROM **EMP** WHERE **EMPNO** NOT IN **MGR**

82. DISPLAY THE JOBS WHICH ARE UNIQUE TO THE DEPARTMENT NUMBER 10:
SELECT DISTINCT **JOB** FROM **EMP** WHERE **DEPTNO**=20

83. DISPLAY THE JOBS FOUND IN DEPARTMENT NUMBER 10 AND 20 ELIMINATE DUPLICATE JOBS:
SELECT **JOB** FROM **EMP** WHERE **DEPTNO**=10
UNION
SELECT **JOB** FROM **EMP** WHERE **DEPTNO**=20

84. DISPLAY THE DETAILS OF EMPLOYEES WHO ARE IN DEPT TABLE AND GRADE IS 3:
SELECT **E.ENAME**, **D.DNAME**, **G.GRADE** FROM **EMP E**, **GRADE G**, **DEPT D** WHERE **G.GRADE**=3 AND **E.SAL** BETWEEN **G.LOSAL** AND **G.HISAL**
AND **E.DEPTNO**=**D.DEPTNO**

85. DISPLAY THOSE WHO ARE NOT MANAGER AND WHO ARE MANAGER ANY ONE:
SELECT **ENAME**, **JOB**, **EMPNO**, **MGR** FROM **EMP** WHERE **JOB** NOT IN ('MANAGER')
AND **EMPNO** NOT IN **MGR**

86. DISPLAY THOSE EMPLOYEES WHOSE NAME CONSTAIN NOT LESS THAN 4 CHARACTERS:
SELECT **ENAME** FROM **EMP** WHERE LENGTH(**ENAME**)>3

87. DISPLAY THOSE DEPARTMENTS WHOSE NAME STARTS WITH 'S' WHILE LOCATION NAME END WITH '0':
SELECT **DNAME**, **LOC** FROM **DEPT** WHERE **DNAME** LIKE '%S' AND **LOC** LIKE'%O'

88. DISPLAY THOSE EMPLOYEES WHOSE MANAGER NAME IS JONES:
SELECT **ENAME**,**MGR** FROM **EMP** WHERE **MGR**=(SELECT **EMPNO** FROM **EMP** WHERE **ENAME**='JONES')

89. DISPLAY THOSE EMPLOYEES WHOSE SALARY IS GREATER THAN 3000 AFTER GIVING 20% INCREMENT:
SELECT **ENAME**, **SAL**,(**SAL**+**SAL***20/100) **INCR_SAL** FROM **EMP**
WHERE (**SAL**+**SAL***20/100)>3000

90. DISPLAY ALL EMPLOYEES WITH THE DEPARTMENT NAME:
SELECT **E.ENAME**, **D.DNAME** FROM **EMP E**, **DEPT D** WHERE **D.DEPTNO**=**E.DEPTNO**

91. DISPLAY ENAME WHO ARE WORKING IS SALES DEPARTMENT:
SELECT **E.ENAME**, **D.DNAME** FROM **EMP E**, **DEPT D** WHERE **D.DNAME**='SALES' AND **D.DEPTNO**=**E.DEPTNO**

92. DISPLAY EMPLOYEE NAME, DEPARTMENT NAME, SALARY AND COMM FOR WHOSE SALARY IN BETWEEN 2000 AND 5000 WHILE LOCATION IN CHICAGO:
SELECT **E.ENAME**, **D.DNAME**, **E.SAL**, **E.COMM** FROM **EMP E**, **DEPT D** WHERE **E.SAL** BETWEEN 2000 AND 5000 AND **D.LOC** IN ('CHICAGO') AND **D.DEPTNO**=**E.DEPTNO**

93. DISPLAY THOSE EMPLOYEE WHOSE SALARY IS GREATER THAN HIS MANAGER SALARY:
SELECT **W.ENAME** FROM **EMP W**, **EMP M** WHERE **W.MGR**=**M.EMPNO** AND **W.SAL**>**M.SAL**

94. DISPLAY THOSE EMPLOYEES WHO ARE WORKING IN THE SAME DEPARTMENT WHERE HIS MANAGER IS WORKING:
SELECT **W.ENAME WORKER**, **M.ENAME MANAGER**, **W.DEPTNO**, **M.DEPTNO** FROM **EMP W**, **EMP M** WHERE **W.MGR**=**M.EMPNO** AND **W.DEPTNO**=**M.DEPTNO**

95. DISPLAY THOSE EMPLOYEES WHO ARE NOT WORKING UNDER ANY MANAGER:
SELECT * FROM **EMP** WHERE **MGR** IS NULL

96. DISPLAY GRADE AND EMPLOYES NAME FOR THE DEPARTMENT NUMBER 10 AND 30 BUT GRADE IS NOT 4, WHILE JOINED THE COMPANY BEFORE 31-DEC-82:
SELECT **E.ENAME**, **D.DEPTNO**, **G.GRADE**, **E.HIREDATE** FROM **EMP E**, **DEPT D**, **GRADE G** WHERE **E.DEPTNO**=**D.DEPTNO** AND **E.SAL** BETWEEN **G.LOSAL** AND **G.HISAL** AND **E.DEPTNO** IN (10,30) AND **G.GRADE** NOT IN (4)
AND **E.HIREDATE**<'31/DEC/1982' ORDER BY **D.DEPTNO**

97. UPDATE THE SALARY OF EACH EMPLOYEE BY 10% INCREATENTS THAT ARE NOT ELIGIBLE FOR COMMISION:
UPDATE **EMP2** SET **COMM**=**SAL***10/100 WHERE NVL(**COMM**,0)=0

98. DELETE EMPLOYEES WHO JOINED THE COMPANY BEFORE 31-DEC-82 WHILE THERE DEPARTMENT LOCATION IS 'NEW YORK' OR 'CHICAGO':
DELETE FROM **EMP2** WHERE **DEPTNO** IN (SELECT **DEPTNO** FROM **DEPT** WHERE **LOC** IN ('NEW YORK','CHICAGO')) AND **HIREDATE**<'31/DEC/1982'

99. DISPLAY EMPLOYEE NAME, JOB, DEPARTMENT NAME, AND LOCATION FOR ALL WHO ARE WORKING AS MANAGERS:
SELECT **E.ENAME**, **E.JOB**, **D.DNAME**, **D.LOC** FROM **EMP E**, **DEPT D**
WHERE **E.JOB**='MANAGER'

100. DISPLAY THE NAME AND SALARY OF FORD IF HIS SALARY IS EQUAL TO HIGH SALARY OF HIS GRADE:
SELECT **E.ENAME**, **E.SAL**, **G.HISAL** FROM **EMP E**, **GRADE G** WHERE **E.ENAME**='FORD' AND **E.SAL**=**G.HISAL**

101. DISPLAY THE NAME OF THOSE EMPLOYEES WHO ARE GETTING HIGHEST SALARY:
SELECT * FROM **EMP** WHERE **SAL**=(SELECT MAX(**SAL**) FROM **EMP**)

102. DISPLAY THOSE EMPLOYEES WHOSE MANAGER NAME IS JONES AND ALSO DISPLAY THE MANAGER NAME:

SELECT **W.ENAME EMPLOYEE**, **M.ENAME MANAGER** FROM **EMP W**, **EMP M** WHERE **W.MGR**=(SELECT **EMPNO** FROM **EMP** WHERE **ENAME** ='JONES') AND **M.EMPNO**=(SELECT **EMPNO** FROM **EMP** WHERE **ENAME**='JONES')-100

103. DISPLAY THE EMPLOYEE NAME, JOB, DEPARTMENT NAME, LOCATION FOR ALL WHO ARE WORKING AS MANAGER:

SELECT **W.ENAME**, **W.JOB**, **D.DNAME**, **M.ENAME MANAGER** ,**G.GRADE** FROM **EMP W**,**EMP M**, **DEPT D**, **GRADE G** WHERE **W.MGR**=**M.EMPNO** AND **W.SAL** BETWEEN **G.LOSAL** AND **G.HISAL** AND **M.DEPTNO**=**D.DEPTNO** ORDER BY **W.DEPTNO**

104. DISPLAY EMPLOYEE NAME, JOB, HIS MANAGER AND DISPLAY ALSO EMPLOYEES WHO ARE WITHOUT MANAGER:

SELECT **W.ENAME**, **W.JOB**, **M.ENAME MANAGER** FROM **EMP W**,**EMP M** WHERE **W.MGR**=**M.EMPNO**

105. LIST OUT ALL THE EMPLOYEES NAME, JOB, SALARY GARDE, DEPARTMENT NAME FOR EVERY ONE IN THE COMPANY EXCEPT 'CLERK' SORT ON SALARY DISPLAY THE HIGHEST SALARY:

SELECT **E.ENAME**, **E.JOB**, **D.DNAME**, **G.GRADE** FROM **EMP E**, **DEPT D**, **GRADE G** WHERE **E.SAL** BETWEEN **G.LOSAL** AND **G.HISAL** AND **E.DEPTNO**=**D.DEPTNO** AND **E.ENAME** NOT IN ('CLERK')ORDER BY **E.SAL** DESC

106. DISPLAY THOSE EMPLOYEE WHOSE SALARY IS EQAUAL TO AVERAGE OF MAXIMUM AND MINIMUM:

SELECT*FROM **EMP** WHERE **SAL**> (SELECT AVG(**SAL2**) FROM (SELECT MAX (**SAL**) **SAL2** FROM **EMP** UNION SELECT MIN(**SAL**)FROM **EMP**))

107. DISPLAY THE COUNT OF EMPLOYEE IN EACH DEPARTMENT WHERE COUNT GREATER THAN 3:

SELECT **DEPTNO**, COUNT (*) FROM **EMP** WHEN HAVING COUNT(*)>3 GROUP BY **DEPTNO**

108. DISPLAY DEPARTMENT NAME WHERE AT LEAST 3 ARE WORKING AND DISPLAY ONLY DEPARTMENT NAME:

SELECT **DNAME** FROM (SELECT **DEPTNO** FROM **EMP** GROUP BY **DEPTNO** HAVING COUNT (*)> =3)**E**,**DEPT D** WHERE **E.DEPTNO** = **D.DEPTNO**

109. DISPLAY NAMES OF THOSE MANAGERS WHOSE SALARY IS MORE THAN THAT AVERAGE SALARY OF COMAPNY:

SELECT **ENAME** FROM **EMP** WHERE **JOB** ='MANAGER' AND **SAL**>(SELECT AVG(**SAL**) FROM **EMP**)

113. FIND OUT THE LAST 5 (LEAST) EARNER OF THE COMPANY:

SELECT * FROM (SELECT **ENAME**, **JOB**, **SAL** FROM **EMP** ORDER BY **SAL**)WHERE ROWNUM BETWEEN 1 AND 5

121. DISPLAY THOSE EMPLOYEE WHOSE SALARY IS ODD VALUE:

SELECT *FROM **EMP** WHERE MOD(**SAL**,2)=1-121

110. DISPLAY THOSE MANAGER NAMES WHOSE SALARY IS MORE THAN AN AVERAGE OF HIS EMPLOYEES:

```
SELECT * FROM EMP E WHERE SAL> (SELECT AVG (SAL) FROM EMP D WHERE
D.EMPNO=E.MGR)
```

111. DISPLAY EMPLOYEE NAME, SALARY, COMMISION, NET PAY AND WHOSE NET PAY IS GREATER THAN OR EQUAL TO THEIR SALARY OF THE COMPANY:

```
SELECT E.ENAME, E.SAL, E.COMM, A.NET_PAY FROM EMP E,
(SELECT EMPNO, SAL+NVL (COMM, 0)-SAL*10/100 NET_PAY FROM EMP)A
WHERE E.EMPNO=A.EMPNO AND A.NET_PAY >=SAL;
```

112. DISPLAY THOSE EMPLOYEES WHOSE SALARY IS LESS THAN HIS MANAGER SALARY BUT MORE THAN SALARY OF ANY OTHER MANAGER:

```
SELECT DISTINCT T.ENAME, T.SAL FROM (SELECT W.ENAME, W.SAL FROM EMP W,
EMP M WHERE W.MGR=M.EMPNO AND W.SAL<M.SAL ) T, (SELECT M.SAL FROM
EMP W, EMP M WHERE W.MGR=M.EMPNO) R  WHERE T.SAL>R.SAL;
```

114. FIND OUT THE NUMBER OF EMPLOYEES WHOSE SALARY IS GREATER THAN THERE MANAGER SALARY:

```
SELECT COUNT (*) FROM (SELECT W.ENAME FROM EMP W, EMP M WHERE
W.MGR=M.EMPNO AND W.SAL > M.SAL)
```

115. DISPLAY THOSE MANAGER WHO ARE NOT WORKING UNDER PRESEDENT BUT WORKING UNDER ANY OTHER MAANAAGER:

```
SELECT * FROM EMP WHERE MGR IS NOT NULL AND MGR NOT IN(SELECT EMPNO
FROM EMP WHERE JOB='PRESIDENT')
                    ---------OR----------
SELECT * FROM EMP W, EMP M WHERE W.MGR=M.EMPNO
AND W.MGR IS NOT NULL AND M.JOB NOT IN ('PRESIDENT')
```

116. DELETE THOSE DEPARTMENTS WHERE NO EMPLOYYES WORKING:

```
DELETE FROM DEPT WHERE DEPTNO NOT IN
(SELECT DISTINCT(DEPTNO) FROM EMP)
```

117. DELETE THOSE RECORDS FROM EMP TABLE WHOSE DEPARTMENT NUMBER NOT AVAILANBLE IN DEPARTMENT TABLE:

```
DELETE FROM EMP2 WHERE DEPTNO NOT IN
(SELECT D.DEPTNO FROM EMP2 E,  DEPT D WHERE E.DEPTNO(+)=D.DEPTNO)
```

118. DISPLAY THOSE EARNERS WHOSE SALARY IS OUT OF THE GRADE AVAILABLE IN SALARY GRADE TABLE:

```
SELECT *FROM EMP E,GRADE G WHERE GRADE=
(SELECT MAX(GRADE)FROM GRADE) AND E.SAL BETWEEN G.LOSAL AND G.HISAL
```

119. DISPLAY EMPLOYEE NAME, SALARY, COMMISION AND WHO'S NET PAY IS GREATER THAN ANY OTHER IN THE COMPANY:

```
SELECT DISTINCT E.EMPNO, E.ENAME, E.SAL, E.COMM, A.NET_PAY FROM EMP E,
(SELECT EMPNO, SAL+NVL (COMM,0)-SAL*10/100 NET_PAY FROM EMP)A
WHERE A.NET_PAY > (SAL+NVL (COMM, 0)-SAL*10/100);
```

120. DISPLAY NAME OF THE EMPLOYEE WHO ARE GOING TO RETIRE 31-DEC-99. IF THE MAXIMUM JOB PERIOD IS 18 YEARS:

SELECT * FROM (SELECT **ENAME**, **HIREDATE**,TRUNC (**MONTHS_BETWEEN** (**TO_DATE**('31/12/1999', 'DD/MM/YYYY'), **HIREDATE**)/12) **EXPERIENCE** FROM **EMP**) WHERE **EXPERIENCE**>18

122. DISPLAY THOSE EMPLOYEE WHOSE SALARY CONTAINS AT LEAST 4 DIGITS:

SELECT*FROM **EMP** WHERE LENGTH(**SAL**)>3-122

123. DISPLAY THOSE EMPLOYEES WHO JOINED IN THE COMPANY IN THE MONTH OF DECEMBER:

SELECT * FROM **EMP** WHERE **TO_CHAR** (**HIREDATE**,'MM')=12

124. DISPLAY THOSE EMPLOYEE WHOSE NAME CONTAIN 'A':

SELECT * FROM **EMP** WHERE **ENAME** LIKE '%A%'

125. DISPLAY THOSE EMPLOYEE WHOSE DEPARTMENT NUMBER AVAILABLE IN SLARY:

SELECT **ENAME**, **SAL**, **DEPTNO** FROM (SELECT **ENAME**, **SAL**, SUBSTR (**SAL**,1,2)**SUB_SAL**, **DEPTNO** FROM **EMP2**) WHERE **SUB_SAL** IN **DEPTNO**

126. DISPLAY FIRST 2 CHARECTER OF HIREDATE AND LAST 2 CHARECTER OF SALARY OF THE EMPLOYEES:

SELECT SUBSTR (**HIREDATE**,1,2)||'-'||SUBSTR(**SAL**,-2,2)**HIREDATE_SAL** FROM **EMP**

127. DISPLAY THOSE EMPLOYEE WHOSE 10% SALARY IS EQUAL TO THE YEAR OF JOINING:

SELECT * FROM **EMP** WHERE(**SAL***10/100)= (**TO_CHAR** (**HIREDATE**,'YY'))

128. DISPLAY THOSE EMPLOYEE WHO ARE WORKING IN SALES OR RESEARCH:

SELECT * FROM **EMP E** , **DEPT D** WHERE **D.DNAME** IN ('SALES','REASEARCH') AND **E.DEPTNO**=**D.DEPTNO**

129. DISPLAY THE GRADE OF JONES:

SELECT **E.ENAME**, **G.GRADE** FROM **EMP E**, **GRADE G** WHERE **E.SAL** BETWEEN **G.LOSAL** AND **G.HISAL** AND **E.ENAME**='JONES'

130. DISPLAY THOSE EMPLOYEES WHO JOINED THE COMPANY BEFORE 15TH OF THE MONTH:

SELECT * FROM **EMP** WHERE **TO_CHAR** (**HIREDATE**,'DD')<15

131. DELETE THOSE EMPLOYEES WHO JOINED THE COMPANY 21 YAERS BACK FROM TODAY:

SELECT * FROM **EMP** WHERE (SYSDATE-**HIREDATE**)/365>21

132. DISPLAY THOSE EMPLOYEES WHO ARE WORKING AS MANAGER:

SELECT * FROM **EMP** WHERE **JOB**='MANAGER'

133. COUNT THE NUMBER OF EMPLOYEES WHO ARE WORKING AS MANAGER:

SELECT **JOB**, COUNT (*) FROM **EMP** GROUP BY **JOB** HAVING **JOB**='MANAGER'

--------- (OR) ---------

SELECT DISTINCT (**M.ENAME**), **M.JOB** FROM **EMP W**, **EMP M** WHERE **W.MGR**=**M.EMPNO**

134. DISPLAY THE DEPARTMENT NAME, THE NUMBER OF CHARECTERS OF WICH IS EQUAL TO NUMBER OF EMPLOYEES IN ANY OTHER DEPARTMENT:
SELECT * FROM **DEPT** WHERE LENGTH(**DNAME**) IN (SELECT COUNT(*) FROM **EMP** GROUP BY **DEPTNO**)

---------- (OR) ----------

SELECT * FROM **DEPT D**,(SELECT **DEPTNO**, COUNT(*) **C_DPT** FROM **EMP** GROUP BY **DEPTNO**)**S** WHERE LENGTH (**D.DNAME**) IN **S.C_DPT**

135. DISPLAY THE NAME OF THE DEPARTMENT THOSE EMPLOYEES WHO JOINED THE COMPANY ON THE SAME DATE:
SELECT * FROM **EMP** WHERE **HIREDATE** IN (SELECT **HIREDATE** FROM **EMP** GROUP BY **HIREDATE** HAVING COUNT (*) >1)

---------- (OR) ----------

SELECT * FROM **EMP E**, (SELECT **HIREDATE** FROM **EMP** GROUP BY **HIREDATE** HAVING COUNT (*)>1) **AWHEREE.HIREDATE=A.HIREDATE**

136. DISPLAY THE MANAGER NAME WHO HAS MAXIMUM NUMBER OF EMPLOYEES WORKING UNDER HIM:
SELECT **E.ENAME**, **C.NO_EMP** FROM **EMP E**, (SELECT * FROM (SELECT **MGR**, COUNT (*) **NO_EMP** FROM **EMP** GROUP BY **MGR** ORDER BY COUNT (*)DESC) WHERE ROWNUM=1)**C** WHERE **E.EMPNO=C.MGR**

137. LIST OUT EMPLOYEES NAME AND SALARY INCREASED BY 15% AND EXPREDDED AS WHOLE NUMBER DOLLAR:
SELECT **ENAME**, TRUNC((**SAL**+**SAL***15/100)/72,2) **SAL_DOLLORS** FROM **EMP**

138. PRODUCE THE OUTPUT OF THE EMP TABLE "EMPLOYEE_AND_JOB" FOR NAME AND JOB:
SELECT **ENAME**||'_AND_'||**JOB** FROM **EMP**

139. LIST ALL EMPLOYEES WITH HIREDATE IN FORMAT 'JUNE 4 1988':
SELECT **ENAME**||'-'|| **TO_CHAR** (**HIREDATE**, LOWER ('MONTH')||' '||'DD'||' '||'YYYY') **JOIN_DATE** FROM **EMP**

140. PRINT A LIST OF EMPLOYEES DISPLAYING 'LESS SALARY' IF LESS THAN 1500 IF EXACTLY 1500 DISPLAY AS 'EXACT SALARY' AND IF GREATER THAN 1500 THEN DISPLAY 'MORE SALARY':
SELECT **ENAME**, **SAL**, CASE WHEN **SAL**<1500 THEN 'LESS_SAL'WHEN **SAL**=1500 THEN 'EXACT_SAL' ELSE 'MORE_SAL' END **SAL_INFO** FROM **EMP**

141. WRITE A QUERY TO CALUCULATE THE LENGTH OF EMPLOYEE NAME HAS BEEN WITH THE QUERY:
SELECT **ENAME**, LENGTH (**ENAME**) FROM **EMP**

142. DISPLAY THOSE MANAGERS WHO ARE GETTING LESS THAN HIS EMPLOYEES SALARY:
SELECT * FROM **EMP W**, **EMP M** WHERE **W.MGR=M.EMPNO** AND **W.SAL>M.SAL**

143. DISPLAY THOSE WHO WORKING AS MANAGER USING CO-REALTED SUB QUERY:
SELECT * FROM **EMP** WHERE **EMPNO** IN (SELECT **MGR** FROM **EMP**)

144. PRINT THE DETAILS OF ALL THE EMPLOYEES WHO ARE SUB ORDINATE TO BLAKE:

```
SELECT * FROM EMP WHERE MGR=(SELECT EMPNO FROM EMP
WHERE ENAME='BLAKE')
```

145. DEFINE VARIABLE REPRESENTING THE EXPRESSIONS USED TO CALUCULATE ON EMPLOYEES TOTAL ANNUAL REMUNIRATION:

```
SELECT (SAL+NVL(COMM,0))*12 ANNUAL_SAL FROM EMP
```

146. USE THE VARIABLE IN A STATEMENT WHICH FINDS ALL EMPLOYEES WHO CAN EARN 3000 A YEAR OR MORE:

```
SELECT ENAME, (SAL+NVL(COMM,0))*12 ANNUAL_SAL FROM EMP WHERE
((SAL+NVL(COMM,0))*12)>=30000
```

147. FIND OUT HOW MANY MANAGERS ARE THERE WITH OUT LISTENING THEM:

```
SELECT COUNT (*) FROM EMP WHERE EMPNO IN (SELECT MGR FROM EMP)
```

148. DISPLAY THOSE EMPLOYEES WHOSE MANAGER NAME IS JONES ALSO WITH HIS MANAGER NAME:

```
SELECT W.ENAME WORKER, M.ENAME MANAGER FROM EMP W, EMP M WHERE
W.MGR IN (SELECT EMPNO FROM EMP WHERE ENAME= 'JONES')
AND W.MGR=M.EMPNO
UNION
SELECT W.ENAME WORKER, M.ENAME MANAGER FROM EMP W, EMP M
WHERE W.EMPNO IN (SELECT EMPNO FROM EMP WHERE ENAME='JONES')
AND W.MGR=M.EMPNO
                    --------- (OR) ---------
SELECT W.ENAME WORKER, M.ENAME MANAGER FROM EMP W, EMP M WHERE
W.MGR IN ((SELECT EMPNO FROM EMP WHERE ENAME='JONES'), (SELECT MGR
FROM EMP WHERE ENAME='JONES'))AND W.MGR=M.EMPNO
```

149. FIND OUT THE AVERAGE SALARY AND AVERAGE TOTAL REMUNIRATION FOR EACH JOB TYPE REMEMBER SALESMAN EARN COMMISSION:

```
SELECT JOB, TRUNC(AVG(SAL+NVL(COMM,0))) AVG_SAL,
TRUNC(AVG((SAL+NVL(COMM,0))*12)) AVG_ANNUAL FROM EMP GROUP BY JOB
```

150. CHECK WHEATHER ALL EMPLOYEE NUMBER ARE INDEED UNIQUE:

```
SELECT DISTINCT EMPNO FROM EMP
```

151. DISPLAY THE DEPARTMENT NAME WHERE THERE IS NO EMPLOYEES:

```
SELECT DNAME,D.DEPTNO FROM EMP E,DEPT D
WHERE E.DEPTNO(+)=D.DEPTNO AND E.ENAME IS NULL
```

152. LIST OUT THE LOWEST PAID EMPLOYEES WORKING FOR THE EACH MANAGER, EXCLUDE ANY GROUPS WHERE MIN SALARY IS LESS THAN 1000 SORT THE OUTPUT BY SALARY:

```
SELECT W.ENAME, W.SAL , M.ENAME, M.SAL FROM EMP W, EMP M
WHERE M.EMPNO=W.MGR AND W.SAL<M.SAL AND W.SAL>1000 AND
W.SAL IN(SELECT MIN(SAL) SAL FROM EMP GROUP BY MGR) ORDER BY W.SAL
```

153. (A) LIST NAME, JOB, ANNUAL SALARY, DEPTNO, DEPARTMENT NAME, GRADE WHO EARN MORE THAN 30000 PER YEAR AND WHO ARE NOT CLERK:
(B) DISPLAY THE EMPLOYEES THOSE WHO JOINED EARLIER THAN HIS MANAGER:

SELECT **E.ENAME**, **E.SAL**, **E.EMPNO**, **E.JOB**, **D.DNAME**, **G.GRADE** FROM **EMP E**, **DEPT D**, **GRADE G** WHERE ((**SAL**+NVL (**COMM**, 0))*12)> 30000 AND **JOB** NOT IN 'CLERK' AND **E.SAL** BETWEEN **G.LOSAL** AND **G.HISAL** AND **E.DEPTNO**=**D.DEPTNO**

---------- AND ----------

SELECT **W.ENAME**, **W.HIREDATE**, **M.ENAME MANAGER**, **M.HIREDATE** FROM **EMP W**, **EMP M** WHERE **W.MGR**=**M.EMPNO** AND **W.HIREDATE**<**M.HIREDATE**

154. DISPLAY THE AVERAGE SAL OF FIGURE FOR THE DEPT:

SELECT **DEPTNO**, TRUNC (AVG (**SAL**)) **AVG_SAL** FROM **EMP** GROUP BY **DEPTNO**

155. LIST OUT ALL THE EMPLOYEES BY NAME AND NUMBER ALONG WITH THEIR MANAGERD NAME AND NUMBER ALSO DISPLAY 'NO MANGER' WHO HAS NO MANAGER:

SELECT DISTINCT **W.EMPNO**, **W.ENAME**, CASE WHEN **W.MGR** IS NOT NULL THEN **M.ENAME** WHEN **W.MGR** IS NULL THEN 'NO MANAGER' END **MANAGER** FROM **EMP W**, **EMP M** WHERE **W.MGR** IS NULL OR **W.MGR**=**M.EMPNO**

156. FIND OUT THE EMPLOYEE WHO EARNED THE MIN SAL FOR THEIR IN ASCENDING ORDER:

SELECT **ENAME**, **JOB**, **SAL** FROM **EMP** WHERE **SAL** IN(SELECT MIN(**SAL**) FROM **EMP** GROUP BY **JOB**) ORDER BY **SAL**

157. FIND OUT THE MOST RECENTLY HIRED EMPLOYEES IN EACH DEPARTMENT ORDER BY HIREDATE:

SELECT **D.DNAME**, **E.ENAME**, **E.HIREDATE** FROM **EMP E**, **DEPT D** WHERE **E.HIREDATE** IN (SELECT MAX (**HIREDATE**) FROM **EMP** GROUP BY **DEPTNO**) AND **E.DEPTNO**=**D.DEPTNO**

-------- (OR)---------

SELECT DISTINCT **ENAME**, **HIREDATE** FROM **EMP** WHERE **HIREDATE** IN (SELECT MAX(**HIREDATE**) FROM **EMP** GROUP BY **DEPTNO**) ORDER BY **HIREDATE**

158. DISPLAY NAME, SALARY AND DEPTNO FOR EACH EMPLOYEE WHO EARN SALARY GREATER THAN THE AVERAGE OF THE THEIR DEPARTMENT ORDER BY DEPTNO:

SELECT **E.DEPTNO** ,**E.ENAME**, **E.SAL**, **S.AVG_SAL** FROM **EMP E**,(SELECT **DEPTNO**, TRUNC(AVG(**SAL**))**AVG_SAL** FROM **EMP** GROUP BY **DEPTNO**)**S** WHERE **S.DEPTNO**=**E.DEPTNO** AND **E.SAL**>**S.AVG_SAL**

158. FIND OUT ALL DEPARTMENTS WICH HAVE MORE THAN 3 EMPLOYEES:

SELECT **DEPTNO**, COUNT (*) FROM **EMP** HAVING COUNT(*)>3 GROUP BY **DEPTNO**

159. IN WHICH YEAR DID MOST PEOPLE JOIN THE COMPANY. DISPLAY THE YEAR AND NUMBER OF EMPLOYEES:

SELECT **YEAR_OF_JOIN**, **NO_OF_EMP** FROM(SELECT **HIREDATE YEAR_OF_JOIN**, COUNT (*) **NO_OF_EMP** FROM (SELECT **ENAME**, **TO_CHAR**(**HIREDATE** ,'YYYY') **HIREDATE** FROM **EMP**) GROUP BY **HIREDATE** ORDER BY COUNT(*) DESC) WHERE ROWNUM=1

160. DISPLAY THE DEPARTMENT NUMBER WITH HIGHEST ANNUAL REMUNIRATION BILL AS COMPENSATION:

```
SELECT K.DEPTNO, K.ANSL FROM (SELECT N.*,DENSE_RANK () OVER(ORDER BY
N.ANSL DESC) DRK FROM (SELECT DEPTNO, ((SAL+NVL(COMM,0))*12) ANSL FROM
EMP) N) K WHERE DRK=1
```

---------- (OR) ----------

```
SELECT DEPTNO, ANSL, ROWNUM FROM (SELECT DEPTNO,SUM(ANSL) ANSL FROM
(SELECT DEPTNO,((SAL+NVL(COMM,0))*12)ANSL FROM EMP)GROUP BY DEPTNO
ORDER BY ANSL DESC) WHERE ROWNUM=1
```

161. WRITE A QUERY OF DISPLAY AGAINIST THE ROW OF THE MOST RECENT HIRED EMPLOYEE. DISPLAY ENAME HIREDATE AND COLUMN MAX DATE SHOWING:

```
SELECT * FROM (SELECT ENAME, HIREDATE FROM EMP
ORDER BY HIREDATE DESC) WHERE ROWNUM=1
```

------- (OR) ---------

```
SELECT ENAME, HIREDATE FROM EMP ORDER BY HIREDATE DESC
```

162. DISPLAY EMPLOYEES WHO CAN EARN MORE THAN LOWEST SALARY IN DEPARTMENT NO.30:

```
SELECT * FROM EMP WHERE SAL>
(SELECT MIN(SAL) FROM EMP WHERE DEPTNO=30)
```

-------- (OR) ----------

```
SELECT * FROM EMP WHERE SAL>(SELECT MIN(SAL) FROM EMP
HAVING DEPTNO=30  GROUP BY DEPTNO)
```

163. FIND EMPLOYEES WHO CAN EARN MORE THAN EVERY EMPLOYEE IN DEPARTMENT NO.30:

```
SELECT * FROM EMP WHERE SAL>
ANY (SELECT SAL FROM EMP WHERE DEPTNO=30)
```

164. FIND OUT AVERAGE SALARY AVERAGE TOTAL REMAINDERS FOR EACH JOB TYPE:

```
SELECT J.JOB, J.M NO_OF_EMP, J.N AVG_SAL, TRUNC (J.N/J.M) REMINDER FROM
 (SELECT JOB, COUNT (JOB)M, TRUNC (AVG(SAL)) N FROM EMP GROUP BY JOB)J
```

165. DISPLAY THE HALF OF THE ENAME IN UPPER AND HALF OF THE ENAME IN LOWER CASE:

```
SELECT UPPER (SUBSTR (ENAME, 1, LENGTH (ENAME)/2)) ||LOWER (SUBSTR
(ENAME, (LENGTH (ENAME)/2)+1)) FROM EMP
```

166. CREATE COPY OF EMP TABLE:

```
CREATE TABLE EMP2 TABLE AS SELECT * FROM EMP
```

167. DISPLAY ENAME IF ENAME EXISTS MORE THAN ONE:

```
SELECT ENAME FROM EMP GROUP BY ENAME HAVING COUNT (ENAME)>1
```

168. DISPLAY ALL ENAMES IN REVERSE ORDER:

```
SELECT REVERSE (ENAME) FROM EMP
```

169. DISPLAY THOSE EMPLOYEES WHO'S JOING OF THE MONTH AND GRADE IS EQUAL:

```
SELECT E.ENAME, E.HIREDATE, G.GRADE FROM EMP E,GRADE G WHERE E.SAL
BETWEEN G.LOSAL AND G.HISAL AND TO_CHAR(HIREDATE,'MM')=GRADE
```

170. DISPLAY THOSE EMPLOYEES NAME AS FOLLOWS A ALLEN, B BLAKE:
SELECT SUBSTR(**ENAME**, 1,1)||'_'||**ENAME** FROM **EMP**

171. DISPLAY THOSE EMPLOYEE WHOS JOING DATE IS AVAILABLE IN DEPARTMENT NUMBER:
SELECT * FROM (SELECT **ENAME**,**TO_CHAR**(**HIREDATE**,'DD') **J_DATE**, **DEPTNO** FROM **EMP2**) WHERE **J_DATE** IN **DEPTNO**

172. PRINT PINEAPPLE AS EACH LETTER IN SEPARATE ROW:
SELECT SUBSTR('PINEAPPLE',LEVEL,1) **PINEAPPLE** FROM **DUAL**
CONNECT BY LEVEL <= LENGTH('PINEAPPLE');

173. LIST OUT THE EMPLOYEE NAME, SALARY AND PF FROM EMP:
SELECT **ENAME**, **SAL**, **SAL***12/100 **PF** FROM **EMP**

174. CREATE TABLE WITH ONLY COLUMMN EMPNO:
CREATE TABLE **EMPNO** AS SELECT **EMPNO** FROM **EMP**

175. ADD THIS COLUMN TO EMP TABLE ENAME VARCHAR2 (20):
ALTER TABLE **EMP** ADD **ENAME** VARCHAR2(20)

176. OOPS.! GIVE THE PRIMARY KEY CONSTRAINT ADD IT NOW:
ALTER TABLE **EMP** ADD CONSTRAINT **PK_EMPNO** PRIMARY KEY (**EMPNO**)

177. NOW INCREASE THE LENGTH OF ENAME, COLUMN TO 30 CHARACTER:
ALTER TABLE **EMP** MODIFY **ENAME** VARCHAR2 (30)

178. ADD SALARY COLUMN TO EMP TABLE:
ALTER TABLE **EMP** ADD **SALARY** NUMBER

179. I WANT TO GIVE A VALIDATION SAYING THAT SALARY CONNOT BE GREATER THAN 10,000, (NOTE GIVE NAME TO THIS COLUMN):
ALTER TABLE **EMP** ADD CONSTRAINT **CK_SAL** CHECK (**SAL**<10000)

180. FOR THE TIME BEING I HAVE DECIDED THAT I WILL NOT IMPOSE THIS VALIDATION. MY BOSS HAS AGREED TO PAY MORE THAN 10000:
ALTER TABLE **EMP** DROP CONSTRAINT **CK_SAL**

182. MY BOSS HAS CHANGED HIS MIND. NOW HE DOES'T WANT TO PAY MORE THAN 10000 SALARY. SO REVOKE THAT SALARY CONSTRAINT:
ALTER TABLE **EMP** ADD CONSTRAINT **CK_SAL** CHECK(**SAL**<10000)

183. CREATE TABLE CALLED NEW_EMP.
CREATE TABLE **NEW_EMP** AS SELECT * FROM **EMP**

184. OH! THIS COLUMN SHOULD BE RELATED TO EMPNO. GIVE A COMMAND TO ADD THIS CONSTRAINT ADD DEPTNO COLUMN TO EMP TABLE THIS DEPTNO NO COLUMN SHOULD BE RELATED TO DEPT TABLE:
"ALTER TABLE **DEPT** ADD CONSTRAINT **UK_DEPTNO** UNIQUE (**DEPTNO**)"
"ALTER TABLE **EMP** ADD CONSTRAINT **FK_DEPTNO** FOREIGN KEY(**DEPTNO**) REFERENCES **DEPT**(**DEPTNO**)"

185. PROVIDE A COMMISSION TO EMPLOYEES WHO ARE NOT EARNING COMMISION:
UPDATE **EMP** SET **COMM**=1000 WHERE **COMM** IS NULL OR **COMM**=0

186. CREATE TABLE CALLED AS NEW_EMP. THIS TABLE SHOULD CONTAIN EMPNO, ENAME and DNAME ONLY:
CREATE TABLE **NEW_EMP** AS SELECT **ENAME**, **EMPNO**, **DNAME** FROM **EMP E**, **DEPT D** WHERE **E.DEPTNO**=**D.DEPTNO**

187. DELETE THE ROWS OF EMPLOYEES WHO ARE WORKING IN THE COMPANY FOR MORE THAN TWO YEARS:
DELETE FROM **EMP** WHERE **HIREDATE** IN (SELECT **HIREDATE** FROM **EMP** WHERE (SYSDATE-**HIREDATE**)/365>39)

188. IF ANY EMPLOYEE HAS COMMISION, HIS COMMISION SHOULD BE INCREAMENTED BY 10% OF HIS SALARY:
UPDATE **EMP** SET **COMM**=**COMM** + (**SAL***10/100)
WHERE **COMM** IS NOT NULL OR **COMM** NOT IN (0)

189. DISPLAY EMPLOYEE NUMBER, NAME AND LOCATION OF THE DEPARTMENT IN WHICH HE IS WORKING:
SELECT **EMPNO**,**ENAME**, **DNAME**, **LOC** FROM **EMP E**,**DEPT D** WHERE **E.DEPTNO**=**D.DEPTNO**

190. DISPLAY NAME, DEPARTMENT NAME, EVEN IF THERE IS NO EMPLOYEES WORKING IN A PARTICULAR DEPARTMENT (USE OUTER JOIN);
SELECT **E.EMPNO**, **D.DEPTNO**, **D.DNAME** FROM **EMP E**, **DEPT D**
WHERE **E.DEPTNO** (+) = **D.DEPTNO**

191. DISPLAY EMPLOYEE NAME AND DEPARTMENT NAME FOR EACH EAMPLOYEE:
SELECT **ENAME**, **DNAME** FROM **EMP E**, **DEPT D** WHERE **E.DEPTNO**=**D.DEPTNO**

192. DISPLAY EMPLOYEE NAME AND HIS MANAGER NAME:
SELECT **E.ENAME**, **M.ENAME** FROM **EMP E**, **EMP M** WHERE **E.MGR**=**M.EMPNO**

193. DISPLAY DEPARTMENT NUMBER ALONG WITH TOTAL SALARY IN EACH DEPARTMENT:
SELECT **DEPTNO**, SUM(**TOTAL_SAL**) **TOTAL_SAL** FROM (SELECT **DEPTNO**, **SAL**+ NVL(**COMM**,0)**TOTAL_SAL** FROM **EMP**) GROUP BY **DEPTNO**

---------- (OR) ----------

SELECT **DEPTNO**, SUM (**SAL**+NVL(**COMM**,0)) **TOTAL_SAL** FROM **EMP** GROUP BY **DEPTNO**

194. DISPLAY THE CURRENT DATE AND TIME:
SELECT SYSDATE FROM **DUAL**

195. DISPLAY THE DEPARTMENT NUMBER AND TOTAL NUMBER OF EMPLOYEES IN EACH DEPARTMENT:
SELECT **DEPTNO**, COUNT (*) FROM **EMP** GROUP BY **DEPTNO**

196. WE WANT TO DELETE DUPLICATE VALUES THEN:
DELETE FROM **EMP2** WHERE ROWID NOT IN(SELECT MIN(ROWID)
FROM **EMP2** GROUP BY **EMPNO**)

197. WHEN WANT TO TOTAL SAL STEP BY STEP AND ORDERWISE THEN:

```
SELECT DEPTNO, ENAME, JOB, SUM (SAL) OVER (ORDER BY EMPNO) SUM_SAL
FROM EMP
```

198. WE WANT ALL SATUR_DAYS AND SUNDAYS IN A YEAR:

```
SELECT (NEXT_DAY (TRUNC(SYSDATE,'YYYY'),  'SAT')-7)+ LEVEL*7
SATUR_DAYS_OF_2020,(NEXT_DAY(TRUNC(SYSDATE, 'YYYY'),'SUN') -7)+ LEVEL*7
SUN_DAYS_OF_2020 FROM DUAL CONNECT BY LEVEL<53
```

200. DISPLAY MOST OF THE JOININGS IN WHICH YEAR

```
SELECT * FROM (SELECT TO_CHAR (HIREDATE, 'YYYY')JOIN_YEAR, COUNT(*)
NO_EMP FROM EMP GROUP BY TO_CHAR (HIREDATE,'YYYY') ORDER BY
COUNT (*) DESC) WHERE ROWNUM=1
```

PROCEDURAL LANGUAGE EXTENSION OF SQL

PL/SQL is a programming language. It is extension of Structured Query Language (SQL) that is used in Oracle. PL/SQL allows the programmer to write code in a procedural format. It combines the data manipulation power of SQL with the processing power of procedural language to create super powerful SQL queries. PL/SQL means instructing the compiler 'what to do' through SQL and 'how to do' through its procedural way.

The PL/SQL architecture mainly consists of following three components:

1. PL/SQL block
2. PL/SQL Engine
3. Database Server

PL/SQL:

- ✓ It supports to execute a block of statements as a unit.
- ✓ It supports variables and constants.
- ✓ It supports error handling.
- ✓ It supports define composite data type variables.
- ✓ It supports to execute multiple DDL, DML, TCL commands at a time.

NOTE: ' ; ' is not acceptable for declare, as, begin, if, loop. If you want to print a hardcore value in PL/SQL you have to put it in between single quote (''). And variable should not write between single quote. If you write variables in between single quotes its print like hardcore values. Variables should be declared between declare and begin.

Structure of PL/SQL block:

```
DECLARE
    --VARIABLE SECTION;
BEGIN
     --CODING (STATEMENT), MINIMUM ONE STATEMENT IS REQIRED;
EXCEPTION
    --ERROR HANDLING SECTION;
END;
```

Ex_1: Declare

```
DECLARE
 X NUMBER:=10;
 Y NUMBER:=20;
 Z NUMBER;
BEGIN
   Z:= X+Y;
   DBMS_OUTPUT.PUT_LINE(Z);
END;
```

LOOP

- **BASIC LOOP:**

We have to use the word 'exit' with proper condition to end this loop. If you don't put any condition on this it runs continuously. That's why this loop is called as infinite loop.

Ex:
```
DECLARE
 X NUMBER:=0;
BEGIN
   LOOP
   X:=X+1;
   DBMS_OUTPUT.PUT_LINE(X);
   EXIT WHEN X=5;
   END LOOP;
END;
```

If we have don't written 'EXIT WHEN X=5' condition it will be infinite loop.

- **FOR LOOP:**

We have to give lower and upper limits to run the FOR LOOP.

Ex: Arm strong number
```
DECLARE
 X NUMBER:= 153;
 Y NUMBER:= 0;
BEGIN
  FOR I IN 1..LENGTH(X) LOOP
      Y:=Y+POWER(SUBSTR(X,I,1),LENGTH(X));
  END LOOP;
    IF X=Y THEN
     DBMS_OUTPUT.PUT_LINE('GIVEN NO IS ARM STRONG NUMBER');
     ELSE
     DBMS_OUTPUT.PUT_LINE('NOT A ARM STRONG NUMBER');
    END IF;
END;
```

- **WHILE LOOP:**

It is condition based loop. If don't satisfy the condition at any time it run's continuously infinity times.

Ex:
```
DECLARE
 X NUMBER;
BEGIN
   WHILE (X<10) LOOP
   DBMS_OUTPUT.PUT_LINE(X);
   X:= X+1;
   END LOOP;
END;
```

If we have don't satisfy '(X<10)' condition in the program it will run infinite times.

INTO CLAUSE:

Generally we can't write select statement in the PL/SQL blocks. If you want to write select statement in the PL/SQL block you should go for into clause. Into class will support to fetch maximum one row of data at a time. If you want write another row you need to write one more into class.

Ex:
```
DECLARE
   X VARCHAR2(30);
   Y VARCHAR2(30);
   Z NUMBER:= 100;
BEGIN
   SELECT ENAME,JOB,SAL INTO X,Y,Z FROM EMP
        WHERE EMPNO=7369;
   DBMS_OUTPUT.PUT_LINE(X||' '||Z||' '||Y);
END;
```

BLOB:

```
DECLARE
  SRC_LOB  BFILE:=BFILENAME('DATA_PUMP_DIR','RAMANA_ID.JPG');
  DEST_LOB BLOB;
BEGIN
  INSERT INTO EMP_INFO (NAME, PHOTO) VALUES ('RAMANA', EMPTY_BLOB())
     RETURNING PHOTO INTO DEST_LOB;
  DBMS_LOB.open (SRC_LOB, DBMS_LOB.LOB_READONLY);
  DBMS_LOB.LOADFROMFILE (DEST_LOB, SRC_LOB,
     DBMS_LOB.GETLENGTH(SRC_LOB));
  DBMS_LOB.CLOSE (SRC_LOB);
END;
```

PRAGMA AUTONOMOUS_TRANSACTION:

Autonomous transaction defines any transaction as independent transaction from the parent transaction allows commit, rollback without any effecting of parent transaction.

In any procedure we write "PRAGMA AUTONOMOUS_TRANSACTION" between as and begin, and we write commit or rollback in code is works for that procedure only there is no chance to commit or rollback previous programs DML actions.

Because we used "PRAGMA AUTONOMOUS_TRANSACTION" transaction in that procedure means that commit or rollback is limited for corresponding procedure only.

DYNAMIC SQL:

Generally we can't perform DDL operations in the PL/SQL block if you want to perform DML operations in PL/SQL blocks we should go for the "Dynamic SQL".

DDL operations are auto commit. If PL/SQL allows DDL operations that DDL operation will be committed then previous operations also commit automatically that's why PL/SQL won't allow DDL operations. But we want to use DDL operation in PL/SQL block compulsory we will go for "Dynamic SQL". It keyword is "EXECUTE IMMEDIATE".

CURSOR

♦ IMPLICIT CURSOR:

If we need to know the DML operation status in PL/SQL block we should go for implicit cursor. Means if we write any DML operation in PL/SQL block and we executed that program even that statement is executed or not executed successfully then also we won't get any error we don't know exactly how many rows are affected. Even zero rows are effected or one row effected we don't know then if we want no that status of DML operation we need implicit cursor.

Ex:
```
BEGIN
  UPDATE EMP SET COMM=SAL*10/100 WHERE DEPTNO=30;
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
  DELETE FROM EMP WHERE JOB='SOFTWARE';
    IF SQL%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('RECORDS DELETED');
    ELSE
      DBMS_OUTPUT.PUT_LINE('NO RECORDS DELETED');
    END IF;
 END;
```

It has three attributes:

- %Found
- %Notfound
- %Rowcount

- If we use 'select into' class in implicit cursor. It gives only two values either 0 or 1. Because INTO clause gives values of only one row in PL/SQL block.
- It is only for status of DML operations or select into class.
- It is single type and "SQL%" is the key word for implicit cursor.

♦ EXPLICIT CURSOR:

Generally we cannot write select statement in the PL/SQL block if you want to write select statement in the PL/SQL block we should go for into clause. Into clause will allow fetching maximum one row I want to fetch more than one row I should for the cursor. Cursor is a private area created by PL/SQL block to store what is the data fetching by the cursor select statement.

Cursor has three models

✓ Basic cursor:

```
DECLARE
 CURSOR C1 IS SELECT EMPNO,ENAME,DNAME FROM EMP E, DEPT D
                 WHERE E.DEPTNO=D.DEPTNO;
 LN_EMPNO EMP.EMPNO%TYPE;
 LV_ENAME VARCHAR2(30);
 LV_DNAME DEPT.DNAME%TYPE;
 BEGIN
  OPEN C1;
   LOOP
    FETCH C1 INTO LN_EMPNO,LV_ENAME,LV_DNAME;
     EXIT WHEN C1% NOTFOUND;
DBMS_OUTPUT.PUT_LINE(LN_EMPNO||', '||LV_ENAME||', '||LV_DNAME);
   END LOOP;
  CLOSE C1;
 END;
```

✓ **Cursor for loop:**

```
DECLARE
  CURSOR C1 IS SELECT * FROM EMP;
BEGIN
  FOR I IN C1 LOOP
   DBMS_OUTPUT.PUT_LINE(I.EMPNO||', '||I.ENAME);
  END LOOP;
END;
```

✓ **Cursor for loop with select statement:**

```
BEGIN
 FOR I IN (SELECT * FROM DEPT) LOOP
 DBMS_OUTPUT.PUT_LINE(I.DEPTNO||', '||I.DNAME);
 END LOOP;
END;
```

- Even declaration section also not necessary for cursor for loop with select statement. We can directly write select statement in loop;

It has three attributes:

- %Found
- %Notfound
- %Rowcount
- %Isopen

♦ **REFERENCE CURSOR:**

Cursor is always associated with same result set but ref cursor can be assigned to different result sets. It has two types:

1. Weak Refcursor

```
TYPE TYP_REF IS REF CURSOR;
TYP TYP_REF;
```

Q: What is sys Refcursor..?
A: SYS_REFCURSOR; is pre defined data type. If you declare any variable for sys refcursor that variable you can open for any select statement.

2. Strong Refcursor

```
TYPE TYP_REF IS REF CURSOR RETURN EMP%ROWTYPE;
TYP TYP_REF;
```

✓ **Weak Refcursor:**

By using only one weak ref cursor we can assign to no. of select statements. We can able to join different tables in select statement.

Explicit cursor always associated with same result set but ref cursors can assign to different result sets. Means one ref cursor we can open for emp table for once and next time we can open same ref cursor for dept table data and again we can open same ref cursor for salgrade table data but explicit cursor is always open with same result set.

Ex:
```
DECLARE
  TYP SYS_REFCURSOR;
  VEC EMP%ROWTYPE;
  VED DEPT%ROWTYPE;

 BEGIN
  OPEN TYP FOR SELECT * FROM EMP;
    LOOP
      FETCH TYP INTO VEC;
      EXIT WHEN TYP%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(VEC.ENAME||' '||VEC.SAL);
    END LOOP;
  CLOSE TYP;

 OPEN TYP FOR SELECT * FROM DEPT;
    LOOP
      FETCH TYP INTO VED;
      EXIT WHEN TYP%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(VED.DEPTNO||' '||VED.DNAME);
    END LOOP;
 END;
```

✓ **Strong Refcursor:**

The difference between explicit cursor and strong refcursor is explicit cursor gives information every time with same select statement but strong refcursor able to change where condition every time.

The difference between explicit cursor, weak refcursor and strong refcursor is explicit cursor, weak refcursor allow joins in select statement but strong refcursor not allow joins because its key word is `RETURN EMP%ROWTYPE;`

Ex: For deference between Explicit, Strong Refcursor, Weak Refcursor.

```
DECLARE
 CURSOR C1 IS SELECT E.EMPNO,E.ENAME,E.JOB,D.DNAME
 FROM EMP E, DEPT D WHERE SAL>1000 AND E.DEPTNO=D.DEPTNO;

 TYPE TYP_REF IS REF CURSOR RETURN EMP%ROWTYPE;
 TYP TYP_REF;

 TYP2 SYS_REFCURSOR;
BEGIN
  OPEN C1;
  CLOSE C1;

  OPEN TYP FOR SELECT * FROM EMP WHERE SAL>2000;
  OPEN TYP FOR SELECT * FROM EMP WHERE DEPTNO=10;

  OPEN TYP2 FOR SELECT * FROM DEPT;
  OPEN TYP2 FOR SELECT * FROM EMP WHERE JOB IN 'MANAGER';

  OPEN TYP2 FOR SELECT E.EMPNO,E.ENAME,E.JOB,D.DNAME
  FROM EMP E, DEPT D WHERE E.DEPTNO IN (10,20)
  AND E.DEPTNO=D.DEPTNO;
END;
```

It has three attributes:

- %Found
- %Notfound
- %Rowcount
- %Isopen

CURSOR FOR UPDATE:

If we write for update along with cursor select statement exclusively row level lock will be applied for all the rows which are identified by that cursor select statement. Until we give commit no one cannot be modify those rows.

WHERE CURRENT OF:

Generally we cannot perform DML operations when we using cursor for update in our program if we want to perform DML operations on which are locked by the cursor for update the we should go for the "where current of".

Ex:

```
CREATE OR REPLACE PROCEDURE SP_FOR_UPDATE
AS
CURSOR C1 IS SELECT * FROM EMP FOR UPDATE;
VEC EMP%ROWTYPE;
BEGIN
  OPEN C1;
      LOOP
        FETCH C1 INTO VEC;
          EXIT WHEN C1%NOTFOUND;
        UPDATE EMP SET COMM=SAL*10/100 WHERE CURRENT OF C1;
      END LOOP;
  CLOSE C1;
END;
```

FUNCTIONS

If you excepting some values on the screen you can go for the select statement or you can go for functions. But directly you want to do some calculations and you want to bring the value on screen then you can go for the 'function'.

- It's named block and it is reusable.
- It is PL/SQL object, but we can call it in both SQL and PL/SQL.
- Some predefined function are there (Date, Aggregate, Char, Arithmetic functions etc..,).
- If we write a new function is called as user defined function.
- Function must return a value.
- If you want to write DML operations in functions we can write but these functions we cannot can in SQL. Then if we want to call those functions in SQL we should write "`pragma autonomous_transaction +TCL`".
- If we want to write out parameter for function we can use but these function not eligible to call in SQL. There is no medicine for this.

Ex_1: Normal function

```
CREATE OR REPLACE FUNCTION SF_EMPLOYEE (P_DEPTNO NUMBER)
RETURN SYS_REFCURSOR
AS
DATA SYS_REFCURSOR;
 BEGIN
  OPEN DATA FOR SELECT * FROM EMP WHERE DEPTNO=P_DEPTNO;
  RETURN DATA;
 END;
```

This normal function we can use in both SQL and PL/SQL, there no restrictions for this.

Note: SF means Stored Function.

Ex_2: Function with DML

```
CREATE OR REPLACE FUNCTION SF_COMM_UPDATE (P_EMPNO NUMBER,
P_COMM NUMBER) RETURN NUMBER
AS
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
 UPDATE EMP SET COMM=P_COMM WHERE EMPNO=P_EMPNO;
 COMMIT;
 RETURN SQL%ROWCOUNT;
END;
```

Note: If you write DML operations in function it is not eligible to use in SQL. But you are using DML operations with "`pragma autonomous_transaction+TCL`" it allowed in SQL.

Ex_3: Function with Out Parameter

```
CREATE OR REPLACE FUNCTION SF_NAME_SAL (P_EMPNO NUMBER,
P_SAL OUT NUMBER)
RETURN VARCHAR2
AS
LV_ENAME EMP.ENAME%TYPE;
 BEGIN
   SELECT ENAME,SAL INTO LV_ENAME,P_SAL FROM EMP WHERE
   EMPNO=P_EMPNO;
   RETURN LV_ENAME;
END;
```

Note: If you write out parameter in the function. It is not eligible in SQL there is no medicine for this.

- **PREDEFINED FUNCTIONS:**
  - Date functions
  - Char functions
  - Arithmetic functions
  - Aggregate functions
  - Analytical functions

- **PREDEFINED PACKAGED FUNCTIONS:**
  - **UTL_FILE.fopen;**
  - **DBMS_PROFILER.start_profiler;**
  - **DBMS_PROFILER.stop_profiler;**
  - **DBMS_UTILITY.format_error_backtrace;**
  - **DBMS_LOB**
  - **UTL_SMTP.mail;**
  - **DBMS_METADATA.get_ddl** etc..,

PROCEDURE

If you want to do some calculations but you never expert any value on the screen better to go for the "procedure". If you want to return a value in procedure then you should go for out parameter. Functions, procedure both are eligible to write return statement. But procedure return completely different when compared to function return.

Procedure return just it will terminate the program then it will go for end part but it don't bring any value on the screen. But function return must return a value on the screen.

- If you want to use any DML operations in the program better to go for "procedure".
- We cannot use procedures in SQL. We use out parameters for return values but these are also not allowed in SQL.
- There is no predefined procedure's.

Ex: Procedure without parameter

```
CREATE OR REPLACE PROCEDURE SP_COMM_CAL (P_DEPTNO NUMBER,
P_COMM_PER NUMBER,P_OUT_DATA OUT SYS_REFCURSOR)
AS
 BEGIN
  UPDATE EMP SET COMM=SAL*P_COMM_PER/100 WHERE DEPTNO=P_DEPTNO;
  COMMIT;
  OPEN P_OUT_DATA FOR SELECT * FROM EMP WHERE DEPTNO=P_DEPTNO;
 END;
```

We don't have any pre defined procedures. We have predefined packages in that we have some packaged procedures.

- **PREDEFINED PACKAGED PROCEDURE:**
  - **UTL_FILE.fgetattr;**
  - **UTL_FILE.putf;**
  - **UTL_FILE.fclose;**
  - **DBMS_MVIEW.refresh;**
  - **DBMS_JOB.submit;**
  - **DBMS_JOB.remove;**
  - **DBMS_UTILITY.compile_schema;**
  - **DBMS_LOB.open;**
  - **DBMS_LOB.loadfromfile;**
  - **DBMS_LOB.close;** etc..,

COMPOSITE VARIABLE TYPES

- **NESTED TABLE:**
  - ↳
```
TYPE TYP_NEST IS TABLE OF NUMBER;
TYP TYP_NEST;
```

- **ASSOCIATE ARRAY or PL/SQL TABLE:**
  - ↳
```
TYPE TYP_ASRY IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
TYP TYP_ASRY;
```

- **VARRAY:**
  - ↳
```
TYPE TYP_VAR IS VARRAY(100) OF NUMBER;
TYP TYP_VAR;
```

- **RECORD:**
  - ↳
```
TYPE TYP_REC IS RECORD(ENAME VARCHAR2(30),DNAME VARCHAR2(15));
TYP TYP_REC;
```
  - ↳ Record is nothing but collections of elements
  - ↳ Record is not a composite data type because record does not support bulk collect
  - ↳ Record brings only one row from any table
  - ↳ If you want to bring all rows by using bulk collect for record then should write a composite data type on record.
  - ↳ If you write nested table as sub data type for record then you can use bulk collect in record.

Ex:
```
DECLARE
  TYPE TYP_REC IS RECORD (EMPNO NUMBER, ENAME VARCHAR2(30),
                          DNAME VARCHAR2(15));
  TYPE TYP_NEST IS TABLE OF TYP_REC;
  TYP TYP_NEST;
  BEGIN
  SELECT E.EMPNO, E.ENAME, D.DNAME BULK COLLECT INTO TYP
  FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;
  END;
```

RETURNING CLAUSE:

Ex_1:
```
DECLARE
  X NUMBER;
BEGIN
    UPDATE EMP SET COMM=NVL (COMM,0)+SAL*20/100
                WHERE EMPNO=7369 RETURNING COMM INTO X;
END;
```

Ex_2:
```
DECLARE
  TYPE TYP_TAB IS TABLE OF VARCHAR2 (100);
  TYP TYP_TAB;
BEGIN
     DELETE FROM EMP WHERE DEPTNO=30
          RETURNING ENAME BULK COLLECT INTO TYP;
END;
```

METHODS OF COMPOSITE TYPES:

- **LAST, FIRST, COUNT:** These three methods support to all types of composite types.

```
TYP_NEST.COUNT
TYP_NEST.LAST
TYP_NEST.FIRST
```

COUNT	LAST
It will give number of elements in a collection	It will give largest index number
Ex: Suppose collection having 5 records, in that 3rd record deleted by using delete method then count will give 4	Ex: Suppose collection having 5 records, in that 3rd record deleted by using delete method then last will give 5

- **EXISTS:** This also supports to all composite data types, exist will give true or false only.

```
TYP_VAR TYP_VARRAY:= TYP_VARRAY (100,200,300);
   TYP_TAB TYP_PLSQL_TAB;
TYP_NEST TYP_NEST_TAB:= TYP_NEST_TAB (111,222,333,444);

---VARRAY
IF TYP_VAR.EXISTS(3) THEN TYP_VAR(3) := 999;

---PL/SQL TABLE
IF TYP_TAB.EXISTS(1) THEN TYP_TAB(1) := 888;
---THIS IS FAILED BECAUSE NO RECORDS IN PL/SQL TABLE DATATYPE

---NESTED TABLE
IF TYP_NEST.EXISTS(3) THEN TYP_NEST(3) := 666;
```

- **EXTEND:** This is support to nested, varray tables.
  There is no need of extend in PL/SQL table because its key word is index by binary integer.
  Note: Binary integer means infinitive.

```
--NESTED TABLE
    TYP_TEST.EXTEND;
    TYP_TEST (1):= 200;

--PL_SQL TABLE
    TYP_T (1):= 10;

--VARRAY
    TYP_V.EXTEND;
    TYP_V (1):= 20;
```

- **DELETE:** Delete method will support all composite types.
  But deleting particular/range data in varray is not possible. It will not support for that in varray.

TYP_NEST.DELETE:- Removes all elements from a collection
TYP_NEST.DELETE(10):- Removes the 10th element from a collection
TYP_NEST.DELETE(8,10):- Removes all elements in the range, 8th to 10th from a collection.

- **TRIM:** It is works for last value.
  It will not support to nested and varray tables.
  It does not support for PL/QL table.

```
TRIM:  Trim removes one element from the end of a collection
TRIM (N): Removes N elements from the end of a collection
TYP_NEST.TRIM:       --IT WILL DELETE LAST RECORD
TYP_NEST.TRIM(4):    --IT WILL DELETE LAST RECORD
```

- **LIMIT:** Limit method not useful for all composite types. It useful for only varray to show the limit of varray. Because there is no limit for nested and PL/SQL table, Varray is the only composite type having limit.

```
TYP_VAR.LIMIT
```

- **NEXT, PRIOR:** These are only useful for first box or last box numbers finding. Means if you pass a value number for NEXT then if the next box exist then gives the next box number else if the next box is not exist then gives null, if you pass a value number for PRIOR then if the previous box exist then gives the previous box number else if the previous box is not exist then gives null.

Ex:

```
DECLARE
   TYPE TYP_NEST IS TABLE OF VARCHAR2(10);
   TYP TYP_NEST:=TYP_NEST('ORANGE','APPLE','MANGO','GRAPS');
   BEGIN
      DBMS_OUTPUT.PUT_LINE('NEXT----'||TYP.NEXT(2));
                           -------OUT PUT =3 ;
      DBMS_OUTPUT.PUT_LINE('PRIOR------'||TYP.PRIOR(2));
               -------OUT PUT =1 ;

      DBMS_OUTPUT.PUT_LINE('NEXT----'||TYP.NEXT(4));
      DBMS_OUTPUT.PUT_LINE('PRIOR------'||TYP.PRIOR(1));
          --BOTH OF THE RESULTS-----OUT PUT IS NULL ;
   END;

               TYP_VAR.NEXT(3);
               TYP_VAR.PRIOR(3);

               TYP_NEST.NEXT(2);
               TYP_NEST.PRIOR(3);

               TYP_TAB.NEXT(2);
               TYP_TAB.PRIOR(3);
```

NOTE: Last record of collection **NEXT** value is always null;
First record of collection **PRIOR** values is always null;

MULTISET OPERATORS FOR COLLECTIONS

✓ **MULTISET UNION:**

If you want to mix two collection types data and place it in another collection type you should go for multiset union.

Ex:
```
TYP3:= TYP1 MULTISET UNION TYP2;
TYP3:= TYP1 MULTISET UNION DISTINCT TYP2;
```

✓ **MULTISET INTERSET:**

If you want to store common data in two collections into another collection then you should go for multiset intersect.

Ex:
```
TYP3:= TYP MULTISET INTERSECT TYP2;
```

✓ **MULTISET INTERSET:**

If you want check some data of one collection is found in another collection then go with sub multiset.

Ex:
```
IF TYP2 SUBMULTISET TYP THEN;
```

♦ **MEMBER OF:**

If any value in normal variable you want to check that the value is a member collection value then it useful. This ' member of ' is useful only in collections to check.

Ex:
```
X VARCHAR2(30):= 'YELLOW';
IF X NUMBER OF TYP THEN
 DBMS_OUTPUT.PUT_LINE('YES IT IS MEMBER OF TYP');
END IF;
```

BULK COLLECT

Bulk collect means PL/SQL engine tells to SQL engine to collect all the rows at once and place them into a composite variable by using only one context switch. Entire data will be moved from SQL engine to PL/SQL engine at once and we should have collection/composite variable.

Ex:
```
DECLARE
 TYPE TYP_TAB IS TABLE OF EMP%ROWTYPE;
 TYP TYP_TAB;
BEGIN
 SELECT * BULK COLLECT INTO TYP FROM EMP;
  FOR I IN 1..TYP.COUNT LOOP
   DBMS_OUTPUT.PUT_LINE(TYP(I).EMPNO||' '||TYP(I).ENAME);
  END LOOP;
END;
```

- **BULK BIND:**

  It will bind all the DML operations and moves from PL/SQL engine SQL engine at one-shot. If you write DML operation inside the collection for loop each DML operation consider as a separate DML operation then each DML operation create one context switch for execution purpose if loop is running 10lac times 10lac context switches will be created from Pl/SQL to SQL.

- **BULK LIMIT:**

  Bulk limit divide the data into pieces we can do the process with each piece of data. Generally collections are stored in the RAM. If table having massive data then there is chance to storage error/system hang because of memory to avoid that we can go for the bulk limit.

- **Difference between Bulk collect and Bulk bind:**
  - Bulk collect will work from SQL engine to PL/SQL engine.
    Bulk bind will work from PLSQL engine to SQL engine.
  - Bulk collect we will write along with select statement
    Bulk bind we will write when we are writing DML operation inside collection. That time only we can use bulk bind.

PACKAGES

Package is nothing but a collection of programs that programs should be related programs. Once you call any packaged program then entire package will come and sit in the RAM until session end (session end means login and logout). Package contains two parts those are spec, body.

- Spec nothing but a specification section of the package, if you declare any variable, cursor or collection you should write in spec.
- Body means whatever the functions, procedures you want write in that package you should write in the package body only.
- Without body you can write spec but without spec you cannot write body.

- If you declare any variable/cursor/collection/function/procedure in the package spec those are become global. Those you can use in that package body or anywhere in the schema.
- If you declare any variable directly in package body it will be a private variable that's global for that package body only you cannot use that out of the package body in the schema.
- If you declare two variables 1st one in spec and 2nd one in body with same name, if call that variable system will give first preference to local variable which you have declared in package body. And you want to call that spec declared variable you should write "package_name.variable _name" because we have two variables with same name we have to tell the system that variable only you should cal.
- Package has three special concepts:
  - ✓ One time procedure.
  - ✓ Forward declaration.
  - ✓ Function over loading.

  These three concepts available in packages only.

♦ FORWARD DECLARATION:

If you write any private program you can use it in package body only. You can't use outside the package body. But wherever you want use it in the body before that this program should be created this concept called as forward declaration.

♦ ONE TIME PROCEDURE:

At the end of the package body if you write begin and some statements it is called one time procedure. Here you should not write any END statement. If you want write exception in this you can write but you cannot write separate end for that.

Means if you want to put a restriction on package then you should go for one time procedure and you should write it at end of the package body.

Ex:
```
BEGIN
  IF TO_CHAR (SYSDATE, 'HH24')>23 THEN
 RAISE_APPLICATION_ERROR (-20450,'YOU CANNOT INSERT AT THIS TIME');
  END IF;
```

Note: If any application shows error notification that capacity only '**raise_application_error**' have.

♦ FUNCTION OVER LOADING:

If we are creating more than one program with same name but number of parameters should different. Then that program runs based on the parameters. Means if you create three procedures with same name and 1st procedure contain two parameters, 2nd procedure contain three parameters, 3rd procedure contain four parameters. Then if you run the procedure with three parameters then 2st procedure will be executed else if run the procedure with four parameters than 3rd procedure will be executed.

Best example for this cancelation or expire. If user cancels the transaction parameters required to canceling the transaction, if system expires transaction there is no need of parameters but the program output is same, both the programs are working for cancel the transaction only.

❖ EXAMPLE:

```
-----------------------------------------------PACKAGE SPECIFICATION:
CREATE OR REPLACE PACKAGE PKG_INSERTS AS
LV_TYP VARCHAR2(4);
LN_NUM NUMBER(5);
LV_CUST_ID VARCHAR2(10);

FUNCTION SF_CUST_ID_GEN (P_LOAN VARCHAR2) RETURN VARCHAR2;
FUNCTION SF_TYPE_ID(P_LOAN VARCHAR2)RETURN NUMBER;
FUNCTION SF_CUST_ID(P_APPLI_ID VARCHAR2) RETURN VARCHAR2;

PROCEDURE SP_CUST_INFO(P_LOAN VARCHAR2,P_APPLI_ID VARCHAR2,P_CUST_NAME
VARCHAR2,P_CUST_DOB DATE,P_PST_ADD VARCHAR2,P_PMT_ADD VARCHAR2,P_JOB_TYPE
VARCHAR2,P_ANL_SAL NUMBER,P_JOB_SECTOR VARCHAR2,P_DISIGNATION VARCHAR2,
P_REQ_AMT NUMBER, P_EMI_DATE NUMBER,P_EMI_AMT NUMBER,P_TENURE NUMBER);

PROCEDURE SP_ACCT_DTLS(P_APPLI_ID VARCHAR2,P_ACT_NO NUMBER,P_IFSC VARCHAR2,
P_BANK VARCHAR2);

PROCEDURE SP_NOMMI_DTLS(P_APPLI_ID VARCHAR2,P_NAME VARCHAR2, P_DOB DATE,
P_REALATION VARCHAR2,P_PST_ADRS VARCHAR2,P_PMT_ADRS VARCHAR2,P_BNK
NUMBER,P_BIFSC VARCHAR2,P_BANKNAME VARCHAR2);

PROCEDURE SP_DOC_DTLS(P_APPLI_ID VARCHAR2,P_AADHAR NUMBER,P_ADAR_COPY
BFILE,P_PAN VARCHAR2,P_PAN_COPY BFILE,P_PAY BFILE,P_STAT BFILE,P_NOM_ADAR
NUMBER,P_NOM_ADAR_COPY BFILE,P_NOM_PAN VARCHAR2, P_NOM_PAN_COPY BFILE);

PROCEDURE SP_LOAN_APRL(P_APPLI_ID VARCHAR2);
END;
```

```
----------------------------------------PACKAGE BODY:
CREATE OR REPLACE PACKAGE BODY PKG_INSERTS AS

FUNCTION SF_CUST_ID_GEN (P_LOAN VARCHAR2) RETURN VARCHAR2 AS
BEGIN
    LV_TYP:= CASE WHEN P_LOAN='HOUSING LOAN' THEN 'HL%'
                  WHEN P_LOAN='GOLD LOAN' THEN 'GL%'
                  WHEN P_LOAN='PERSONAL LOAN' THEN 'PL%' END;

  SELECT MAX(CUST_ID) INTO LV_CUST_ID FROM LO_CUST_INFO WHERE CUST_ID LIKE
LV_TYP;

         IF LV_CUST_ID IS NULL THEN
           LV_CUST_ID := SUBSTR(LV_TYP,1,2)||'00001';
           RETURN LV_CUST_ID;

      ELSIF LV_CUST_ID IS NOT NULL THEN
  SELECT SUBSTR(MAX(CUST_ID),3,5)+1 INTO LN_NUM FROM LO_CUST_INFO
                                                WHERE CUST_ID LIKE LV_TYP;
   LV_CUST_ID :=SUBSTR(LV_TYP,1,2)||SUBSTR('0000'||LN_NUM,-5,5);
   RETURN LV_CUST_ID;
  END IF;
  END;


FUNCTION SF_CUST_ID(P_APPLI_ID VARCHAR2) RETURN VARCHAR2 AS
BEGIN
     SELECT CUST_ID INTO LV_CUST_ID FROM LO_CUST_INFO WHERE
     APPLI_ID=P_APPLI_ID;
          RETURN LV_CUST_ID;
END;


FUNCTION SF_TYPE_ID(P_LOAN VARCHAR2)RETURN NUMBER AS
BEGIN
  SELECT TYPE_ID INTO LN_NUM FROM MST_LOAN_TYPE WHERE LOAN_NAME=P_LOAN;
  RETURN LN_NUM;
  END;


PROCEDURE SP_CUST_INFO(P_LOAN VARCHAR2,P_APPLI_ID VARCHAR2, P_CUST_NAME
VARCHAR2,P_CUST_DOB DATE,P_PST_ADD VARCHAR2,P_PMT_ADD VARCHAR2,P_JOB_TYPE
VARCHAR2,P_ANL_SAL NUMBER,P_JOB_SECTOR VARCHAR2,P_DISIGNATION VARCHAR2,
P_REQ_AMT NUMBER, P_EMI_DATE NUMBER,P_EMI_AMT NUMBER,P_TENURE NUMBER)
AS
  BEGIN

INSERT INTO LO_CUST_INFO VALUES
(SF_CUST_ID_GEN(P_LOAN),SF_TYPE_ID(P_LOAN),P_APPLI_ID,SYSDATE,P_CUST_NAME,
P_CUST_DOB,P_PST_ADD,P_PMT_ADD,P_JOB_TYPE,P_ANL_SAL,P_JOB_SECTOR,P_DISIGNATI
ON,P_REQ_AMT,P_EMI_DATE,P_EMI_AMT,P_TENURE,NULL,'P');
  END;
```

```
PROCEDURE SP_ACCT_DTLS(P_APPLI_ID VARCHAR2,P_ACT_NO NUMBER,P_IFSC VARCHAR2,
P_BANK VARCHAR2)
  AS BEGIN
      INSERT INTO LO_CUST_ACCT_DTLS VALUES
      (SF_CUST_ID(P_APPLI_ID),P_ACT_NO,P_IFSC,P_BANK,'P');
     END;


PROCEDURE SP_NOMMI_DTLS(P_APPLI_ID VARCHAR2,P_NAME VARCHAR2, P_DOB DATE,
P_REALATION VARCHAR2,P_PST_ADRS VARCHAR2,P_PMT_ADRS VARCHAR2,P_BNK
NUMBER,P_BIFSC VARCHAR2,P_BANKNAME VARCHAR2)
  AS BEGIN
       INSERT INTO LO_CUST_NOMMI_DTLS VALUES
      (SF_CUST_ID(P_APPLI_ID),P_NAME,P_DOB,P_REALATION,P_PST_ADRS,
      P_PMT_ADRS,P_BNK,P_BIFSC,P_BANKNAME,'P');
  END;

PROCEDURE SP_DOC_DTLS (P_APPLI_ID VARCHAR2,P_AADHAR NUMBER,P_ADAR_COPY
BFILE,P_PAN VARCHAR2,P_PAN_COPY BFILE,P_PAY BFILE,P_STAT BFILE,P_NOM_ADAR
NUMBER,P_NOM_ADAR_COPY BFILE,P_NOM_PAN VARCHAR2, P_NOM_PAN_COPY BFILE)

  AS BEGIN
       INSERT INTO LO_CUST_DOC_DTLS VALUES
      (SF_CUST_ID(P_APPLI_ID),P_AADHAR,P_ADAR_COPY,P_PAN,P_PAN_COPY,P_PAY,
      P_STAT,P_NOM_ADAR,P_NOM_ADAR_COPY,P_NOM_PAN,P_NOM_PAN_COPY,'P');
  END;

PROCEDURE SP_LOAN_APRL(P_APPLI_ID VARCHAR2) AS
  BEGIN
      INSERT INTO LO_LOAN_APPROVALS(CUST_ID,STATUS)
      VALUES(SF_CUST_ID(P_APPLI_ID),'P');
  END;

END;
```

- **PREDEFINED PACKAGES:**
  - **DBMS_PROFILER**
  - **DBMS_JOB**
  - **DBMS_MVIEW**
  - **DBMS_UTILITY**
  - **UTL_FILE**
  - **DBMS_LOB**
  - **UTL_SMTP**
  - **DBMS_METADATA** etc.,

DBMS_PROFILER

It is useful know the performance our program taking even it is procedure (or) function. It is used get information like how many times each statement is executing how much time it is taken to execute that statement.

- **DBMS_PROFILER** is a predefined package. We should need one view and one table for this process they are:
  - `SELECT * FROM ALL_SOURCE;`
  - `SELECT * FROM PLSQL_PROFILER_DATA;`

- ```
  BEGIN
    DBMS_PROFILER.start_profiler('SP_DATA_LOADINNG');
    SP_DATA_LOADINNG;
    DBMS_PROFILER.stop_profiler;
  END;
  ```

- Here
  - **start_profiler, stop_profiler** are packaged procedures in the "**DBMS_PROFILER**" package.
  - **SP_DATA_LOADINNG** is a individual profiler which is we want to know the run time.

- After executing this then we want to check the result then we want to result we should go for that two tables join like:

  Ex:
  ```
  SELECT A.TEXT, A.line, P.TOTAL_OCCUR, P.TOTAL_TIME/1000
    FROM ALL_SOURCE A, PLSQL_PROFILER_DATA P
    WHERE A.line=P.LINE# AND A.name='SP_DATA_LOADINNG';
  ```

TO GET PL/SQL TABLES:

- Open my computer
- Open which drive you are installed oracle
- Type "proftab.sql" in search
- Then open the proftab folder, select all and copy.
- Past it in SQL command window
- Automatically three tables created. Those are
  - **PLSQL_PROFILER_DATA;**
  - **PLSQL_PROFILER_RUNS;**
  - **PLSQL_PROFILER_UNITS**
  ```

UTL_FILE

UTL_FILE is a predefined package which is used to import or export data from database to OS (or) OS to database

Ex_1: To export data from database to OS

```
CREATE OR REPLACE PROCEDURE SP_OBJECT_LIST
AS
FO UTL_FILE.file_type;
LV_FILE VARCHAR2(30):='OBJECT_LIST_'||TO_CHAR(SYSDATE,
                        'DD_MM_YYYY')||'.CSV';
TYPE TYP_TAB IS TABLE OF VARCHAR2(100);
TYP TYP_TAB;

BEGIN
FO:= UTL_FILE.fopen('DATA_PUMP_DIR',LV_FILE,'W',400);
UTL_FILE.putf(FO,'%s\n','OBJECT_NAME,OBJECT_COUNT');

SELECT OBJECT_TYPE||','||COUNT(*) BULK COLLECT INTO TYP
     FROM USER_OBJECTS GROUP BY OBJECT_TYPE;

FOR I IN 1..TYP.COUNT LOOP
     UTL_FILE.putf(FO,'%s\n',TYP(I));
END LOOP;
     UTL_FILE.fclose(FO);
END;
```

Ex_2: To import data from OS

```
DECLARE
 FT UTL_FILE.file_type;
 LV_DATA VARCHAR2(100);
 X NUMBER:= 0;

BEGIN
 FT:= UTL_FILE.fopen('DATA_PUMP_DIR','RAMANA.CSV','R',4000);
     LOOP
       BEGIN
     UTL_FILE.get_line(FT,LV_DATA);
     LV_DATA:= CHR(39)||REPLACE(LV_DATA,',',''',''')||CHR(39);

           IF X>0 THEN
  LV_DATA:='INSERT INTO RAMANA VALUES('||LV_DATA||')';
     EXECUTE IMMEDIATE LV_DATA;
          END IF;
       EXCEPTION WHEN NO_DATA_FOUND THEN EXIT;
       END;
                X:= X+1;
     END LOOP;
END;
```

Ex_3: To know file is existing or not in directory

```
DECLARE
 X BOOLEAN;
 Y INTEGER;
 Z BINARY_INTEGER;

BEGIN
 UTL_FILE.fgetattr('DATA_PUMP_DIR','EMP.TXT',X,Y,Z);
      IF X THEN
 DBMS_OUTPUT.PUT_LINE('YES THAT FILE IS EXISTS IN THE DIRECORY');
      ELSE DBMS_OUTPUT.PUT_LINE( 'FILE NOT EXISTS');
      END IF;
END;
```

Ex_4: Combination of fgetattr and get_line:

```
CREATE OR REPLACE PROCEDURE SP_DATA(P_FILE VARCHAR2)
AS
  X BOOLEAN;
  Y INTEGER;
  Z BINARY_INTEGER;

FT UTL_FILE.file_type;
LV_DATA VARCHAR2(100);
A NUMBER:=0;

BEGIN
 UTL_FILE.fgetattr('DATA_PUMP_DIR',P_FILE,X,Y,Z);
       IF X= FALSE THEN RETURN;
   ELSIF X= TRUE THEN

FT:= UTL_FILE.fopen('DATA_PUMP_DIR',P_FILE,'R',4000);
       LOOP
         BEGIN
      UTL_FILE.get_line(FT,LV_DATA);
      LV_DATA:= CHR(39)||REPLACE(LV_DATA,',',''',''')||CHR(39);

            IF A>0 THEN
      LV_DATA:='INSERT INTO RAMANA VALUES ('||LV_DATA||')';
      EXECUTE IMMEDIATE LV_DATA;
            END IF;

EXCEPTION
      WHEN NO_DATA_FOUND THEN EXIT;
END;
       A := A+1;
 END LOOP;
      END IF;
END;
```

DBMS_JOB

It will automatically execute a program based on given interval. If you want execute a program for every hour/every day/every month then better to go for job. If you create a job on a program and provide a suitable interval for that job it will automatically execute the program every time by following the interval period you given.

Ex:
```
DECLARE
 JOB_NO NUMBER;
BEGIN
   DBMS_JOB.submit(JOB_NO,'BEGIN SP_DMCC_PROFIT;END;',
         TRUNC(SYSDATE)+7/24,'SYSDATE+1');
   COMMIT;
END;
```

To stop job:
```
BEGIN
   DBMS_JOB.broken(23,TRUE);
   COMMIT;
END;
```

To restart stopped job:
```
BEGIN
    DBMS_JOB.broken(23,FALSE);
    COMMIT;
END;
```

To remove job permanently:
```
BEGIN
  DBMS_JOB.remove(23);
  COMMIT;
END;
```

To run a job immediately:
```
BEGIN
  DBMS_JOB.run(24) ;
  COMMIT;
END;
```

But it will change the schedule of that job. For this situation better to do like "`BEGIN SP_DATA_VALIDATION; END;`"

To change interval:
```
BEGIN
  DBMS_JOB.interval(24,'ADD_MONTHS(SYSDATE,1)');
  COMMIT;
END;
```

But it changes the interval of that job if want to run a job immediately but without changing the job interval then you should go for test by right clicking on the program and go to test.

To see all the jobs you have created on a procedure:
```
SELECT * FROM USER_JOBS WHERE WHAT LIKE '%SP_DAILY_REPORT%';
```

TRIGGERS

Trigger is a validation project process which is occurs automatically without any explicit column. Trigger is a implicit object explicitly we have to create but works implicitly. We cannot run trigger forcefully it will be run based on the event. Triggers always fired based on the event.

Three types of triggers are there

- ✓ DDL Trigger.
- ✓ DML Trigger.
  - Statement level.
  - Row level trigger
  - Instead of trigger.
- ✓ Database Level Trigger.

♦ **DDL Trigger:**

This DDL trigger able create on any object or entire schema, after creating this trigger if we done any DDL action on the object/schema trigger will be fired then '**raise_application_error**' will be raised. When I want to put any rule or restriction on DDL operations others wise I want to know who is performing the DDL operation on the schema, in these both cases can go for DDL trigger it will be fires on the four events those are create, alter, drop and truncate.

Ex:
```
CREATE OR REPLACE TRIGGER TRG_DDL
    BEFORE DDL ON SCHEMA
     BEGIN
      RAISE_APPLICATION_ERROR
     (-20345, 'YOU CANNOT CEATE/ALTER/TRUNCATE/DROP IN THIS SCHEMA');
     END;
```

Ex:
```
CREATE OR REPLACE TRIGGER TRG_NEW
    BEFORE DDL ON SCHEMA
     BEGIN
      IF TO_CHAR (SYSDATE,'D') IN (1,7) THEN RAISE_APPLICATION_ERROR
       (-20300,'WEEKENDS YOU CANNOT DDL ON THIS SCHEMA');
      END IF;
     END;
```

If you want create a trigger for only one or more DDL commands on only one particular object in the schema. It is possible by using pseudo columns.

- ↳ **ora_dict_obj_name**
- ↳ **ora_dict_obj_type**
- ↳ **ora_sysevent**
- ↳ **ora_login_user**
- ↳ sysdate

These five are system defined key columns and these are useful when we want to trace who is doing DDL operations, which action they are doing and on which object they are doing in the schema .

Ex:
```
CREATE OR REPLACE TRIGGER TRG_EMP BEFORE DDL ON SCHEMA
    BEGIN
      IF ORA_DICT_OBJ_NAME='EMP' AND ORA_DICT_OBJ_TYPE='TABLE' AND
         ORA_SYSEVENT IN ('DROP','ALTER') THEN RAISE_APPLICATION_ERROR
                (-20400,'YOU CANNOT DROP/ALTER ON THIS TABLE');
      END IF;
    END;
```

♦ **DML TRIGGER:**

I want to do the validation after your going for the insert or update or delete on the table then you go for the DML trigger.

✓ **Statement Level trigger:**

Generally when we want to put a condition like 'weekends you can't insert or delete' or 'Fridays you can't update on the table then you can go for the statement level trigger. Statement level trigger mainly we use for avoid the transaction.

Ex_1:
```
CREATE OR REPLACE TRIGGER TRG_DEPT BEFORE DELETE ON DEPT
BEGIN
  DELETE FROM EMP;
END;
```

Ex_2:
```
CREATE OR REPLACE TRIGGER TRG_EMP_3
BEFORE INSERT OR DELETE ON DEPT
BEGIN
  IF TO_CHAR (SYSDATE,'D')=1 OR TO_CHAR(SYSDATE,'DD')=1
     THEN RAISE_APPLICATION_ERROR
     (-20545,'THESE DAYS YOU DELETE/INSERT ON DEPT');
  END IF;
END;
```

✓ **Row Level trigger:**

Row level trigger mainly we are using to maintain the audit. If you want to write rule or restriction for particular rows data else if we are doing update statement what is the previous value, what is the value after update, if you want to maintain the audit for each DML operation then you should go for the row level trigger.

Ex_1: For rule or restriction on each row
```
CREATE OR REPLACE TRIGGER TRG_DEPT
BEFORE DELETE OR INSERT ON DEPT
FOR EACH ROW
BEGIN
  IF :OLD.DEPTNO =30 THEN
    DELETE FROM EMP WHERE DEPTNO=:OLD.DEPTNO;

   ELSIF :OLD.DEPTNO <>30 THEN
    RAISE_APPLICATION_ERROR
       (-20345,'THIS DEPT TABLE DATA YOU CANNOT DELETE');

   ELSIF INSERTING AND TO_CHAR (SYSDATE,'D') IN (1,7) THEN
    RAISE_APPLICATION_ERROR
       (-20600, 'WEEKEMDS YOU CANNOT INSERT ON DEPT');
    END IF;
END;
```

Ex_2: Trigger for audit table maintenance for DML operation

```
CREATE TABLE AUDIT_EMP (EMPNO NUMBER, OLD_SAL NUMBER, NEW_SAL
NUMBER, OLD_COMM NUMBER,NEW_COMM NUMBER, OLD_JOB VARCHAR2(30),
NEW_JOB VARCHAR2(30), ACTION_DATE DATE,ACTION_TYPE
VARCHAR2(30));
```

```
CREATE OR REPLACE TRIGGER TRG_AUDIT_EMP
AFTER DELETE OR INSERT OR UPDATE ON EMP
FOR EACH ROW
 DECLARE
  X NUMBER;
  Y VARCHAR2(30);
 BEGIN
       X:=CASE WHEN :NEW.EMPNO IS NULL THEN :OLD.EMPNO ELSE
  :NEW.EMPNO END;
       Y:=CASE WHEN INSERTING THEN 'INSERT' WHEN UPDATING
       THEN 'UPDATE' ELSE 'DELETE' END;
 INSERT INTO AUDIT_EMP VALUES(X,:OLD.SAL,:NEW.SAL,:OLD.COMM,
       :NEW.COMM,:OLD.JOB,:NEW.JOB, SYSDATE, Y);
 END;
```

✓ **Instead of trigger:**

Generally complex views won't allow DML operations if you want to perform the DML operation on complex views then you should go for the instead of trigger.

- It's mainly for allow DML operations in complex views.
- There is no before/after condition this only ' instead of ' is there.
- There is no chance for mutating/recursive errors in this.

View:

```
CREATE VIEW VW_EMP_DEPT AS SELECT EMPNO,ENAME,JOB,SAL,E.DEPTNO,
DNAME,LOC FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;
```

Instead of trigger:

```
CREATE OR REPLACE TRIGGER TRG_COMPLEX
INSTEAD OF UPDATE ON VW_EMP_DEPT FOR EACH ROW
BEGIN

UPDATE EMP
SET ENAME=:NEW.ENAME, EMPNO=:NEW.EMPNO,JOB=:NEW.JOB,SAL=:NEW.SAL,
DEPTNO=:NEW.DEPTNO WHERE EMPNO=:OLD.EMPNO;

UPDATE DEPT
SET DNAME=:NEW.DNAME, DEPTNO=:NEW.DEPTNO,LOC=:NEW.LOC
WHERE DEPTNO=:OLD.DEPTNO;
END;
```

❖ **Creating trigger is disable mode:**

```
CREATE TRIGGER TRG_X DISABLE...
```

Then the trigger will be created in disable mode after that you can enable the triggers when you want to use the trigger. Before 11g we have done like this "`alter table trg_1 disable`".

♦ DATABASE LEVEL TRIGGER:

Mostly DBA only use this database level trigger to put restrictions on entire database like drop/create the schema not allow in the schema and maintain the audit like who is login into schema. If we want to do all these things then only you should go for database level trigger.

> **NOTE:** Already we have constraints for rule or restriction but these are the limited for some rules and restriction only. If you want to put more restrictions on based on the client requirement and maintain the audit then you should go for trigger only.

♦ COMPOUND TRIGGER:

If you can write more than one trigger at one area is called "compound trigger". (allows min:1 , max:4);

It allows only four conditions:

1. One before statement level
2. One before row level
3. One after row level
4. One after statement level.

❖ TRIGGERS FOLLOWS / PRECEDES CLAUSE:

Follows and precedes are the key words which is used to set a order for trigger firing when all are before or all are after. When we create no. of before after triggers and you want fire all these triggers in a particular order then you should go for 'follows' or 'precedes'.

Follows tells should fire after this trigger will be fired.

Follows tells should fire before this trigger will be fired.

❖ MUTATING ERROR:

Mutating error occur only in row level triggers on which table you are performing the DML operation same table if you are using inside the trigger body. Then you will get **mutating error.** If you want to resolve this then you should go for "`pragma autonomous_transaction +TCL`".

Means: Mutating error (ORA-04091) occurs whenever a row level trigger tries to modify or select data from the table that is already undergoing change.

Ex:

```
CREATE OR REPLACE TRIGGER TRG_UPDATE
 BEFORE UPDATE ON EMP
 FOR EACH ROW
   DECLARE
   X NUMBER;
   PRAGMA AUTONOMOUS_TRANSACTION;
 BEGIN
   SELECT COUNT(*) INTO X FROM EMP;
   COMMIT;
 END;
```

❖ RECURSIVE ERROR:

If we are writing trigger on an action and we are using same action in the trigger body we will get **recursive error.**

Note: Recursive error occur in row level trigger by (insert/insert)

And statement level trigger by (insert/insert, update/update, delete/delete).

EXCEPTIONS

An exception is an event which occurs during the execution of a program. It is used to resolve the run time errors.

Errors are two types

1. Compilation error (syntax error).
2. Runtime error (exceptions).

Exceptions are three types:

- Pre defined exception.
- User defined exception.
- Non predefined / User named exception.

♦ PREDEFINED EXCEPTION:

↳ **no_data_found:**

When select into clause not fetch any record (or) When select statement is unable to retrieve data into PL/SQL variables.

↳ **Too_many_rows:**

Select into fetched more records or when select statement is retrieve more than one row into PL/SQL table.

↳ **Zero divide:**

When user try to divide zero with any number (simply: when divisor =0);

↳ **Value error:**

When data types are not matched or when we are trying to store larger value into smaller variable.

↳ **Cursor already open:**

When without closing cursor try to open cursor again.

```
EXCEPTION
      WHEN CURSOR_ALREADY_OPEN THEN CLOSEC1;
      WHEN INVALID_CURSOR THEN OPEN C1;
END;
```

↳ **Invalid cursor:**

When user try to fetch rows without opening the cursor or when we are trying to open the cursor with a wrong name.

↳ **Program error:**

It occurs when internal SQL or PL/SQL error happens means background processor (BG, LGWR etc.,) are not working properly.

↳ **Storage error:**

When server is out of memory.

When collection exceeds RAM memory.

↳ **Dup_val_on_index:**

When we are trying to insert duplicate values in unique index columns.

↳ **Time_out_on_resorces:**

Activated when user performs infinite loop process.

↳ **When others:**

It will handle any type of oracle runtime error this exception you have to write end of all the exceptions.

♦ **USER DEFINED EXCEPTION:**

If user will raise an error on their requirement or based on their client business logic that is known as user defined "user defined exception".

It has two types they are:

1. **RAISE APPLICATION ERROR:**

Declare section is not required and exception handling section is not required. Program will terminate their itself, it won't give a chance to execute next statement and it will throw a message to the screen. It will roll back all previous DML operations.

Ex:
```
BEGIN
IF P_DEPTNO=10 THEN
RAISE_APPLICATION_ERROR
      (-20300, 'FOR THIS WE SHOULD NOT GIVE COMM');
END IF;
END;
```

2. **RAISE:**

Declare section is required and exception section is required it will stop the program then it will go to exception part.

Ex:
```
CREATE OR REPLACE PROCEDURE SP_COMM_UPDATE(P_DEPTNO NUMBER,
    P_COMM NUMBER)
 AS
EX EXCEPTION;
DX EXCEPTION;
LN_COMM NUMBER;
 CURSOR C1 IS SELECT * FROM EMP
                  WHERE DEPTNO=P_DEPTNO;
VEC EMP%ROWTYPE;

BEGIN
  OPEN C1;
    LOOP
      FETCH C1 INTO VEC;
      EXIT WHEN C1%NOTFOUND;
    BEGIN
       LN_COMM:= VEC.SAL*P_COMM/100;
     IF LN_COMM<100 THEN RAISE EX;
      ELSIF LN_COMM>1000 THEN RAISE DX;
      ELSIF LN_COMM IS NULL THEN RAISE EX;
      ELSE UPDATE EMP SET COMM=LN_COMM WHERE EMPNO=VEC.EMPNO;
     END IF;

    EXCEPTION
          WHEN EX THEN
    UPDATE EMP SET COMM=100 WHERE EMPNO=VEC.EMPNO;
          WHEN DX THEN
    UPDATE EMP SET COMM=1000 WHERE EMPNO=VEC.EMPNO;
     END;
       END LOOP;
   CLOSE C1;
END;
```

♦ **NON PREDEFINED EXCEPTION:**

Oracle will raise 10,000 - 1,00,000 errors except (10-15 user predefined errors) these type of error exception is called as non predefined/ user named exception. These are not called as user defined exceptions because these not raised user/client these errors will raised by system and there is no predefined exception for this error that the user will create a exception and provide a name for these kind of errors. That's why these are called as non predefined/ user named exception.

Ex:
```
CREATE OR REPLACE PROCEDURE SP_ADD_COLUMN
 AS
 LV_QUERY VARCHAR2(1000);
 CURSOR C1 IS SELECT TABLE_NAME FROM TABS;
 EX EXCEPTION;
 PRAGMA EXCEPTION_INIT (EX,-1430);
BEGIN
          FOR I IN C1 LOOP
   BEGIN
  LV_QUERY:='ALTER TABLE '||I.TABLE_NAME||' ADD STATUS NUMBER(10)';
  EXECUTE IMMEDIATE LV_QUERY;
  EXCEPTION
    WHEN EX THEN NULL;
    END;
          END LOOP;
 END;
```

BULK EXCEPTION:

When we are going for the bulk bind concept we have to do multiple operations at one shot then there is a chance to get bulk of errors at a time. So if we want to catch those errors then we should go for bulk exception.

↳ **Step_1:**

```
CREATE TABLE BULK_EXP_ERRORS (MODULE_NAME VARCHAR2 (15),
ERROR_DATE DATE, ERROR_NO VARCHAR2 (10), ERROR_MSG VARCHAR2 (512),
ROW_NUMBER NUMBER);
```

↳ **Step_2:**

```
CREATE OR REPLACE PROCEDURE SP_BULK_ERROR(P_MODULE VARCHAR2,
          P_ERROR_NO VARCHAR2,P_ERROR_MSG VARCHAR2, P_ROW_NO NUMBER)
AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
     INSERT INTO BULK_EXP_ERRORS
     VALUES (P_MODULE, SYSDATE, P_ERROR_NO, P_ERROR_MSG, P_ROW_NO);
COMMIT;
END;
```

↳ **Step_3:**

```
CREATE OR REPLACE PROCEDURE SP_BULK_EXP
AS
  TYPE TYP_NEST IS TABLE OF EMP%ROWTYPE;
  TYP TYP_NEST;
  CURSOR C1 IS SELECT * FROM EMP;
  EX EXCEPTION;
  PRAGMA EXCEPTION_INIT(EX, -24381);
  LN_COUNT NUMBER:=0;

BEGIN
  OPEN C1;
     LOOP
     FETCH C1 BULK COLLECT INTO TYP LIMIT 4;
     EXIT WHEN TYP.COUNT=0;
   BEGIN
     FORALL I IN 1..TYP.COUNT SAVE EXCEPTIONS
          INSERT INTO EMP_2 VALUES TYP(I);
     LN_COUNT:=LN_COUNT+4;
   EXCEPTION
     WHEN EX THEN
         FOR I IN 1.. SQL%BULK_EXCEPTIONS.COUNT LOOP
       LN_COUNT:=LN_COUNT+SQL%BULK_EXCEPTIONS(I).ERROR_INDEX;
         SP_BULK_ERROR ('SP_BULK_EXP', SQLCODE,
            SQLERRM (-SQL%BULK_EXCEPTIONS(I).ERROR_CODE), LN_COUNT);
       LN_COUNT:=LN_COUNT+1;
         END LOOP;
         END;
     END LOOP;
  CLOSE C1;
END;
```

ERROR HANDLING

If we are working for any project many programs we will write those can be functions, procedure, packages or triggers, each program should have a 'when others exception section we should not write any program why because in client location tomorrow we are getting any error we didn't know which procedure is fail. So by using when others exceptions immediately that error details you have to insert in error log table. So if you want to insert error details in error _log table after inserting you have to write commit but this commit should not effect on parent transaction.

That's why we have to written a separate procedure for this. to insert error details in error log table. That procedure should have a '`pragma autonomous_transaction`' and '`COMMIT`'. Because this procedure should insert errors details into error_log table that only should be committed this commit should not effect on the where this procedure calling in another procedure.

So when others exception after that we have to call this error log procedure when error is occur immediately this procedure is executes that error details insert into error log table.

Error log table mainly should have five columns "module_name, error_date, error_msg, error_number, line_number". After this procedure we have to write '**`raise_application_error`**'. Why because after inserting error details into error_log table then '**`raise_application_error`**' statement will executes and it will be roll back all previous DML operations.

So without writing error handling section in any procedure that procedure we won't send to the client.

ERROR LOG TABLE:

```
CREATE TABLE ERROR_LOG (MODULE_NAME VARCHAR2 (30), ERROR_DATE
DATE, P_ERROR_NO VARCHAR2 (15), ERROR_MSG VARCHAR2 (512),
ERROR_LINE_NO VARCHAR2(3));
```

ERROR LOG PROCEDURE:

```
CREATE OR REPLACE PROCEDURE SP_ERROR_LOG (P_MODULE_NAME VARCHAR2,
P_ERROR_NO VARCHAR2,P_ERROR_MSG VARCHAR2,P_LINE_NO VARCHAR2)
AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO ERROR_LOG VALUES
  (P_MODULE_NAME, SYSDATE, P_ERROR_NO, P_ERROR_MSG, P_LINE_NO);
COMMIT;
END;
```

EXCEPTION SECTION:

```
EXCEPTION
   WHEN OTHERS THEN
      SP_ERROR_LOG ('SP_COMM_CAL',SQLCODE,SQLERRM,
            SUBSTR (DBMS_UTILITY.format_error_backtrace,-5));
      RAISE_APPLICATION_ERROR(-20400,SQLERRM);
END;
```

DATA CONCURRENCY

Mostly we are using data concurrency for avoid the dead lock concept. If we have performed two different actions on same row in a table it leads get dead lock error to avoid that we will go for data concurrency concept. For data concurrency concept we will create a separate table and procedure in that procedure we have to write "`pragma autonomous_transaction + COMMIT`", if any transaction is going on any account that account number we should insert in a lock table. Once the transaction is completed then immediately we should delete the record from lock table.

When somebody trying to do any transaction then system should go and check in lock table is there any transaction is undergoing process or not. If lock table having any record for that account number that means 'yes there is some transaction is going under process'. If lock table didn't having any record for that account number they do their transaction successfully there is no issue. If you we use data concurrency concept in this kind situation we can easily avoid the dead lock error.

Step_1:

```
CREATE TABLE LOCK_TAB (ID_NO NUMBER);
```

Step_2:

```
CREATE OR REPLACE PROCEDURE SP_DATA_CON(P_EMPNO NUMBER)
AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
   INSERT INTO LOCK_TAB VALUES (P_EMPNO);
   COMMIT;
END;
```

Step_3:

```
CREATE OR REPLACE PROCEDURE SP_CON_DELETE(P_EMPNO NUMBER)
AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
   DELETE FROM LOCK_TAB WHERE ID_NO=P_EMPNO;
   COMMIT;
END;
```

Step_4: Withdrawal procedure

```
CREATE OR REPLACE PROCEDURE SP_WITHDRAW(P_EMPNO NUMBER,P_AMT NUMBER)
AS
  LN_COUNT NUMBER;
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
   SELECT COUNT (*) INTO LN_COUNT FROM LOCK_TAB WHERE ID_NO=P_EMPNO;

        IF LN_COUNT>0 THEN
   RAISE_APPLICATION_ERROR
        (-20345,'SOME TRANSACTION IS UNDERGOING PLEASE WAIT FOR
             SOME TIME AND TRY AGAIN');
        ELSE
        SP_DATA_CON (P_EMPNO);
        END IF;
             UPDATE EMP SET SAL=SAL-P_AMT WHERE EMPNO=P_EMPNO;
             DELETE FROM LOCK_TAB WHERE ID_NO=P_EMPNO;
             COMMIT;
```

```
        EXCEPTION
           WHEN OTHERS THEN
            SP_CON_DELETE (P_EMPNO);
           SP_ERROR_LOG ('SP_WITHDRAW', SQLERRM,
                    SUBSTR (DBMS_UTILITY.format_error_backtrace,-3));
              RAISE_APPLICATION_ERROR (-20200, SQLERRM);
        END;
```

Depositor procedure

```
↳ CREATE OR REPLACE PROCEDURE SP_DEPOSIT(P_EMPNO NUMBER,P_AMT NUMBER)
  AS
    LN_COUNT NUMBER;
    PRAGMA AUTONOMOUS_TRANSACTION;
  BEGIN
     SELECT COUNT(*) INTO LN_COUNT FROM LOCK_TAB WHERE ID_NO=P_EMPNO;
     IF LN_COUNT>0 THEN
RAISE_APPLICATION_ERROR
          (-20345,'SOME TRANSACTION IS UNDERGOING PLEASE WAIT FOR
               SOME TIME AND TRY AGAIN');
     ELSE
     SP_DATA_CON(P_EMPNO);
     END IF;
          UPDATE EMP SET SAL=SAL-P_AMT WHERE EMPNO=P_EMPNO;
          DELETE FROM LOCK_TAB WHERE ID_NO=P_EMPNO;
        COMMIT;
 EXCEPTION
     WHEN OTHERS THEN
        SP_CON_DELETE (P_EMPNO);
        SP_ERROR_LOG ('SP_DEPOSIT', SQLERRM,
  SUBSTR (DBMS_UTILITY.format_error_backtrace,-3));
RAISE_APPLICATION_ERROR (-20200, SQLERRM);
END;
```

TABLE LEVEL FUNCTION:

If any function return type is collection then if you use that function as table is called 'table level function'.

Ex: `CREATE TYPE TYP_TAB IS TABLE OF NUMBER;`

- ↳ If you create a collection variable like this then it's become a global collection variable you can use it anywhere in the entire schema.
- ↳ It works like data type. It needs subtype to store data.
- ↳ Like

```
CREATE OR REPLACE FUNCTION SF_NUMBERS
RETURN TYP_TAB
AS
  TYP TYP_TAB:= TYP_TAB();
BEGIN
   FOR I IN 1000..9999 LOOP
     TYP.EXTEND;
     TYP(I)=I;
   END LOOP;
 RETURN TYP;
END;
```

↳ If we write a select statement for this function like

```
SELECT * FROM (SELECT * FROM (SELECT * FROM TABLE(SF_NUMBER))
ORDER BY DBMS_RANDOM.RANDOM) X WHERE ROWNUM=1;
```

This select statement will brings only one 4 digit number but it brings different numbers every time we executing this query. That "**DBMS_RANDOM.RANDOM**" is predefined package it useful to provide different numbers every time. This is mostly useful generate OTP in every project.

OBJECT TABLE:

Object is nothing but a collection of elements. It is also works like record whatever the columns you want to add you can add in this and it is also store maximum one record like record (collection).

↳
```
CREATE TYPE TYP_OBJ IS OBJECT (NAME VARCHAR2(30), SEAT_NO NUMBER);
```
↳
```
CREATE TYPE TYP_O IS TABLE OF  TYP_OBJ;
```

Note: Record is local variable, if you declare record becomes global. But every time you want to call that record at out of that package you have write like package. record.

But object is global by creation you can call it in anywhere in the schema by suing the object name.

Ex_1:

↳
```
CREATE OR REPLACE FUNCTION SF_EMP_DATA
RETURN TYP_O
AS
TYP TYP_O;
BEGIN
  SELECT TYP_OBJ(ENAME,EMPNO) BULK COLLECT INTO TYP FROM EMP;
  RETURN TYP;
END;
```

Ex_2:

↳
```
CREATE OR REPLACE PROCEDURE SP_OBJECT (P1 TYP_O, P2 OUT TYP_O)
AS
BEGIN
SELECT TYP_OBJ (ENAME,EMPNO) BULK COLLECT INTO X FROM EMP;
   FOR I IN P1.COUNT LOOP;
   DBMS_OUTPUT.PUT_LINE(P1(I).NAME ||' '||P1(I).SEAT_NO);
END LOOP;
END;
```

Ex_3: Use full for flight ticket in airline projects.

↳
```
CREATE TYPE TYP_VAR2 IS VARRAY (300) OF TYP_OBJ;
```
↳
```
CREATE SEQUENCE SEQ1 START WITH 5 INCREMENT BY 1 CYCLE MINVALUE 4
MAX VALUE 50;
```
↳
```
CREATE TABLE FLIGHT_INFO(FLIGHT_NO VARCHAR2(60),PASS_DTLS TYP_VAR2)
```
↳
```
INSERT INTO FLIGHT_INFO VALUES
('AIR_ASIA_301', TYP_VAR2(TYP_OBJ ('RAMANA',101),
TYP_OBJ('LUCKY',102), TYP_OBJ('MURTHY',103),TYP_OBJ('PRASUNA',104),
TYP_OBJ('GOVINDH',104),TYP_OBJ('MADHAVI',105)));
```

TK PROOF

If we want to know the system performance we have to generate trace files that trace files are not readable files if you open the trace file that will be displayed in binary format. Then if we want to read that trace file we have to convert that file into user readable format then we should go for TK proof.

- Open my computer and type " *.TRC " in search bar.
- Copy the trace file name.
- Then open prompt command.
- Type TKPROOF and paste the trace file name.

  Ex: TKPROOF tracefilename.trc.

- **Trace file:** Oracle will save the information trace file in certain time like last 30mins or last 2hrs how many programs has run and how much time they taken to run.

To start trace file generation:

- Open command window.
- `ALTER SYSTEM SET SQL_TRACE=TRUE;`

To start trace file generation:

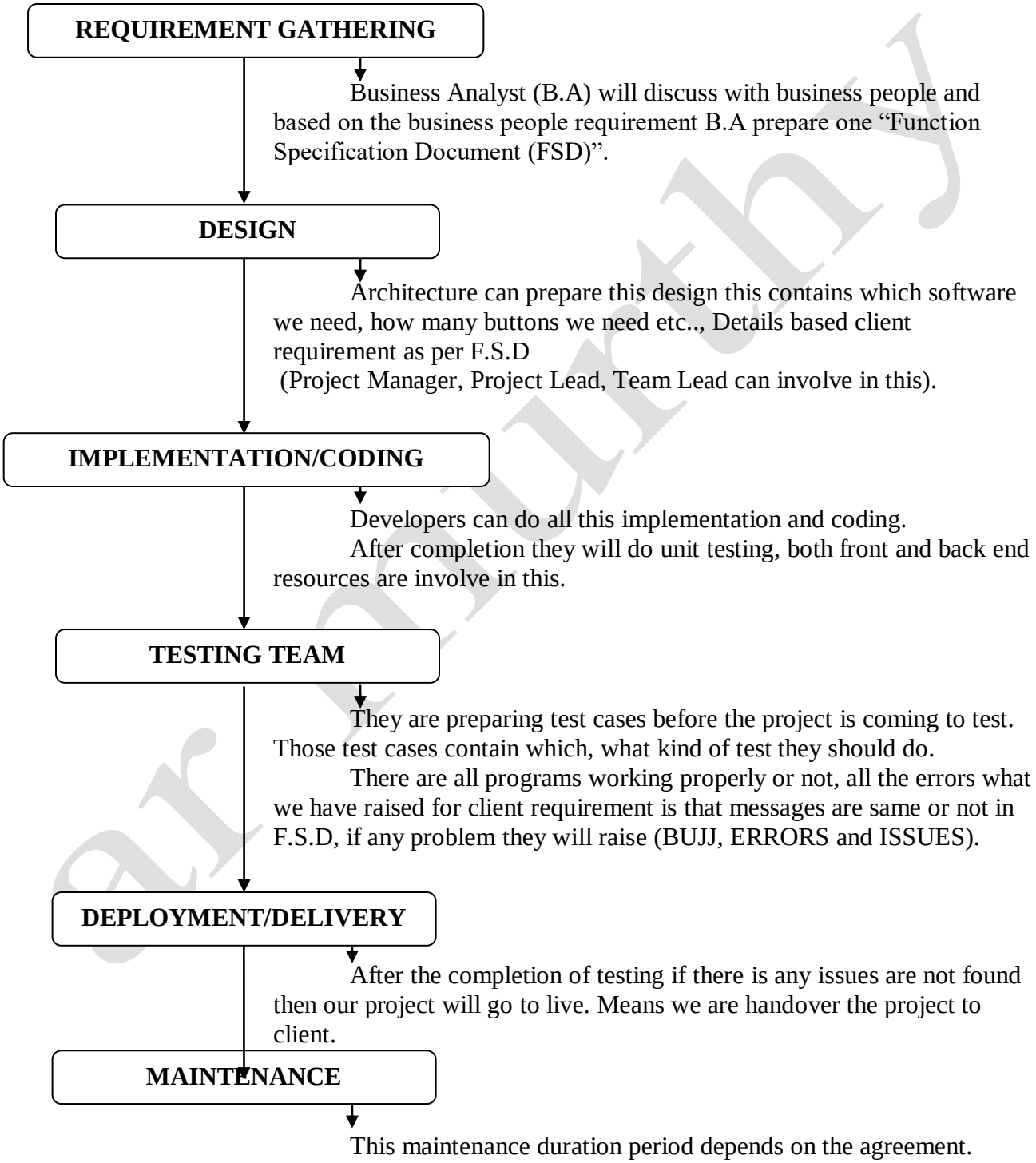
- Open command window.
- `ALTER SYSTEM SET SQL_TRACE=FALSE;`

Then if you want to see the log file open my computer and type " *.TRC " in search bar then trace files will be open but document format is not readable if you open the file it will be displayed in binary format.

System development cycle

There are three types of development cycles are there water fall model, zebra model and Azalea model, now a day's most of the companies fallowing azalea model.

AZALEA MODEL:



COMPARISONS

ROWID	ROW NUMBER
• Row id is static value (Always same row contain same row id)	• Row number is not a static value (Based on the output of select statement row number will dynamically change)
• Row id is a **HEXA DICIMAL** number (18 digit values)	• Row number is numeric value
• Row id is combination of (Object ID, File Number, Block Number, Row Number)	• Row number is numeric value
• Oracle automatically provide row id when user insert new row into the table	• Based on select statement output row number will generate
• Row id will be stored in the data base	• Row number won't store in database (temporarily it will generate for select statement output)

TRUNCATE	DELETE
• It removes all the rows from a table	• It can removes all rows or selected rows
• Where clause not support	• Where clause support
• After truncate we can't apply TCL commands (Means: auto commit) Because it is a DDL command, all DDL commands are auto commit	• After delete TCL is mandatory (commit/rollback/savapoint)
• Clears the memory allocated to rows (Internal memory will be clear) (Means: high watermark position will be change)	• After delete internal memory still exist (Means: high water mark in same position)
• Truncate = Delete + Commit	

MAX	GREATEST
• Max is aggregate function	• Greatest is arithmetic function
• Max will picks the largest value from given column and return in single row	• Greatest will picks the highest values from given columns and return in a single column
• Max can be used to go down the column	• Greatest can be used across the row
• Max gives one value in single row	• Greatest gives multiple rows and return single column from given multiple columns of data

HAVING	WHERE
• Once the data is grouped then we want put a condition on that data we should go for the having clause	• When want to put a condition on row data then we should go for where clause
• Having is filters grouped data	• Where is filters rows data
• Having supports with aggregate functions	• Where is does not supports while aggregate functions used

SYNONYM	VIEW
• Synonym hide the original name of the object It is alias name of the object	• View is stored select statement
• Always we can create synonym on a single object	• We can create view on multiple objects
• Synonym can create on views, tables, procedures, functions	• We can create views on tables, views, synonyms
• It will not hold the data physically on it	• It will not hold the data physically on it
• Whatever the changes your doing in original object it will impact on synonym, Whatever the changes your doing on the synonym that will impact on original object	• Whatever the changes your doing in original object it will impact on view, Whatever the changes your doing on the view that will impact on original object
• Select * from user_synonyms	• Select * from user_views

CASE	DECODE
• Case is statement and it is used to provide the if then else type of logic	• Decode is a function and it is used to provide if then else type of logic
• Case we can use inside the select statement and without select statement also we can use	• Inside the select statement only we can use decode, without select statement we cannot use decode
• Supports operators like Ex:>,<, >=,=<,=, in, between, like, not like	• Decode does not support that kind of operators
• By using case we can directly assign a value to variable in PL/SQL block.	• We can't do it decode function (select statement is required)
• Case easily we can understand	• Decode is complex to understand

UNIQUE KEY	PRIMARY KEY
• Unique key will allow any number of null values	• Primary key won't allow null values (Unique Key +NOT NULL = Primary Key)
• We can create any number of unique constraints on a table	• We can create maximum one primary key on a table
• Automatically unique key index will be create	• Automatically primary key index will be create
• Unique key won't allow duplicate values	• Primary also won't allow duplicate values and won't allows null values also

PRIMARY KEY	FOREIGN KEY
• Primary key won't accept duplicate values and null values	• Foreign key allows duplicate values and null values
• We can create only primary key on a table	• We can create any number of foreign keys on a table
• Index automatically create	• If we need we have to create
• We can create primary key on any column	• We can create foreign key constraint on any column but reference column should be primary or unique

UN NAMED BLOCKS	NAMED BLOCKS
• Blocks which is not have header section are known as anonymous blocks	• That's PL/SQL blocks which having Header (or) Labels are known as 'Named Blocks'
• It cannot stores in database. We cannot recall anonymous blocks. Means these are not reusable	• It will stores in database. Whenever you want to call you can the named blocks. Means these are reusable.
• It cannot allow any mode of parameters	• It accepts the mode of parameters like in, out
• It won't return a value	• Subprograms called functions must return a value
• Need to compile every time when we execute.	• Once you complied after compilation it is permanently stored as p-code in the shared pool of the system global area
• We cannot call in any other block	• We can call it anywhere in the schema.
• We cannot permissions to other user on unnamed blocks.	• We can give permissions to other users.

TABLE	VIEW
• Table is a combination of rows and columns	• View is a stored select statement
• Table occupies some space in the database	• View won't occupy any space
• Table stores some data in their self	• View is logical component (it won't store any data, it stores only select statement)
• Table is preliminary storage for storing data and information in RDBMS	• Views does not exist as a stored set of data values in data base
• A table is a collection of related data entry's and it consists column and rows	• View is virtual table who's contents are defined by a query
• If write DML commands on table that will be impact on the related views	• If write DML commands on view that will be impact on the related table
• If you drop the view that does not efforts on table data	• If you drop the table then view will be invalid status
• Select * from user_tabs	• Select * from user_views

BULK COLLECT	BULK BIND
• Bulk collect will work from SQL engine to PL/SQL engine	• Bulk bind will work from PL/SQL engine to SQL engine
• We will write bulk collect along with select statement	• We will write bulk bind only when DML operation inside the collection

EXPLICIT CURSOR	REF CURSOR
• Explicit cursor gives information every time same as select statement	• Strong ref cursor able to change where conditions in every time
• Explicit cursor allows joins in select statements	• Strong ref cursor not allows joins because its keyword is emp%rowtype
• We want to fetch data from 10 different tables we should write 10 explicit cursor	• One weak ref cursor is enough for different result sets
• Types o Basic cursor o Cursor for loop o Cursor for loop with select statement	• Types o Strong ref cursor o Weak ref cursor (sys ref cursor)
• Attributes ✓ %found ✓ %notfound ✓ %rowcount ✓ %isopen	• Attributes ✓ %found ✓ %notfound ✓ %rowcount ✓ %isopen

CURSOR	BULK COLLECT
• Cursor fetching data record by record	• Bulk collect moves entire data in single shot from SQL to PL/SQL engine. But PL/SQL engine should have collection variable.
• Number of context switches require	• Only one context switch is enough for all data
• Consumes less memory	• Consumes more memory
• Supports only forward navigation	• Supports both forward and backward navigation
• It does not support random accessing	• It support random accessing

SQL	PL/SQL
• Structured query language	• Procedural Language extension of SQL
• SQL is declarative language	• PL/SQL is procedural language
• We can execute a single operation at a time	• PL/SQL can perform multiple operations at a time
• SQL directly interacts with database server	• PL/SQL does not directly interact with database server
• SQL not have for loop, if control and similar structures	• PL/SQL has for loop, while loop, if control and similar structures
• SQL does not have variables	• PL/SQL has variables and data types etc.,
• SQL is data oriented language	• PL/SQL is application oriented language
• Mainly used manipulate data	• Mainly used to create an application

FUNCTION	PROCEDURE
• Function we can call inside the select statement	• Procedure we can't call inside select statement
• Function must return a value	• Procedure may or may not return a value
• We can use return statement in function	• We can use return in procedure but this return will be terminate the program (this return statement not work like function return statement)
• We can use DML operation in function but that the function we can't call in inside the select statement	• We can use DML operations in procedure
• If we want call DML function in select statement then pragma autonomous transaction is require	• We can't call procedure in select statement
• Out parameter is not require in function because function must return a value	• If you want retrieve something in procedure then out parameter is require
• If we use out parameter in function then that function we can't call inside the select statement	• We can't call procedure in select statement

IMPLICIT CURSOR	EXPLICIT CURSOR
• Its provided by oracle and generates automatically when executing DML operations or into clause.	• It is created & executed by user whenever they want to execute

• It is single type
  ○ SQL%
	• It is three types
	  ○ Basic cursor
	  ○ Cursor for loop
	  ○ Cursor for loop with select statement

• Used to know the status of the 'DML operation' or 'select into clause' in PL/SQL block	• Used to fetch more than one row
• It gives information about given job done or not, and how many records effected	• It is fetch the data from PL/SQL engine to collection variable

• Attributes
  ✓ %Found
  ✓ %Notfound
  ✓ %Rowcount
	• Attributes
	  ✓ %Found
	  ✓ %Notfound
	  ✓ %Rowcount
	  ✓ %Isopen

TRIGGER	PROCEDURE
• Implicit object	• Explicit object
• DECLARE is necessary for declare variable	• AS is necessary for declare variable
• JAVA/DOT NET etc., front end resources cannot call trigger	• Anyone can call and use procedure
• We can't write parameters (in/out) in triggers	• We can write parameters (in/out) in procedures
• We can able to disable/enable the trigger	• Here only drop

• Main purpose:
  ✓ To avoid the transaction
  ✓ To maintain audit
	• Main purpose:
	  ✓ To do the calculations

BEFORE TRIGGER	AFTER TRIGGER
• Trigger will be fired before update or insert or delete on table	• Trigger will be fired after update or insert or delete on table
• If table having unique key constraint and that table having before row level trigger then if you are trying to insert duplicate values on that table first before level trigger will be fired	• If table having unique key constraint and that table having after row level trigger then if you are trying to insert duplicate values on that table first constraint will be fired
• We can modify the new values **Ex**: :NEW.EMPNO	• If you are trying to modify new values then that create with compilation error means we can't modify new values in after row level trigger
• Best for raise_application_error	• Best for audit maintenance
• Data won't store in buffer (Means: it is not occupy space in memory because before insert trigger will be fire)	• Data will be stored in buffer first (Means: it occupy space in memory)

STATEMENT LEVEL	ROW LEVEL
• Even single row also not effected blindly fired once	• How many rows are effected that many times it will fire
• If 10 rows effected it fired once	• If 10 row effected it fires 10 times
• Recursive error	• Mutating error ✓ Recursive error ✓ Dead lock (Pragma+TCL)
• To avoid the transaction is main purpose	• To maintain audit is main purpose of row level trigger

TRIGGER	CONSTRAINT
• Trigger is user defined	• Constraint is pre defined
• Whatever the code you want to add you can add in triggers (user friendly)	• Always follow same rules and restrictions
• We can see and modify	• We can't see and modify they are in-built and encrypted
• We can write triggers on objects and schema	• Constraint writes only on columns
• User_triggers	• User_constraint

DDL TRIGGER	DML TRIGGER
• Operates on CREATE, DROP, ALTER and TRUNCATE	• Operates on INSERT, UPDATE and DELETE
• Applied on databases and schema	• Applied on tables and views
• It cannot be used as instead of triggers	• It can be used as instead of triggers
• Cannot create inserted and deleted tables	• Creates inserted and deleted tables
• DDL triggers run only after a T-SQL statement is completed	• DML triggers run either before or after a T-SQL statement is completed
• Schema level	• Object / Statement level

PROCEDURE	PACKAGE
• Procedure is standard alone object	• Package is a collection of related objects
• Procedure mainly used to do the calculations	• Package is used put all programs in one area which are related to same module
• Procedure is a named PL/SQL program procedure are used for to perform a particular	• Package is grouping the collection of database objects like procedure, functions, type, variable, cursors and exceptions etc..,
• Variable declared in procedure global for that procedure only	• Variables which are declared in package spec those are global for entire schema, we can use it anywhere in the schema.
• Function over loading, One time Procedure, Forward declaration concept are not allowed in standard alone procedure	• Here over loading concept is valid. we can define same name for database objects with different type of parameters or data types
• For procedure both declare, body sections we have to write at one area.	• For package we have to declare global variables in spec. if you want write any functions are procedure you have to write in package body only.

Important Programs

1. ERROR HANDLING:

↳ STEP_1: CREATING ERROR LOG TABLE:

```
CREATE TABLE ERROR_LOG (MODULE_NAME VARCHAR2 (30), ERROR_DATE
DATE,P_ERROR_NO VARCHAR2 (15), ERROR_MSG VARCHAR2 (512),
ERROR_LINE_NO VARCHAR2(3));
```

↳ CREATING ERROR LOG PROCEDURE:

```
CREATE OR REPLACE PROCEDURE SP_ERROR_LOG (P_MODULE_NAME VARCHAR2,
P_ERROR_NO VARCHAR2,P_ERROR_MSG VARCHAR2,P_LINE_NO VARCHAR2)
AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO ERROR_LOG VALUES
  (P_MODULE_NAME, SYSDATE, P_ERROR_NO, P_ERROR_MSG, P_LINE_NO);
COMMIT;
END;
```

↳ EXCEPTION SECTION:

```
EXCEPTION
   WHEN OTHERS THEN
      SP_ERROR_LOG ('SP_COMM_CAL',SQLCODE,SQLERRM,
            SUBSTR (DBMS_UTILITY.format_error_backtrace,-5));
      RAISE_APPLICATION_ERROR(-20400,SQLERRM);
END;
```

2. QUERY TUNING/EXECUTION PLAN:

There are 3 methods to get this plan

4. Select the statement and press (F5).
5. Process is
   - ↳ Select the statement
   - ↳ Tools
   - ↳ Explain plan.
6. Process is
   - ↳
   ```
   EXPLAIN PLAN SET STATEMENT_ID='R' FOR
   SELECT E.ENAME,E.HIREDATE,D.DNAME FROM EMP E, DEPT D WHERE
   E.DEPTNO=D.DEPTNO;
   ```
   - ↳
   ```
   SELECT * FROM PLAN_TABLE WHERE STATEMENT_ID='R';
   ```

Cost: No. of hits to the database (means how many times it goes to database to bring the data).
Cardinality: How many records it's searched.
Bytes: How much of area it's searching for the data.

3. TO KNOW THE PROGRESS OF PROGRAM:

It is used calculate time of our program taking even it is procedure (or) function. It is used get information like how many times each statement is executing how much time it is taken to execute that statement.

- **DBMS_PROFILER** is a predefined package. We should need two tables for this process they are:
  - `SELECT * FROM ALL_SOURCE;`
  - `SELECT * FROM PLSQL_PROFILER_DATA;`

```
BEGIN
  DBMS_PROFILER.start_profiler('SP_DATA_LOADINNG');
  SP_DATA_LOADINNG;
  DBMS_PROFILER.stop_profiler;
END;
```

- Here
  - **start_profiler, stop_profiler** are procedures in the "**DBMS_PROFILER**" package.
  - **SP_DATA_LOADINNG** is a individual profiler which is we want to know the run time by using this package.

- After executing this then we want to check the result then we want to result we should go for that two tables join like:

Ex:
```
SELECT A.TEXT, A.line, P.TOTAL_OCCUR, P.TOTAL_TIME/1000
  FROM ALL_SOURCE A, PLSQL_PROFILER_DATA P
  WHERE A.line=P.LINE# AND A.name='SP_OBJECTS_REPORT';
```

TO GET PL/SQL TABLES:

- Open my computer
- Open which drive you are installed oracle
- Type "proftab.sql" in search
- Then open the proftab folder
- Select all and copy
- Past it in SQL command window
- Automatically three tables created. Those are
  - **PLSQL_PROFILER_DATA;**
  - **PLSQL_PROFILER_RUNS;**
  - **PLSQL_PROFILER_UNITS**

4. TO KNOW THE PERFORMANCE OF ANY APPLICATION:

If you want to know the system performance you have to generate trace files that trace files are not in user readable format, if you open the trace file that will be displayed in binary format. Then if you want to read that trace file you have to convert that file into user readable format then you should go for TK proof.

- Open my computer and type " *.TRC " in search bar.
- Copy the trace file name.
- Then open prompt command.
- Type TKPROOF and paste the trace file name.

  Ex: TKPROOF tracefilename.trc.

- **Trace file:** Oracle will save the information trace file in certain time like last 30mins or last 2hrs how many programs has run and how much time they taken to run.

  To start trace file generation:
  - Open command window.
  - `ALTER SYSTEM SET SQL_TRACE=TRUE;`

  To start trace file generation:
  - Open command window.
  - `ALTER SYSTEM SET SQL_TRACE=FALSE;`

Then if you want to see the log file open my computer and type " *.TRC " in search bar then trace files will be open but document format is not readable if you open the file it will be displayed in binary format.

5. TO GENERATE OTP:

STEP_1: CREATE A TABLE

```
CREATE TYPE TYP_TAB IS TABLE OF NUMBER;
```

STEP_2: WRITE A FUNCTION FOR 4 DIGIT NUMBERS

```
CREATE OR REPLACE FUNCTION SF_NUMBERS
RETURN TYP_TAB
AS
  TYP TYP_TAB:= TYP_TAB();
BEGIN
   FOR I IN 1000..9999 LOOP
     TYP.EXTEND;
     TYP(I)=I;
   END LOOP;
 RETURN TYP;
END;
```

STEP_3: WRITE A FUNCTION OTP GENERATION

```
CREATE OR REPLACE FUNCTION SP_OTP
RETURN NUMBER
AS
LN_OTP NUMBER;
BEGIN
  SELECT COLUMN_VALUE INTO LN_OTP FROM (SELECT * FROM
  (SELECT * FROM TABLE (SF_NUMBER))ORDER BY
  DBMS_RANDOM.random)X WHERE ROWNUM=1;
     RETURN LN_OTP;
     END;
```

INTERVIEW

1. Explain about yourself and experience..?

I'm Ramana murthy. I have completed my B.Tech in the year of 2018.

Currently I'm working with "Wipro Technologies Private Limited" as PL/SQL developer. I have overall 2.2 years of experience as PL/SQL developer. Currently we are working for "Merchant Transactions" project client is EQUITAS bank.

2. Tell me about your project..?

Currently we are working for Merchant Transaction project. This project client is EQUITAS bank. Wherever the EQUITAS swipe machines provided those are called merchants. The main aim of the project is who are using EQUITAS swipe machines in their shopping malls or hotels or any institutions in all over India that swipe machines transaction details we will get every day morning at 6AM. This data is -1 data means yesterday's data, all over India what are the transactions happens in the EQUITAS swipe machines that information will collect the from the client in directory that data they can provide us then we can load that data from directory to database by using external tables.

Once data is loaded then we have to go for the validation, in that validation we have to check that data is proper data or not means all the mandatory columns data they can provide or not like merchant id, transaction id, transaction type, transaction status etc.., all these things we have to check .

After the validation that data we have to divide into two parts. 1st one is off source transaction and 2nd one is on source transaction. Off source transaction nothing but if any other bank customers using their cards in EQUITAS swipe machines that is called off source transactions. On source transaction means EQUITAS bank customers using their cards in this EQUITAS swipe machines.

If that file they have not send by 6am we have store one alert message in a table based on that record front end resources will send a mail to them 'today we are not yet getting file till now' like that one message they will pass.

And that file having any mistakes means any mandatory fields doesn't contain any information in that case also we have to pass a message means based on the functional document we have to validate some mandatory column values if there is any mistakes for those details also we have to pass a alert message to business people.

That data they can provide in excel sheet, in that excel sheet we will get both transaction amount and charges. Means if EQUITAS charge some amount for off source transactions that amount they can pass in source fee column and net fee, service fee etc.., charges they can provide in charges column. Charges amount we have divide in net fee, service fee etc.., these percentage details we can get in the app configure details table. And mostly source fee will be applied for off source transactions only.

For this off source transactions we have to generate reports to send their respective banks. For example to SBI bank we have send the list how many SBI customers has used their card in EQUITAS swipe machines yesterday, that reports contains transactions amount and all charges including source fee. Like this how many banks in India all the banks we have to generate report like this. Then they will check the reports and pay the money EQUITAS bank. For this project we have generated number of reports and created some external tables, database table, functions, procedure, packages etc.., and we have done lots of validations for this project we have working from last 1.5 years.

I have worked for loan secure deals project also, it is a banking project this project client is EQUITAS bank. The main aim of the project whatever the branches they have in allover India all the branches will give the loans to customers then bank need money to give new loans so bank want to sell the loans to some organization, for example Mumbai vashi branch had given 50cr loans to customer now that 50cr loans bank will sell to reliance organization for 45cr, but bank need to pay 50cr to organization in some period like (10yrs). Here bank losing 5cr but by collecting EMI bank will get nearly 65cr means bank getting 10cr profit. And now bank have 45cr they are ready to give new loans.

Like that EQUITAS bank has nearly 150 deals they can sell each deal to some big shots in India. Each deal can have thousands of customers and each deal can have any number of assets like gold loan, home loan etc.., and those deals may have participation of some insurance companies and also deal should have some liquidity amount.

If any customer didn't pay their EMI for a month then insurance companies will pay the EMI to the bank after customer has paid their EMI they will get back their money from the bank, for this bank will pay some percentage of amount to insurance company. And bank should pay the EMI every month to the organization.

If bank not yet collected some EMI amount for a month and there are no insurance persons for that series, then bank need to pay that amount from the bank pocket, for this bank will be separately maintains some amount from the deal amount that is called liquidity amount.

When deal is happened we will get five excel sheets those are deal setup, asset details, series details, loan details and liquidity details. We have to load that data into external tables and do the validation for the data, after completion of the validation we have to load the data into database tables, and we have to generate reports for approved deals and maintain audit for liquidity amount etc.., for that we are created external tables, database tables, functions, procedure and we used bulk collect and bulk bind and dynamic SQL concepts for those modules.

3. What are the environment are you worked..?

I have worked mostly with development and unit testing environment sometimes when we are executing the script in testing schema so that time I will take the script from SVN portal and I will execute in testing also.

If testing team need any schema or they need any database for that also I have worked and I have worked for change request.

4. You have any experience in production..?

No sir, I didn't go for production in my 2years of experience. I worked here only what are the errors they are raised in the production that we are resolving in our location.

5. You have any experience in UNIX..?

No sir, whenever I completed my B.Tech immediately I land in oracle, I didn't get any chance yet to use UNIX in my previous company for last 2 years of my experience.

6. How will you get the work..?

Every week beginning day we have a team meeting in that team meeting they will assign the work. Or every day morning if there is any urgent work my reporting will send a mail "please complete this task you have any queries in this you can directly come and discuss with me or B.A or whenever you doing this task you have to refer the page number 52 in FSD. Sometimes they will call us for team meeting at every week beginning day that entire week/month task they can assign at one day.

But weekend we have reviews for what is progress, is that unit testing completed, or it is going on are not meetings are for issues or assign a work.

7. What are your responsibilities..?

My responsibilities are writing functions, procedures, triggers, views and synonyms based on the requirement. They assigned any task to me that task is one module may be that module contain procedures, functions then I can write those programs. I'm completely handled one module for that module I have written different concepts I'm not saying I used all the concepts for developing that module but many concepts I used like functions, procedures, bulk collect, bulk bind and sequences. And maintaining script and sometimes I'm giving the guidance for technical support to the junior resources.

8. For what purpose you are used procedure in your project..?

- Front end resources will give some data that data we have to load in tables for that I have written a procedure.
- We have to generate a job every day morning 6am, that job should be generate one excel sheet that excel sheets contain what is the percentage we have to give for each inventory and each supplier all these reports should be generated automatically for that I have written a procedure.
- Front end there is button when front end resources clicking that button this action is should happen from example that action is commission calculation for that I was written a procedure.

9. What are the different types of queries..?

- Sub Query
- Co-related sub query
- In line query
- Scalar query.

10. Write a co-related sub query and explain how it is works..?

```
SELECT * FROM EMP E WHERE E.SAL IN
(SELECT MAX(SAL) FROM EMP D WHERE D.DEPTNO=E.DEPTNO)
```

In this query

- Outer query send deptno's to inner query.
- Then inner query will send the higher salaries of that deptno's to outer query.
- Then outer query will give the output of the department wise higher salaries by using inner query data.

Means:

In co-related sub one query result depends on another query result both are not individual queries.

- 1st outer query will run and send the data to inner query.
- Then inner query filter the data and send to outer query.
- Then outer query will bring the output by using the result of inner query.

11. What TCL will do..?

TCL: Transaction Control Language.

It will control complete action of a transaction by either we give commit or rollback. Means every transaction completely under control of TCL (commit, rollback)

Note: Transaction means insert or update or delete.

12. What are the types of statements..?

- **DDL:** Data Definition Language.
  - ↳ `CREATE, ALTER, DROP, TRUNCATE.`
- **DML:** Data Manipulation Language.
  - ↳ `UPDATE, INSERT, DELETE.`
- **DCL:** Data Control Language.
  - ↳ `GRANT, REVOKE.`
- **TCL:** Transaction Control Language.
  - ↳ `COMMIT, ROLLBACK.`
- **DQL:** Data Query Language.
  - ↳ `SELECT.`

19. Difference between view and snapshot..?

- ♦ View is stored select statement snapshot nothing but a materialized view.
- ♦ View is logical component.
  Materialized view is physical component.
- ♦ If you perform any DML on operations in the view then it immediately impact on original object as well as if you perform any DML on the main table it immediately impact on the view. If you perform any DML operations on materialized view that won't impact on the main table but if you perform any DML operation on the main table it may or may not impact on materialized view it depends on method of materialized view.
- ♦ Normal view won't store the data physically
  Materialized view store the data physically and able to create constraints and index on the materialized view.
- ♦ Materialized view need a tablespace normal view not needed.
- ♦ All the materialized views are won't allow DML operations but some materialized views will allow DML operations.

  Materialized view will be refreshed based on the method. Those are
  - ↳ Basic method
  - ↳ Refresh fast on commit method
    this method need primary key and log in the table
    and it refresh automatically when gives commit to the main table after DML.
  - ↳ Force refresh method
    means build immediate for refresh in this method we need one predefined package "**`dbms_mview.refresh`**".
  - ↳ Interval refresh method

13. Difference between cursor for loop and for loop which is faster..?

For loop is faster. Because for loop mostly we are executed in the PL/SQL block / PL/SQL engine but for cursor for loop two engines is required because it has select statement need go for SQL engine and collect data from SQL engine. Means cursor for loop takes some additional time for go and bring the data from SQL engine to PL/SQL engine.

14. What is the difference between procedure and function..?

- Function must return a value because we write function with return statement without return statement we can't write the function procedure it can be may or may not return a value because if you are going for the out parameters then only it return the value. There no rule like procedure must return a value.
- We can write return statement in the procedure but this return not work like function return. Procedure return will terminate the program there itself and come to end part directly. But function return compulsory it will bring some value on the screen is it can be null or it can be value.
- Function we can use in the SQL and as well as we can use in PL/SQL. But procedures we cannot use in the SQL.
- We can use procedure inside the function, procedure, packages and triggers (or) between the declare, begin and end; Means procedure we call in PL/SQL blocks only. But functions we can call in SQL statements or we can call in PL/SQL statements.
- But if you write any DML operations in the functions again some rules are there we cannot call those functions in the select statements or SQL. We need a "`pragma autonomous_transaction +TCL`" commands to call those functions in SQL statements. In procedures does not matter your using DML operation or not using DML operation there is no deference in difference in procedure case.
- Mostly function don't need out parameter because there is already a return statement. But some cases it is required may be if want to return different type data.
  But in procedure case if you want to show something on the screen compulsory you can go for out parameter without out parameter you cannot show anything on the screen.
- Mostly we can use functions for retry something but procedure mostly we are using for to do calculations.
  Means if they want to pay the loan then I will go for procedure. If they are asking how much loan they have to pay I will go for function.

15. How you handle errors..?

If we are working for any project many programs we will write those can be functions, procedure, packages or triggers program should have a 'when others exception section we should not write any program why because in client location tomorrow we are getting any error we didn't know which procedure is fail. So by using when others exception immediately that error details you have to insert in error log table. So if want to insert error details in error _log table after inserting you have to write commit but this commit should not effect on parent transaction.

That's why we have to written a separate procedure for this. to insert error details in error log table. That procedure should have a '`pragma autonomous_transaction`' and '`COMMIT`'. Because this procedure should insert errors details into error_log table that only should be committed this commit should not effect on the where this procedure calling in another procedure. So when others exception after that we have to call this error log procedure when error is occur immediately this procedure is executes that error details insert into error log table.

Error log table mainly should have five columns " module_name, error_date, error_msg, error_number, line_number". After this procedure we have to write '**`raise_application_error`**'. Why because after inserting error details into error_log table then '**`raise_application_error`**' statement will executes and it will be roll back all previous DML operations.

So without writing error handling section in any procedure that procedure we won't sent to the client.

16. What is %TYPE and %ROWTYPE..?

- %TYPE is a column type declaration.
  %ROWTYPE is a record type declaration.

- %TYPE used to define variable according to specific column in data base table.
  %ROWTYPE used to define, variable representing complete structure.

- When we want to declare a variable and that variable data type should be particular column data type in any table then we should go for **%type**.
  When we want to declare a record representing a complete row of any table we should go for **%rowtype**

17. (a). What are the different types of cursors..?

Types of cursor:

- Implicit cursor
- Explicit cursor
- Ref cursor
- Sys ref cursor

(b). When you will go for explicit cursor and ref cursor..?
When you will go for implicit cursor..?

When we want to fetch the data from the tables in PL/SQL block into clause will fetch one record only but we want to fetch more than one record we should go for the explicit cursor or we can go for ref cursor.

Explicit cursor always associated with a single result set. Ref cursor will be associated with the different result sets. If you want to fetch any table data one ref cursor should be work for any table.

But we will go for the implicit cursor separate uses. When we are writing DML operation in the PL/SQL block and we want to know is that DML is working or not and how many rows are affected by that DML then compulsory we should go for implicit cursor.

18. How implicit cursor works..?

Implicit cursor we are able to use in PL/SQL programs only that to we want to know DML operation status or select into clause status in the PL/SQL program. Then only we should go for the implicit cursor.

19. (a). What is trigger..?

Trigger is a validation project process which is occurs automatically without any explicit column. Trigger is a implicit object explicitly we have to create but works implicitly. We cannot run trigger forcefully it will be run based on the event.

Mainly we are using triggers

1. To maintain the audit
2. To avoid the transaction
3. If transaction is doing at the same time you want to do some another action.

(b). Types of triggers..?

- **DDL TRIGGER:** I want to put any rule or restriction on DDL operations others wise I want to know who is performing the DDL operation on the schema in these both cases can go for the four events those are create, alter, drop and truncate.

  In DDL trigger we need 5 pseudo columns
  - `ora_dict_obj_name`
  - `ora_dict_obj_type`
  - `ora_sysevent`
  - `ora_login_user`
  - `sysdate`

  These five are system defined key columns and these are useful when I want to trace who is doing DDL operations in the schema.

- **DML TRIGGER:** I want to do the validation after your going for the insert or update or delete on the table then you go for the DML trigger.

  Generally when we want to put a condition like 'weekends you can't insert or delete' or 'Fridays you can't update on the table then you can go for the statement level trigger. Statement level trigger mainly we use for avoid the transaction.

  Row level trigger mainly we are using to maintain the audit. If you want to write rule or restriction for particular rows data else if we are doing update statement what is the previous value, what is the value after update if you want to maintain the audit for each DML operation then you should go for the row level trigger.

  Generally complex views won't allow DML operations if you want to perform the DML operation on complex views then you should go for the instead of trigger. Instead of trigger we can use on complex views only.

  These three or the DML triggers. DML triggers will be fired on insert, update, delete only.

- **DATABASE LEVEL TRIGGER:**

  Mostly DBA only use this database level trigger to put restrictions on like should not allow to drop/create the schema and maintain the audit line who is login into schema when he was login into the schema. If we want do all these things then only you should go for database level trigger.

> **Note:** Already we have constraints for rule or restriction but these are the limited for some rules and restriction only. If you want to put more restrictions on based on the client requirement and maintain the audit then you should go for trigger only.

20. HANSY format joins..?

- Join with using
- Join with on
- Natural join
- Inner join
- Outer join.

21. In procedure and PL/SQL which is faster?

> **Note:** PL/SQL means between declare, begin and end;

In procedure, PL/SQL procedure is faster because procedure is already compiled program. And without any errors this programs is already saved in the database. But you write any program between declare and begin, end; 1st this go to check all statements, syntaxes are right/wrong then it's going to give the output.

That's why named blocks are faster than unnamed blocks.

22. Write a PL/SQL block for trigger at the time of insertion, deleting, updating for inserting old and new values auditing table..?

> **Note:** For inserting there is no :OLD.VALUES only :NEW.VALUES are available,
> For updating, deleting both :OLD.VALUES : NEW.VALUES are available.

```
CREATE TABLE AUDIT_EMP (EMPNO NUMBER, OLD_SAL NUMBER, NEW_SAL NUMBER,
OLD_COMM NUMBER, NEW_COMM NUMBER, OLD_JOB VARCHAR2(30), NEW_JOB
VARCHAR2(30), ACTION_DATE DATE, ACTION_TYPE VARCHAR2(30));

CREATE OR REPLACE TRIGGER TRG_AUDIT_EMP BEFORE DELETE OR INSERT OR
UPDATE ON EMP FOR EACH ROW
DECLARE
X NUMBER;
Y NUMBER;

BEGIN
X:=CASE WHEN :NEW.EMPNO IS NULL THEN :OLD.EMPNO ELSE :NEW.EMPNO END;
Y:= CASE WHEN INSERTING THEN 'INSERT' WHEN UPDATING THEN 'UPDATE' ELSE
'DELETE' END;

INSERT INTO AUDIT_EMP
      VALUES(X,:OLD.SAL,:NEW.EMPNO,:OLD.COMM,:NEW.COMM,:OLD.JOB,
                  :NEW.JOB, SYSDATE,Y);
END;
```

23. What is package..?

Package is a collection of related programs. If you are calling any packaged procedure/function entire package will come and sit in the RAM until the session end. Then if you want to call any other procedure/function I the same package there is no need to go and search in the database because the entire package is already exist in the RAM.

What are the variables you have declare in the package spec all variables are global variables you can use that variables at anywhere in the schema. If you write any collection in the package spec you can use that as in parameter, out parameter, collection type in anywhere in the schema.

If any package have hundreds of procedures/functions and then if you calling only one procedure in the package then entire package will come and store in the RAM means unnecessarily package occupying the RAM memory.

24. (a). Why you are going for the join..?

If you are fetching the data from more than on table there is a chance to go for cartesian product if you are fetching the data from more than one table and you didn't write any proper join condition it will go to the Cartesian product. Cartesian product nothing but cross join.

If we want to get selected data from more than one table compulsory we should go for join.

(b). Different types of joins..?

- Equilent join
- Non Equilent join
- Left outer join
- Right outer join
- Full outer join
- Self join
- Cartesian join.

Self join: Where joining same table it self is called 'self join'.

When you want to print employee name along with their manager name then you should go for self join.

```
SELECT W.ENAME, M.ENAME FROM EMP W, EMP M WHERE
W.MGR=M.EMPNO;
```

Cartesian join:

When you are fetching the data from 2 tables and you not putting any proper join condition between 2 tables then it's called cartesian/ cross join.

Ex: if you are fetching data from a and b table if you not putting any join condition between A and B. if a table having M rows and B table having N rows then you will get the output is =M*N rows;

To avoid we can go for join.

25. What is the difference between 'IN' and '=' operator..?

If we want to compare the data with only one record then better to go for '=' operator when we want compare the data with more than one record then we should go for 'IN' operator.

- Sub query having aggregate function then go for '='.
- Sub query having aggregate with group by clause then go for 'IN'.
- Sub query doesn't having aggregate function then also better to go for 'IN'.

Mainly '=' operator will support to pass only one value after '='. But 'IN' will support to pass list of values.

26. How to access variable of nested procedure..?

Nested Procedure: If you call an procedure inside any other procedure is called "Nested Procedure".

If you are calling P1,P2 procedure in P3 procedures and P3 having variables X,Y then you are able to use that X,Y variables as parameters of in P1,P2 procedures. But you are not able to use those variables as variables in P1,P2 procedures. If you want use then you have to declare those X,Y variables in P1,P2 procedures also. Means we can use that P1 variables in P2, P3 procedure as in or out parameters only.

27. What are the collections in PL/SQL..?

Generally whatever the variables we use number, varchar, date these are SQL variables and these will be store one value at a time. If you want to store multiple rows you have to create composite variables, Composite variables nothing but collection variables.

Mostly we use collection variable along with bulk collect concept. When we are using bulk collect we will definitely need to write collection variable.

Collection variables 3 types:

- ↳ Nested table
- ↳ Varray
- ↳ PL/SQL table.

Varray table have a limit remaining nested, PL/SQL table doesn't have a limit. But if you going for the PL/SQL table index number is already exist when we want to store one by one record into PL/SQL table no need to go for extend method because there is index number is already exists we can directly put the values. But in nested/varray table we want put the one by one record we have to go for the extend method. But when you are going for the bulk collect concept there is no difference for nested, PL/SQL, varray. But for varray we have to know that number of records already have and the limit of varray table.

28. (a) Which is the best method to kill the process in (unix, oracle)..?

Unix is best and faster method to kill the process.

(b). Method of oracle to kill the process..?

```
SELECT U.OBJECT_NAME,V.SESSION_ID,S.SERIAL#,S.OSUSER
FROM USER_OBJECTS U, V$LOCKED_OBJECT V, V$SESSION S
WHERE U.OBJECT_ID=V.OBJECT_ID AND V.SESSION_ID=S.SID;
```

2nd way:

- ↳ `ALTER SYSTEM KILL SESSION '25,3563' IMMEDIATE`

Here: 25 - Session id
3563 - Serial number.

29. Why we cannot use DDL commands in the procedure..?

Because DDL operations will do auto commit. Suppose in middle of the program I'm writing DDL operation what are the previous DML operations are there that will be commit automatically because of this DDL command. What are DML operations after this DDL command is there in that suppose anyone is fail then it will go to the exception part in exception part "**raise_application_error**" is there it will raise error and it though a message on the screen and it will be rollback some of the DML operations but it can't rollback some of the DML operations because those are already committed. Means in that procedure some DML are committed and some are not committed some of the program is success and some of the program is fail not be happen like that program should run completely or should be fail completely that's why we cannot use DDL commands in the programs. But compulsory you want to DDL operation then you should go for dynamic SQL.

30. How to drop user/tablespace.?

↳ `DROP USER RAMANA143 CASCADE`

↳ `DROP TABLESPACE TAB_RAMANA INCLUDING CONTENTS AND DATAFILES`

31. Difference between global, local and constant variable..?

Global variable: What are the variables you have declared in the package specification those are called global variables those we can use inside the package and outside package in entire schema we can give permissions to other users also.

Some variables are there like sysrefcusror, varchar, number, date these are the pre defined variables these are the also called global variables.

Local variable: If you declare a variable in procedure, function or declare in package body and that you should not mentioned in package spec those are local for those programs only. These are called local variables.

Constant variable: If you are passing a constant value to a variable at the time of declaration throughout the program that variable contain same value we are not able to change the value of that variable these kind of variables are known as constant variables.

32. If we have both user exception and system exception and system exception which will be handled first...?

It depends on the code. Exception you can write in any order.

1st you can write predefined exception or

1st you can write user defined exception or

1st you can write non-predefined exception

But if there is any when other exception that exception should be at end of the program. For remaining there is no rule like that. You write any number of exceptions in procedure.

You can able to write other exception after the when others exception but in this case that when others exception is there in separate begin, end;

33. What is the output of these two queries..?

```
SELECT * FROM EMP WHERE COMM IN NULL;
SELECT * FROM EMP WHERE SAL=NULL;
```

In this we won't get any error and we won't get any output. Because null is incomparable value you can't compare null with any value by using '=' or 'IN'. If want to compare null with any value then you should go for the special operator 'IS'. You can compare any value with null, not null by using 'IS' only.

34. Difference between unit testing and debugging..?

In debugging we are checking one particular object/program through step by step checking means if you are testing only one program which program you are getting error on that program if you are going step by step that is called debugging.

In unit testing we are testing entire module what we have completed it contains all the procedures and functions we have written for the program. We have done unit testing with the team of front end and back end by clicking on the buttons. When you are clicking on the button if it is works then ok if it is not working then we have to check our program by debugging.

And doing unit testing in client location is called "User Acceptable Test (UAT)" client call this is unit testing in their location.

35. What is Dual? Is it is database object..?

Yes sir., Dual is a database object but it is a pre defined object. If we create any object with create command that is database object compulsory. It is predefined table we don't have any rides to modify this table we can't to DDL or DML on the table only sys can do any action that table and only admin can do is there any modification they want. But as a user we can use dual table for select only.

36. Difference between anonymous block and normal block..?

Anonymous blocks nothing but unnamed blocks.

- Anonymous blocks there is no identity
  Named blocks have identity.
- Anonymous blocks won't store in the database
  Named blocks stores in database.
- Anonymous blocks are not reusable objects
  Named blocks are reusable.
- Anonymous blocks we can't use at any other block
  Named block we can use at any block in entire schema.
- For anonymous blocks we cannot give permission to other users
  For named blocks we can give permissions to other users also.
- We are not able to pass IN, OUT parameters anonymous blocks
  We able to pass IN, OUT parameter in named blocks.
- Named blocks are different types
  - ↳ Functions
  - ↳ Procedures
  - ↳ Packages
  - ↳ Triggers

37. Difference between TK prof, dbms_profiler and Execution plan..?

TK prof means Transient kernel profile. If you want check how the application is working then you should go for "TK prof" that application contain so many programs.

If you want to check how the application is working then you should go for "dbms_profiler". Means you want to check particular object working process and executing time of the object/program. This program contain so many statements.

If you want to check execution plan of one statement then you should go for execution plan (F5). Execution plan shows how the statement is executing like which join method it is using or is this using index scan or not.

38. How to access table from another database..?

DB link is only option for this.

When two users in same database want to share data from one user to another user we have public synonyms. But one user want to share data for another user in different database then only option we have that is DB link. This data base link we have to create using there user_name, password, and the user database address

Database address you can get in "tnsnames.ora"

39. How many rows and columns dual table contains..?

Dual table contain only one column one row column name is dummy and data type is varchar size is one and value is x.

Note:
```
SELECT '1'+1 FROM DUAL --GIVES 2

SELECT '1' FROM DUAL
UNION
 SELECT 1 FROM DUAL     ---GIVES ERROR
```
(Because when you are going for the set operator the data types should be same);

40. How to count number of records in the table without using count..?
```
SELECT MAX(R) FROM (SELECT ROWNUM R FROM EMP);
```

41. What is highest salary without using max( )..?
```
SELECT SAL FROM (SELECT * FROM EMP ORDER BY SAL DESC) WHERE ROWNUM=1;
```

42. Select only those employees information who are earning same salary..?
```
SELECT * FROM EMP WHERE SAL IN (SELECT SAL FROM EMP GROUP BY SAL
HAVING COUNT(*)>1);
```

43. How to find last inserted records of table..?
```
SELECT MAX(ROWID)FROM EMP
```

44. `SELECT 'RA'||NULL||'MANA' FROM DUAL` **what is the output..?**

= RAMANA

45. Display the duplicate records in table..?
```
SELECT EMPNO FROM EMP GROUP BY EMPNO HAVING COUNT(EMPNO)>1;
```

46. I have a row in a column as 'RA@MA#NA$', Now I want my output is 'RAMANA' how you retrieve..?
```
SELECT regexp_replace ('RA@MA#NA$','\W') FROM DUAL;
```

47. How you retrieve top 3 salaries in from each department .?
```
SELECT * FROM (
SELECT DEPTNO,SAL,DENSE_RANK() OVER(PARTITION BY DEPTNO ORDER BY SAL
DESC) D FROM EMP ) WHERE D<4;
```

48. How to know oracle using my index or not..?
```
↳ EXPLAIN PLAN SET STATEMENT_ID='X' FOR
  SELECT * FROM EMP WHERE EMPNO =7369;

↳ SELECT * FROM PLAN_TABLE WHERE STATEMENT_ID='X';
```

49. When should rebuild one an index..?

If you are performing more operations on the index may be it will go for invalid status. If run one query if it is gives output slowly then should go for execution plan if you are going for the execution plan in execution plan you are not see the index scan then you will realize that index is not working using oracle optimization so then immediately we have to do rebuild the index (active the index).
```
↳ ALTER INDEX IDX_EMPNO REBUILD;
```

50. What is online key..?

If you are trying to rebuild the index at the same time somebody is writing DML operation or writing select statement on that table there is chance to system struck or system hang. If you write online keyword at time of creation of any kind of index then there is no chance to deadlock or system hang error at the rebuild the index we can rebuild the index at anytime.

```
↳ CREATE INDEX IDX_EMPNO ON EMP(EMPNO) ONLINE;
↳ ALTER INDEX IDX_EMPNO REBUILD ONLINE;
↳ DROP INDEX IDX_EMPNO;
```

51. Tell me some errors..?

- no_data_found
- too many rows
- invalid cursor
- cursor id already open
- invalid identifier
- unique key constraint violated
  - If you are inserting duplicate values in primary key or unique key columns.
- dup_val_on_index
  - If you are inserting duplicate values in unique index column.
  - It comes when column having unique index without primary/unique key constraint here 1st preference for constraints.

52. Can you update complex view..? If no why..?

No sir., We cannot update a complex view because here multiple tables are there then there is primary key foreign key relation are there if your updating which table data first it will be modify system will be confused so that's why we cannot perform DML operation in the complex views.

If you want to perform DML operation on the complex view you should go for instead of trigger.

53. What is function based index..? Write syntax..?

If you create index on any function is called 'function based index'.

Syntax:

```
↳ CREATE INDEX INDEX_NAME ON TABLE_NAME ( FUNCTION
  (COLUMN_NAME));
↳ CREATE INDEX IDX_ENAME ON EMP (UPPER (ENAME));
```

54. What is context switching..?

Data moving from PL/SQL engine to SQL engine or SQL engine to PL/SQL engine this called context switching.

When you are using more DML operations in a collection loop also context switches will be used more to avoid that we can go for bulk bind clause. If you are using select statement it is also used more context switches to fetch data in that case you can go for bulk collect.

If select statement using more context switches to avoid that we should go for bulk collect. Else if DML operations in collection loop using more context switches then we should go for bulk bind.

55. Tell me some restrictions on cursor variable..?

If you are open the cursor for fetching the data from the cursor you can fetch in two ways either you can fetch in bulk of data or either you can fetch single row. What type of data your are fetching based on that you are declare the cursor variable.

If you are fetching single column then you can write normal variable. If you are fetching entire row/table data you can write row type variable. If you are fetching the data by using bulk collect then you have to write collection variable.

Cursor will fetch the data in two ways either into clause or either by using bulk collect into. When we are go for bulk limit clause mostly we can go for the bulk collect into clause.

56. What is bulk collect..?

If you are executing any select statement in PL/SQL block either you can writing cursor select statement. If you write cursor select statement we will fetch one record from SQL engine to PL/SQL engine to avoid that we can go for bulk collect.

Bulk collect will fetch all the records at one shot. It will reduce the number context switches from more to 1.

57. What is mutating error..?

Mutating error occur only in row level triggers on which table you are performing the DML operation same table if you are using inside the trigger body. Then you will get **mutating error.** If you want to resolve this then you should go for "`pragma autonomous_transaction +TCL`".

In triggers mostly we get two errors only one is recursive error and second one is mutating error. Sometimes dead lock error also we will get but in rare cases. If you are going for the **"**`pragma autonomous_transaction +TCL`**"** then there is a chance to get dead lock error.

Means: Mutating error (ORA-04091) occurs whenever a row level trigger tries to modify or select data from the table that is already undergoing change.

58. What is recursive error..?

If we are writing trigger on an action and we are using same action in the trigger body we will get **recursive error.**

Note: Recursive error occur in row level trigger by (insert/insert)
And statement level trigger by (insert/insert, update/update, delete/delete).

59. Which will fire default first statement level or row level trigger..?

It is based on what type of event you are written. In before case 1st statement level trigger will fire and after case 1st row level trigger will fire.

Trigger firing order:

- ↳ Before statement level
- ↳ Before row level
- ↳ After row level
- ↳ Before row level
- ↳ After row level
- ↳ After statement level.

60. What is use of limit clause in bulk collect..?

Generally collections are stored in RAM if table having massive data then there is a chance to storage error because of memory to avoid that we can go for the bulk limit. Bulk limit will divide the data into pieces we can do the process with each piece of data.

61. How to debug your code..?

We have PL/SQL developer tool in our project by using PL/SQL developer there is a test button we can go for test step by step by using test menu when we are going for debug on the test step by step we can test the program.

62. Can you alter the procedure with in package..?

Yes sir., We can alter the procedure. We have to open entire package then in that package which procedure we want modify we can modify.

63. How to trace error handling..?

Generally whatever programs we written in our project all the programs should have exception section in that exception section we have to call one procedure that is the sp_error_log procedure. We are writing this procedure for catching the errors into error log table. For that only we have written this procedure so what are the errors we are getting in our program that error should be stored in error log table. That error log table data will be inserted by sp_error_log procedure only.

This procedure we should call in all the programs whatever the data we are inserting by sp_error_log procedure into error log table that data should be committed. But this commit should be committed should not impact on any other programs. So we have to write to "`pragma autonomous_transaction +TCL`".

64. How to find which line error was raised..?

For error line

- `DBMS_UTILITY.format_error_backtrace;`
- `SUBSTR( DBMS_UTILITY.format_error_backtrace,-3);`

For error massage

- `SQLERRM`

  OR
- `DBMS_UTILITY.format_call_stack;`

65. How you give permission for packaged procedure to other user..?

For packaged procedure

```
GRANT EXECUTE ON PKG1.P1 TO RAMANA143;
GRANT EXECUTE ON PKG1 TO RAMANA143;
```

66. Tell me permissions you have given from SYSDBA.?

```
GRANT CONNECT, DBA TO RAMANA_MURTHY;
GRANT READ,WRITE ON DIRECTORY DATA_PUMP_DIR TO RAMANA_MURTHY;
GRANT EXECUTE ON UTL_FILE TO RAMANA_MURTHY;
```

67. Is it possible to open cursor which is in another procedure..?

Yes. If the cursor is declared in package spec then we can use that cursor at anywhere in the in the schema. But that cursor compulsory should be declared in package spec.

68. How to update complex views..?

View:

```
CREATE VIEW VW_EMP_DEPT AS SELECT
EMPNO,ENAME,JOB,SAL,E.DEPTNO,DNAME,LOC
FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;
```

Instead of trigger:

```
CREATE OR REPLACE TRIGGER TRG_COMPLEX
INSTEAD OF UPDATE ON VW_EMP_DEPT FOR EACH ROW
BEGIN
UPDATE EMP
SET ENAME=:NEW.ENAME,EMPNO=:NEW.EMPNO,JOB=:NEW.JOB,SAL=:NEW.SAL,
DEPTNO=:NEW.DEPTNO WHERE EMPNO=:OLD.EMPNO;
```

Updating complex view:

```
UPDATE DEPT
SET DNAME=:NEW.DNAME, DEPTNO=:NEW.DEPTNO,LOC=:NEW.LOC
WHERE DEPTNO=:OLD.DEPTNO;
END;
```

69. How to insert a photo in table..?

```
↳ CREATE TABLE XYZ (ENAME VARCHAR2(30), PHOTO BFILE);
↳ INSERT INTO XYZ VALUES ('RAMANA',
  BFILENAME('DATA_PUMP_DIR','R.JPG'));
```

70. What is substr and instr..?

Sub string will give the portion of the string and in string will give the position of the string. Substr having three expressions for substr we can give two expressions also. Instr having four expressions. Both are char functions.

SUBSTR:

```
↳ SELECT SUBSTR(ENAME,1,3) FROM EMP;
↳ SELECT SUBSTR(ENAME,3) FROM EMP;
```

INSTR:

```
♦ SELECT INSTR('GOOD MORNING','O',2,3) FROM EMP;
```

71. Difference between CASE and DECODE..?

- Case is a statement and decode is a function.
- Case will support the operators like (>, <, <=, >=, =, in, between, like, not like. Decode won't support those operators.
- Case we can inside the select statement and without select statement also we can use. Decode we can use inside the select statement only.
- By using case we can directly assign a value to variable in PL/SQL block. For decode select statement is required.

72. Can you use sysdate in check constraints..? If no why..?

No., we can't use sysdate in constraints because it is not a static value. It is dynamic value every second sysdate can be change.

73. Difference between column level constraint and table level constraint..?

Generally when we create a constraint on a column is called level constraint. But if we create constraint on a table is called trigger. Trigger always we write on table level.

Column level constraints useful to put a condition on way specific column for restrict duplicates or null values. Or put a condition like values should be greater than something or less than something or between two values.

But trigger useful to put a condition on entire table to avoid DML or DDL operations on that table or to avoid operations on the table in specific time or specific days.

Both column level and table level constraints are used put rule or restrictions.

74. What is nested loop join..?

If we are joining two tables then system consider as one is leading table and another one is probing table. While table having less data that is leading table and which table having more data is that is probing table.

When we are going to do join two table we have create selective index on probing table joining column then only it's going for nested loop join. Else it is go for hash join by default.

75. What is optimizer..?

Optimizer nothing but execution plan of the query. Currently system using cost based optimization previously two types of optimizations are there 1. Cost based and 2. Rule based.

But currently system is using cost based optimization only because if there is any benefit then that index/partition table will be used.

76. What is master and detail table..?

Which tables are related with integrity constraints those table is called master, detail tables. On which table we are creating foreign key is a detail/child table and the reference table column have primary key column that table is a master/parent table.

Restrictions:

If you want to delete the data from master table 1st you should delete corresponding records from the child table. When you want to insert the data into child table that parent data should be present in the parent table.

77. Define view..?

View is logical component and it will store the select statement. When different resources using same query then better to create a view on the query then view will hide the complicity of the query we can use the query every time by writing name of the query only.

78. Can we update a view..? What are the conditions..?

Yes., We can update a view when it is a simple view,

If it is a complex view then we need instead of trigger for update the view because there is a complex relation is there in complex view means more than one table and if there integrity constraint relation are there then which table will update 1st system cannot understand so in that case we cannot update complex views then we should go for instead of trigger.

When we go for read only view in that read only view there are virtual columns. Virtual column cannot allow to updating or deleting because that data is not there when we updating another columns it allows but if we updating virtual column it won't allow.

79. What is index..?

Index will point out the physical address of the data. It is a physical component. Index completely designed based on rowid. rowid nothing but address of the data that's why when we are using index column in where condition for update, delete or select output will come very fast. It will store only address of the data means it will store rowid's only.

If it is low cardinality < 20 then B-tree index.
If it is high cardinality > 20 then Bitmap index.
If Cardinality is exactly 1 the unique index.

$$\text{CARDINALITY} = \frac{\text{NUMBER OF RECORDS} - \text{NULL VALUES}}{\text{TOTAL DISTINCT VALUES}}$$

80. What are Oracle 12c Features..?

1. Invisible column:

```
CREATE TABLE ABC (E_NAME VARCHAR2(30),E_PASSWORD VARCHAR2(30)
INVISIBLE, E_DOJ DATE);
```

- Once you created the table like this then if you execute the query "`select * from abc`" system shows **`e_name`**, **`e_doj`** columns only. System does not shown the e_password column on the screen because that **`e_password`** column is "`invisible`" column.
- If you want to see the **`e_password`** column data also then should execute like "`select e_name, e_password from abc`". Here we are asking that **`e_password`** column forcefully.

2. Sequence we can use as default value:

```
CREATE TABLE XYZ(A NUMBER DEFAULT SEQ_1.NEXTVAL,B VARCHAR2(30));
CREATE TABLE XYZ (A NUMBER DEFAULT 'CIVIL_'||SEQ_1.NEXTVAL, B
VARCHAR2(30));
```

- Here if you are like "`insert into xyz values ('ramana')`" then sequence will generate the number for **`a`** column.
- Else if you insert like "`insert xyz values (5,'ramana')`" then sequence don't generate any number for **`a`** column, that five will be inserted into that column.
- Means **`SEQ_1`** is default when you not given any number for **`a`** column at the time inserting then only it is generates a number. If you give any number then it will be silent.

3. Generated as identity:

```
CREATE TABLE XYZ (A NUMBER GENERATED AS IDENTITY, B
VARCHAR2(30));
```

- Ii is always starting with 1 and increment by 1.
- There is no need to create a sequence separately.
- If you write the key word "`generated as identity`" then its automatically generates the numbers with increment of 1.

4. **Fetch first N rows:**

```
SELECT * FROM EMP ORDER BY SAL DESC FETCH FIRST 5 ROWS ONLY;
```

- It gives the output like

```
select * from (
select * from emp order by sal desc)where rownum<6;
```

5. **Varchar2 size**:

Varchar2 size : 32767 (12c);
(up to 11g :4000)

6. **Cascade for TRUNCATE and EXCHANGE partition:**

```
TRUNCATE TABLE DEPT CASCADE;
```

- It will delete all records from parent table and also delete all corresponding records from the child table.
- Means it works like ON DELETE CASCADE.

7. **Multiple indexes on Single column:**

We can create multiple indexes on a single column in 12c. Up to 11g one column can contain one index only.

Note: If we have created two indexes on a column and then run a query for some information then system will use one of the selective index in those indexes for that query execution based on the selectivity.

Disadvantage: Each index stores separate memory. It occupies more memory.

8. **We can write function inside the with clause:**

We can write a function in **with** clause then we use that function inside by using the alias name which is we provided for that function using **with.**

9. **RENAME/NOVE data file:**

```
ALTER TABLE MOVE DATAFILE '/U01/ORADATA/INDX.DBF' TO
'/U01/ORADATA/ORCL/INDX.DBF';
```

81. What are Oracle 11g Features..?

1. **Compound Trigger:**

If you can write more than one trigger at one area is called “compound trigger”. (min:1 , max:4);

It allows only four conditions:

5. One before statement level
6. One before row level
7. One after row level
8. One after statement level.

2. **Triggers follows / precedes clause:**

Follows and precedes are the key words which is used to set a order for trigger firing when all are before or all are after. When we create no. of before after triggers and you want fire all these triggers in a particular order then you should go for ‘follows’ or ‘precedes’.

Follows tells should fire after this trigger will be fired.

Follows tells should fire before this trigger will be fired.

3. Creating trigger in disable mode:

↳ `CREATE TRIGGER TRG_X DISABLE...`

Then the trigger will be created in disable mode after that you can enable the triggers when you want to use the trigger. Before 11g we have done like this "`alter table trg_1 disable`".

4. Read only tables:

↳ `ALTER TABLE DEPT READ ONLY;`

↳ `ALTER TABLE DEPT READ WRITE;`

Then you can't do any DML on this table. Before 11g you should write a trigger for this but in 11g this new feature is introduced.

5. Invisible indexes:

↳ `ALTER TABLE IDX_EMPNO VISIBLE;`

↳ `ALTER INDEX IDEX_EMPNO INVISIBLE;`

This is useful to disable the index for execute a query in a different join method.

6. Virtual column:

↳
```
CREATE TABLE ABC(A NUMBER, B NUMBER, C NUMBER AS (A+B));
```

↳
```
CREATE TABLE XYZ(X NUMBER, Y NUMBER, Z NUMBER, M NUMBER AS
(SF_GET_MAX(X,Y,Z)));
```

↳ Then you should insert like:

```
INSERT INTO XYZ(X,Y,Z) VALUES (10,20,30);
```

Then automatically M column will updated with 30 because you can't insert data into virtual column. It updates automatically based on the condition you have written.

Note: Which function you want use in virtual column is should be deterministic function;

```
CREATE OR REPLACE FUNCTION SF_GET_MAX (P1 NUMBER, P2 NUMBER,
P3 NUMBER) RETURN NUMBER DETERMINISTIC AS
BEGIN
      IF P1 >P2 AND P1>P3 THEN RETURN P1;
   ELSIF P2 >P1 AND P2>P3 THEN RETURN P2;
   ELSE RETURN P3;    END IF;    END;
```

7. Continue statement:

```
BEGIN
      FOR I IN 1..10 LOOP
       IF I=5 THEN CONTINUE;
     ELSE DBMS_OUTPUT.PUT_LINE(I);
      END IF;
      END LOOP;
END;
```

It is replacement for 'then null' in some cases.

8. Pivot :

```
SELECT * FROM (SELECT ENAME,JOB FROM EMP ) PIVOT (COUNT (JOB) FOR JOB IN ('SELESMAN','ANALYST','CLERK','MANAGER')) ORDER BY ENAME;
```

This query gives the output how many members are working with same name in various jobs. And the output will come in a table form only.

9. REGEXP_COUNT:

```
SELECT REGEXP_COUNT('GOOD MORNING','O') FROM DUAL;
```

----3 -(gives count of 'o' in 'good morning');

```
SELECT REGEXP_COUNT(ENAME,'A') FROM EMP;
```

----3 -(gives count a 'a' in each ename (each row));

10. LISTAGG (Analytical Function):

```
SELECT DEPTNO,LISTAGG(ENAME,'-') WITHIN GROUP (ORDER BY ENAME) FROM EMP GROUP BY DEPTNO;
```

It's works like WM_CONCAT with group by clause.

Differences:

- LISTAGG separate the records with different type special characters. WM_CONCAT by default separate the records with comma only.
- We can use order by clause in LISTAGG for any column in the table. We can use order by clause in WM_CONCAT for only column which is we used to group by.

82. What is exception..?

If you want to catch the runtime error you should go for exception tomorrow we can easily understand which program is fail, in which line it is fail and on which reason it is fail.

83. What is M_view..?

MATERIALIZED VIEW:

It is a stored select statement as well as it will store the data of the select statement. Means it will store both select statement and data means it is data base object and it contains result of the query. It may be local copy of the data or may be a sub set row and columns of table or join result.

✓ **BASIC MATERIALIZED VIEW:**

```
CREATE MATERIALIZED VIEW MV_NAME AS SELECT ENAME, DNAME FROM EMP E,DEPT D WHERE E.DEPTNO=D.DEPTNO;
```

In basic materialized view at the time of creation that view stores the data permanently after creation whatever the changes you have done in the table that's not effects on the view.

That view data individual and you able to do DML operations on the view also.

✓ **REFRESH FAST ON COMMIT MATERIALIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_EMP REFRESH FAST ON COMMIT AS SELECT * FROM EMP;`

This materialized view needs

- Primary key and
- Log (`create materialized view log on emp)`

Whatever the changes you have done in the table won't effect on the view until you give the commit. If you apply the commit after making some changes in the table then immediately the view data also modify automatically.

- The log store the script of changes what you have apply commit after until you apply the commit after the commit the log will apply the same changes on the view.
- That primary key helps to store the changes order wise in log .That's why both log and primary key are compulsory for refresh fast on commit method of materialized view.

✓ **FORCE MATERIALIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_REFRESH BUILD IMMEDIATE AS SELECT ENAME, DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;`

In this materialized view changes you have done in the table not effects on the view. Until you have done a separate action for that. If you want to do same changes on view also then you should go for this.

```
BEGIN
DBMS_MVIEW.refresh('MV_GRADE','FORCE');
END;
```

After you have done this forcefully view data also updates exactly like table data. Until that view data in same position.

✓ **INTERVAL MATERAILIZED VIEW:**

➢ `CREATE MATERIALIZED VIEW MV_GRADE REFRESH START WITH SYSDATE NEXT (SYSDATE+1)AS SELECT EMPNO,ENAME,DNAME FROM EMP E, DEPT D WHERE E.DEPTNO=D.DEPTNO;`

In this view data will refresh with uniform interval which you have given. Means in this view what you have done the changes in the table automatically updates in their selves for every interval. There is no need of any separate action for data refreshing view. That's why this is called interval materialized view.

84. What are the pseudo columns..?

- ROWID
- ROWNUM
- NEXTVAL
- CURRVAL
- SYSDATE.
- Ora_sysevent
- Ora_dict_obj_name
- Ora_dict_obj_type
- Ora_dict_obj_owner

These are pseudo columns and these all predefined we can use it anywhere.

85. Difference between procedure and package..?

- Procedure is standard alone object
  Package is a collection of objects.
- Procedure mainly used to do the calculations.
  If you want to develop any module that having different buttons means different processers are there in the module. Like cancelation, refund and booking. For cancelation I will create one procedure (or) For booking I will create a procedure all this procedure and function related same module that is customer dash board for this I will create one package. Package is nothing but collection of related objects.
- If you call one procedure that procedure come and store in the RAM. Once the procedure execution completed immediately it will be erased from the RAM.
  If call any packaged procedure or packaged function the entire package will come and store in the RAM even execution of the program is completed but still the package exist in the RAM until the session end.
- Procedure variables are local variables.
  Packaged variables are global variables.
- For procedure both declare, body sections we have to write at one area.
  For package we have declare global variable in spec and if you want to write function, procedure we have to write body, minimum one program is mandatory in body even it is function or procedure or the combination. We can right any number of programs in body.
- Without body we can create spec but without spec we cannot write body. Spec contain global variables that global variables can be cursors, collections, variables or global functions, global procedures.
- Entire package we cannot execute at one shot.
- Package contains some concepts like
  - Function over loading
  - Forward declaration
  - One time procedure.

86. What is autonomous transaction..?

Generally if you write "`pragma autonomous_transaction`" in any program will be converted as a independent program then if you write commit or rollback in that program that commit or rollback won't effect on the other progress means that commit or rollback will be applied for that program DML only it won't effect on other program DML.

Mostly we are written ""`pragma autonomous_transaction`" in our error log procedure because that error should inserted in error log table and committed but the commit should not impact on parent transaction.

87. What is use of trigger..?

Yes. Mainly we use trigger for business logics means to avoid the transaction (or) to maintain the audit (or) one transaction happen at same time you want to do another transaction or another action.

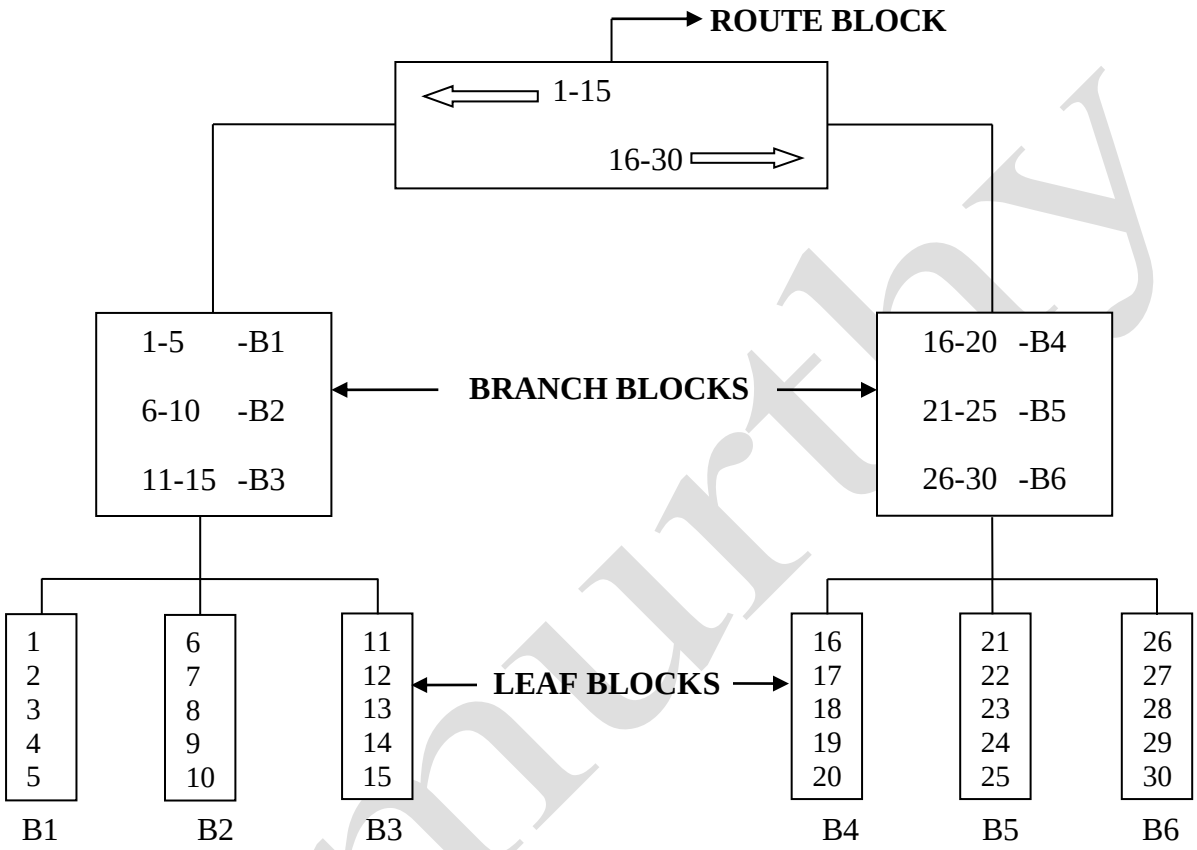
88. How do you handle exception in PL/SQL..?

I will handle exception sometimes I use predefined exception, sometimes I use defined exception and sometimes I use non-predefined exceptions.

89. How does index form internally..?

- **B-TREE INDEX:**
  - `CREATE INDEX IDX_EMPNO ON EMP(EMPNO) TABLESPACE RAMANA_INDEXES;`

B-tree index will separate the data into leaf blocks to make data searching easier. B-tree index contain root, branch, leaf blocks, root blocks, branch blocks contain directions and leaf blocks contain the data.



- **UNIQUE INDEX:**
  - `CREATE UNIQUE INDEX IDX_EMPNO ON EMP(EMPNO) TABLESPACE RAMANA_INDEXES;`

Unique index also works like b-tree index but small differences is there, in b-tree index even searching record founded then also it verifies next row for confirmation of is same record exists in next row if not found then it will bring data from table by using founded rowid's.

But in unique index if searching record is found then immediately its go and bring the data from the table by using the only one rowid. There is no chance to go and search in next row for same record because unique index blindly follows that column does not contain any duplicate values.

When we are creating unique or primary key constraints on any column automatically unique index will be created.

  - `ALTER TABLE STUDENT_DTLS ADD CONSTRAINT PK_ROLL PRIMARY KEY (ROLL_NO)USING INDEX TABLESPACE RAMANA_INDEXES;`

- **BITMAP INDEX:**
  - `CREATE BITMAP INDEX IDX_DEPTNO ON EMP(DEPTNO) TABLESPACE RAMANA_INDEXES;`

If column having repeating data then better to go for bitmap index, mostly for gender, branch id columns we will create bitmap index.

CIVIL	RAMANA (rowid)	0	0	0	LUCKY (rowid)	0
MECH	0	0	ARYAN (rowid)	0	0	0
EEE	0	STARK (rowid)	0	0	0	0
ECE	0	0	0	0	0	KALEESI (rowid)
CSE	0	0	0	ROBBERT (rowid)	0	0

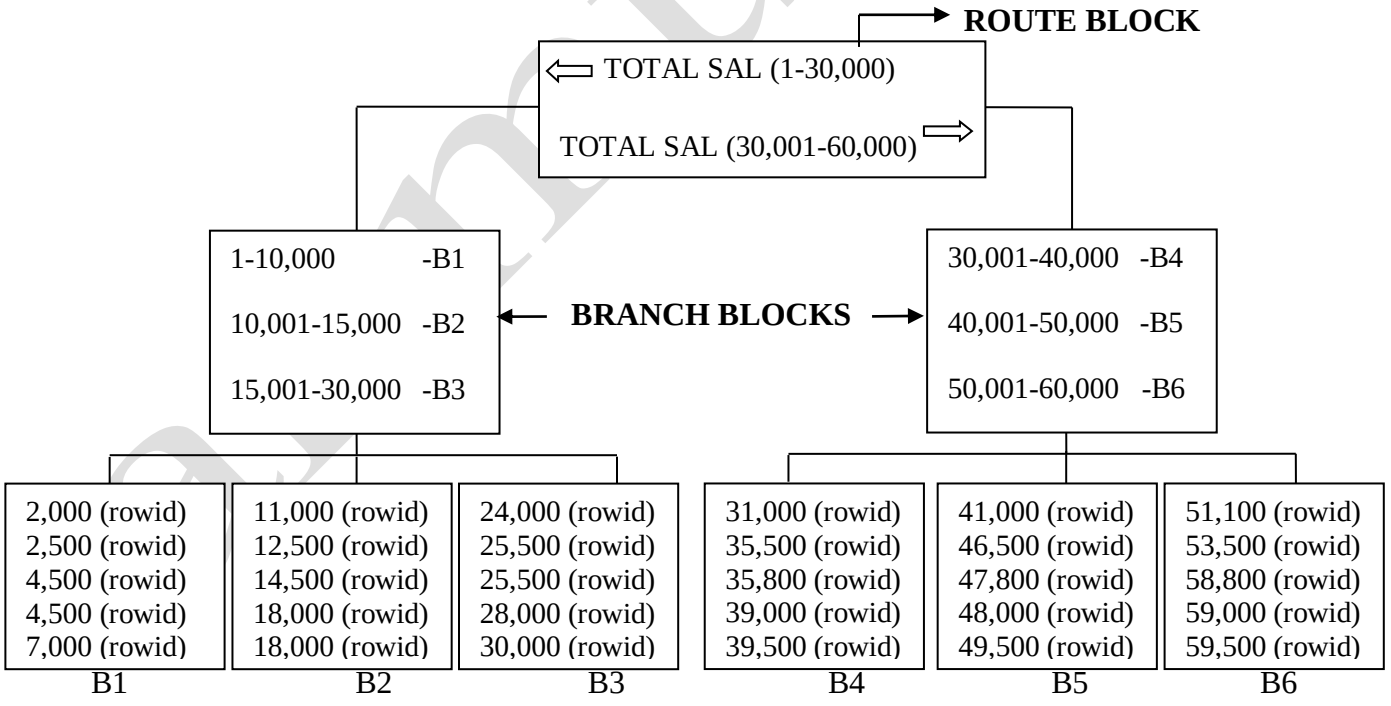
When we create a bitmap index then immediately this table will be created internally.

- **FUNCTION BASED INDEX:**
  - `CREATE INDEX IDX_SF_TOTAL ON EMP(SF_TOTAL_SAL(SAL,COMM,TAX)) TABLESPACE RAMANA_INDEXES;`

Here we are creating a index based on function.

Ex: We have function for total salary SF_TOTAL_SAL (SAL+COMM+TAX).

If we create index based on this total salary function that index will stores like below.



Then we have to search data like

'`Select * from emp where sf_total_sal(sal,comm,tax)=5000`'

Then system directly goes to B2 block and collect that total salary rowid and bring the data from emp table by using that rowid's.

NOTE: If table having two composite indexes for (X, Y) index_1, for (X, Z) index_2 then if you are writing where condition based on X then index will work based on which index having less branch blocks. If both indexes having same number of branch blocks then it works based on the number of leaf of blocks.

↳
```
SELECT * FROM DBA_INDEXES WHERE INDEX_NAME IN
('IDX_LOAN_ID','IDX_CUST_NO');
```

↳ In this it will consider 1st B_LEVEL column, next LEAF_BLOCKS column. If one index has more null values it go for another index even it having less branch, leaf blocks. Because for null values index not powerful.

90. Difference between DELETE and TRUNCATE…?

- After delete TCL commands mandatory.
  For truncate TCL commands not required because it is DDL and auto commit.
- Delete we can use in SQL and as well PL/SQL.
  If we want use truncate command in PL/SQL we need a dynamic SQL concept because it is DDL command it will commit previous DML operations also that's why not only truncate any DDL command we cannot write directly in the PL/SQL block we should go for dynamic SQL concept.
- After deleting internal memory still exists when we are deleted that blocks will be empty but still those blocks are locked because may be that data is again will come when user has given roll back command then that data should be save in same area means high water mark position will be same.
  After truncate internal memory cleared means that high water mark position will be shifted.
- Delete support to where clause means we can delete range data, particular data or entire table data by using delete.
  Truncate does not support to where clause it will delete entire table data permanently.

91. Difference between CONSTRAINT and TRIGGER…?

```
DECLARE
      --VARIABLE SECTION;
BEGIN
      --CODING (STATEMENT), MINIMUM ONE STATEMENT IS REQIRED;
EXCEPTION
      --ERROR HANDLING SECTION;
END;
```

92. What are SQLCODE and SQLERRM..?

These are system defined pseudo columns. Mostly these two columns we are using inside the when others only. When others means it will give any error oracle having thousands of errors you want to know which error is occurred that error number and that error message then you can go for the SQLCODE and SQLERRM.

SQLCODE will give the error number each error have unique error number SQLERRM will give you a error message each error have unique error message for error message we have another package also (**DBMS_UTILITY.format_error_stack**) this both are give same message error message size max =512.

93. When we will go for rebuild the index..?

If you are performing more operations on the index may be it will go for invalid status. If run one query if it is gives output slowly then should go for execution plan if you are going for the execution plan in execution plan you are not see the index scan then you will realize that index is not working using oracle optimization so then immediately we have to do rebuild the index (active the index).

```
ALTER INDEX IDX_EMPNO REBUILD;
```

94. What is online key..?

If you are trying to rebuild the index at the same time somebody is writing DML operation or writing select statement on that table there is chance to system struck or hang. If you write online keyword at the time of creation of any kind of index then there is no chance to dead lock or hang errors at the time of rebuild the index then we can rebuild the index at any time.

```
CREATE INDEX IDX_EMP ON EMP(EMPNO) ONLINE;
ALTER INDEX IDX_EMP REBUILD ONLINE;
DROP INDEX IDX_EMP;
```

95. What is sequence and what is the need of it..?

Sequence will generate unique numbers with a suitable increment which we want that too forward wise. Mostly we use sequence for when we are creating primary or unique key on columns (or) we using ID columns like empno then only we can go for sequence because all these columns should not contain any null and duplicate values that's why we can create sequence for that column because sequence will automatically generate unique values.

If sequence is not there every time we need to check what is the previous value. If we need insert new record new record we have to check what is the next number I have to give every time I have to check.

When two users inserting new values in the column at the same time and there is no sequence it leads get duplicate values if sequence is there fraction of milliseconds it will generate unique values.

Disadvantage: We cannot rollback sequence. Means if you use sequence for 10 times and roll backed your transaction all the transaction will be rollback but sequence not then if you call sequence next time it will give that 11^{th} value only.

96. What happen when cursor is not closed…
What happen when you try to open cursor that is already opened..?

No nothing will happen if the cursor is not closed there is no issue but the issue is again if we are again trying opening the same cursor then we will get the error "cursor is already open". If you are not trying to open the cursor again there is no issue even it is not closed.

97. Types of triggers..?

DDL Trigger
DML Trigger
DML trigger again three types
- Statement level trigger.
- Row level trigger.

Database Level Trigger

98. Difference between CONSTRAINT and TRIGGER…?

- Constraints are pre defined
  Triggers are user defined.
- We can able to enable or disable for both constraints and triggers.
- Mostly we can apply constraints on one column or multiple columns at a time that is called composite constraints.
  Triggers mostly we have to write on the tables or schema or database.
- Constraints always fallow same rules and restrictions we have a user friendly constraint that is check constraint whatever the condition you want to put you can put.
  Trigger is a user defined whatever the code you want to write you can write.
- Mostly we use trigger to avoid the transaction.
  Mainly we use triggers to maintain audit and avoid the transaction.
- Based on the client requirement if we cannot able to write constraint on a column then we can go for triggers because trigger code we can modify and we can see and whatever the condition we want to write we can write.
- One action is going to at same time you want to do another action then also you can go for triggers.
  Constraints are useful for only you can put restriction on column only.

99. What is temporary tablespace? Can you create that..?

Temporary table space mostly we used to store global temporary table data (or) to store group by, order by data.

```
↳ CREATE TEMPORARY TABLESPACE RAMANA_TEMP TEMPFILE
  'E:\ORACLE\ORADATA\ORCL\RAMANA_TEMP.DBF' SIZE 100M;
↳ CREATE USER AR_MURTHY IDENTIFIED BY ACR143 DEFAULT TABLESPACE
  RAMANA_DEFAULT TEMPORARY TABLESPACE RAMANA_TEMPORARY
```

If you are not given any temporary table space at the time of schema creation system will use default temporary table space. Oracle will provide a temporary table space by default.

100. What is meant by function purity levels..?
What are the draw backs functions..?
What are the disadvantages of function..?
There are any limitations for functions..?

We can write DML operations in the function but those are not eligible to call in SQL. If we want call those functions in SQL you should go for "`pragma autonomous_transaction +TCL`". If you give out parameter again you cannot call those functions in SQL. There is no medicine for this.

101. Can you write a trigger of DDL statements..?

Yes, we can write a trigger for DDL statements that is called DDL trigger this trigger we can create on create, alter, drop, truncate.

We can create DDL trigger like

```
↳ CREATE OR REPLACE TRIGGER TRG_DDL BEFORE DDL ON SCHEMA
↳ CREATE OR REPLACE TRIGGER TRG_ALT BEFORE ALTER ON SCHEMA
```

But we cannot create like before create, before drop like that. If you want to write a DDL trigger only one event like drop, create then you should go for pseudo column "**ora_sysevent**".

102. What are the types of Constraints..?

1. Unique Key Constraint
2. Primary Key Constraint
3. Foreign Key Constraint

Foreign key also known as reference key and integrity constraint.

103. What is function over loading..?

This concept exists in the packages only. If you write more than one procedure/function with same name that concept is known as functions over loading but parameters should be different.

Best example is cancelation or expires if user trying to cancel the transaction then parameters required for canceling the transaction. If system canceled the transaction there is no need of parameters but the programs will be same both programs ate work for cancelation only.

104. What is one time procedure..?

At the end of the package body if you write begin and some statement it is called a onetime procedure. Here you should not write any end statement if you want to write end exception in this you can write but you cannot write separate end for that.

105. What is meant by two-phase commit..?

It is useful when you working in multiple servers with multiple users if two users from different servers are giving commit on the same table at a time. But I have worked in single server with multiple users. Means distributed servers (1-many), Dedicated server (1-1).

It handled by RAC DBA (Real Application Cluster Data Base Authority).

106. What is meant by function purity levels..?
What are the draw backs of functions..?
There are any limitations for functions..?

We can write DML operation in the functions but those functions are not aligible to call in SQL. If we want to call those functions in SQL you should go for "`pragma autonomous_transaction +TCL`".

If you write out parameter again you cannot call those functions in SQL there is no medicine for this.

107. What is tnsnames.ora..?

It contains address of the database and that's useful to create database link or taking permission from other database users and also you can get database name which you are using.

108. Can we give commit for the triggers..?

Yes. We can give commit for triggers and we have to write pragma "`pragma autonomous_transaction +TCL`". But we should not give commit for triggers because triggers is auto commit when you give the commit for your transaction then automatically trigger action will be committed, if you are roll backed your transactions automatically trigger action also get roll backed.

When we write DDL trigger then also DDL is auto commit then trigger action also committed automatically.

If you write commit in trigger then it will commit transaction when it fire after that if user have given rollback for the transaction then that transaction only roll backed trigger action never roll back. That will be very big mistake in the program.

109. What is package..? What is the need of it..?

Package is nothing but collection of related programs package contain global variables. Package has concepts like function over loading, one time procedure and forward declaration. These three concepts are available in packages only.

Mostly we will go for package when we want to store all programs at one area all those programs should be related to same module. It will improve system performance when compared to using different individual programs for same module.

For example if you are going for the IRCTC we can cancel the ticket, then immediately we can book the new ticket and also we can PNR status at the same time means we are doing three action at one slot. For this if I called one packaged procedure for ticket cancelation then entire package will come and store in the RAM so next I can go for the ticket booking or PNR status procedure there is no need to go and check in database those already exist in RAM because entire package will exist in the RAM. And package having function overloading, one time procedure and forward declaration concepts.

Function over loading means if you write more than one program with same name that is called function overloading but parameters should be different. If the parameters are same but their data types are not same then also it won't work.

Forward declaration is works for private programs in a package in the package. If you write any procedure in package body without declaring in package spec those are called private procedures those we can call in entire package body but whenever you are using that procedure before that the procedure will be declared this concept is known as forward declaration.

One time procedure means in end of package body if you begin and some statement that is called one time procedure. If you call any packaged procedure or function 1st one time procedure will be execute then only your will execute.

110. How will you connect to the servers from your PC..?

- Go to run
- Enter IP address of server
  Ex: 10.1.46.224
- Enter your user id, password.

111. Define transaction..?

Transaction nothing but one unit of work means one insert is one transaction or one update is one transaction or one delete is one transaction.

112. Can we give commit for procedure and function..?

Yes. We can give commit for procedure and function. When we are giving commit in procedure no issue but if we are giving commit function there is a issue those we cannot call in the SQL. If we want to call those functions SQL then we should write "`pragma autonomous_transaction`".

113. Difference between dedicated server and distributed server..?

- 1 server – 1 user is dedicated server.
  Mostly board of directors can use this dedicated servers for security purpose those only able to see the information in that servers.
  Ex: Bank CEO, Snap deal CEO.

- 1 server – Multiple users
  Ex: Eenad.net, icici.com, redbus.com.

114. What is autonomous transaction..?

Autonomous transaction define any independent transaction from the parent transaction allows commit, rollback without any effecting of parent transaction.

In any procedure we write "`PRAGMA AUTONOMOUS_TRANSACTION`" between as begin and we write commit or rollback is works for that procedure only there is no chance to commit or rollback previous procedures/functions actions.

115. What is partition..?

Partition means larger table data will divide into a smaller more manageable piece when table having more than 2GB data. To know the table size we have a predefined table.

We can create index on partition and also cluster.

116. How to find which database you have connected..?

To database:

- ↳ SQL command window
- ↳ Select * from global_name

(OR) tnsnames.ora

To user

- ↳ SQL command window
- ↳ Show user

117. How to connect different databases..? What is database link..?

By using database link, DB link is the only option to connect different databases. To know the address of our database then we should go for "tnsnames.ora". It is one way communication link between one database to another database.

118. To whom do you report..?

In my company my team lead assigned me the work. Whenever I have completed the task I have to inform the reporting person my reporting person is my team lead Hussain.

119. Is index useful in all cases..?

No sir.

- ↳ If null values is more than index not useful means it works but output not proper.
- ↳ If index column not used in where condition then it's won't use.
- ↳ If we are doing more DML operations on the table then index will be inactive.

120. Have you ever used dynamic SQL..?

Yes., we used many times dynamic SQL concept because sometimes ew adding the columns and sometimes we dropping the columns. If we want to add column in procedure you should go for dynamic SQL because PL/SQL doesn't allow DDL statements directly.

Some times DML also require dynamic SQL because sometimes content they will pass dynamically means if we don't know the table name or column names then can come in parameters then we want to do DML operations by using those parameters then we should go for dynamic SQL for DML operations also.

Ex: `EXECUTE IMMEDIATE DELETE FROM ||P_TABLE;`

121. Can a view be indexed..?

No. only materialized view will be able to create index. Not possible to create index on normal view because it won't store the data it stores select statement only.

122. How to fetch duplicate rows..?

```
SELECT * FROM EMP WHERE EMPNO IN (SELECT EMPNO FROM EMP GROUP BY
EMPNO HAVING COUNT (*)>1);
```

123. Display the employees who are not managers..?

```
SELECT * FROM EMP WHERE EMPNO NOT IN (SELECT NVL (MGR,0) FROM EMP);
```

124. What is bitmap index..? Why it is efficient..?

If cardinality is >20 then better go for bitmap index. Means when column having repeating data then we should go for bitmap index. It will store the data in 1,0 format for example which employees are working in 10^{th} department it will put 1,1,1.. for those employees rowid's.

125. Where we use :OLD and :NEW..?

:OLD and :NEW mostly we use in row level trigger only and these two are mostly we use for maintain the audit.

126. What is cursor..? Using cursor how you increase the salary of employees by 15% whose salary is greater than 20,000.

Cursor is a private area created by PL/SQL block when we are fetching the data by using cursor select statement that data will be stored in some area that area is called cursor.

```
DECLARE
      CURSOR C1 IS SELECT * FROM EMP WHERE SAL>20000;
SAL_2 EMP.SAL%TYPE;
BEGIN
      OPEN C1
            LOOP
      FETCH C1 IN SAL_2;
      UPDATE EMP SET SAL=SAL+(SAL_2*20/100);
      EXIT WHEN C1% NOT FOUND;
            END LOOP;
      CLOSE C1;
END;
```

127. Difference between package spec and body..?

- Spec contain global variable that we can use in entire schema.
  Body also contain global variable but those variable are global for that package body only.
- If we write any program in package spec that we can use in inside package body and outside package body.
  If we write any program in package body without maintaining in package spec that we can use in that package body only.
- If we drop the spec both spec and body will be dropped.
  We can able to drop the package body only.
- Without spec we cannot create body.
  Without body we can create spec. may that spec can be useful for global variable.

128. Can we write nested blocks in PL/SQL..?

Yes. We can write nested blocks in PL/SQL blocks up to 254. Mostly its useful for exceptions.

We can able to write query inside the query up to 24 queries (sub query).

129. Why we can create index on a view..?

View won't store the data physically because view is logical object. But index is a physical object when data is not there where the index will be created! We can create index for materialized view.

130. How do we process records in cursor..?

Cursor fetching the data from SQL engine to PL/SQL engine record by record that records will be stored in one area in the RAM that is called cursor area.

Cursor is a slow processor record by record only fetching. If table having 100 records 100 times it fetching data from SQL engine to PL/SQL engine. Means number of context switches will be required same ref cursors also using number of context switches to avoid that only we use bulk collect.

131. Types of cursors..?

Cursors are three types implicit cursor, explicit and ref cursor. When I'm writing a program in PL/SQL blocks then I want to know the how many rows deleted means I want to know the status of the DML operation in PL/SQL block then we should go for implicit cursor.

I want to fetch the data in PL/SQL block by using into clause into, into clause will support to fetch only one record but I want to fetch more than one record I can go for explicit cursor.

By using one cursor I want to fetch different result sets then I should go for ref cursor because explicit cursor always associated with same result set. By using ref cursor if we don't know the table, column names then also we can fetch the data. Means cursor dynamically we cannot use but ref cursor we can use dynamically.

132. How do you identify autonomous transaction in procedure..?

If any one program DML operation only being committed and that commit should not impacting on any other procedure that program defiantly having autonomous transaction in their self.

133. How we call a procedure..?

- Open SQL command window.
- > EXEC SP_CAL;

If procedure having out parameter (or) we are calling a function 1st we have to declare a variable.

Function:

- Open SQL command window.
- > EXEC :X:=SF_OTP;

Procedure:

- > Variable X number;
- > EXEC SP_CAL (10,20, :X);

134. What is dbms_job..?

If we want to create schedule jobs means our program should be run automatically in a particular time daily which we are given then that programs we call in the dbms_job.

Dbms_job is a predefined package this package contain different programs like submit, run, broken, interval, remove, start, stop.

135. What is dynamic and static value..?

When I'm passing one table name I want that table data but by using cursor we cannot write a program for this because when we want cursor statement we should know the table name. But without knowing table name also we can write the ref cursor select statement we have predefined data type for ref cursor that is sys ref cursor. We can use this ref cursor as in, out parameters and also return type.

```
CREATE OR REPLACE FUNCTION SF_SYS_REF (P_TABLE VARCHAR2)
RETURN SYS_REFCURSOR
AS
 LV_QUERY VARCHAR2(2000);
 X SYS_REFCURSOR;
BEGIN
  LV_QUERY := 'SELECT * FROM '||P_TABLE;
  OPEN X FOR LV_QUERY;
  RETURN X;
END;
```

136. What is partition..? Why you go for partitions and write the syntax..?

Partition will divide larger table data into smaller more manageable pieces. Partition allows tables, indexes and index organized table to be sub divide into smaller pieces.

If table having huge amount of data or table having more than 2GB then even if you are going for the index also may be system will give slow performance because table having huge amount of data it's very difficult to finding the record by using index then if you going for the partition then it will divided into smaller more manageable pieces then one partition has one type of data another partition having another type of data tomorrow we are updating, selecting, deleting the data directly it will go for that partition area where that record is exists.

137. Difference between unique index and primary key/ unique key constrains unique index..?

If we inserting duplicate values on that column:

- Unique key/Primary key constraints show the error "**Unique key constraint violated**".

- Unique index show the error "d**up_val_index**" (it is a predefined exception and this error occur only in unique index).

138. What is oracle instance..?

↳ Background processor.
↳ Temporary memory.
↳ Permanent memory.

139. Have you done performance tuning..?

Yes. I know about performance tuning sometimes may be front end users can be complaint this program is taking more time. Then immediately I will go for the '**dbms_profiler**' and I will tune that program. May be that procedure I was not written somebody was written then also I have to tuned that program.

140. What is database link..?

If one user wants to use another user object and both the users from different databases for that we have only one option that is DB link.

```
CREATE OR REPLACE DATABASE LINK DLN_AR CONNECT TO ar_murthy
IDENTIFIED BY acr143 USING 'ACR =(DESCRIPTION =(ADDRESS = (PROTOCOL =
TCP)(HOST = localhost)(PORT = 1521))(CONNECT_DATA =(SERVER =
DEDICATED)(SERVICE_NAME = acr)))';
```

141. Which two are removed by escape option..?

%, _. These two are Meta characters for like.

If we want bring the names which have '%' in their self.

```
SELECT * FROM EMP WHERE ENAME LIKE '%/%%' ESCAPE '/';
```

If we want bring the names which have '_' in their self.

```
SELECT * FROM EMP WHERE ENAME LIKE '%/_%' ESCAPE '/';
```

142. What is a bind variable..?

At the time of compilation you don't know the values you can bind variables same query no need to write multiple times.

```
DECLARE
  LV_STR VARCHAR2 (1000);
  Y NUMBER: =100;
  Z NUMBER;
BEGIN
  LV_STR: 'UPDATE EMP SET COMM=: X WHERE DEPTNO=: Y';
  SELECT MIN (COMM) INTO X FROM EMP;

  EXECUTE IMMEDIATE LV_STR USING Y, 10;
  EXECUTE IMMEDIATE LV_STR USING Z, 20;
  EXECUTE IMMEDIATE LV_STR USING 300, 30;
END;
```

143. What are the parameters for job..?

(JOB, WHAT, WHEN, INTERVAL).

144. 'SELECT COUNT (*) FROM EMP WHERE ROWNUM>2' what is the output..?

ROWNUM won't support '>' operator. But is supports to '<, <=, =, >='.

145. If there are two after insert triggers on a table for each row which will be fired first..?

Recently written trigger will fire first.

146. 'SELECT SEQ_1.CURRVAL FROM DUAL' what will be the value when the session opens..?

It gives error because without using nextval we cannot use currval.

Error: (ORA-08002 sequence seq_1.currval is not yet defined in this session).

147. 'SELECT TO_CHAR (SYSDATE,'MM/DD/YY HH24:MI') FROM DUAL' if it is 12:00 in the on JAN 1, 2000 what is the result..?

Result: **01/01/00 00:00:00**

148. 'SELECT DISTINCT ENAME, SEQ_1.NEXTVAL FROM EMP' what is the output..?

Error: sequence number not allowed here.

Because sequence won't support distinct but if you write 'SELECT ENAME, SEQ_1.NEXTVAL FROM EMP' then gives output.

149. If there are three users. If U1 gives permission on how will you give that to U1 and how U2 can grant permission to U3..?

If U1 gives all permissions to U2 like: `GRANT ALL ON EMP TO U2;`

Then U2 can give any permission on that object to another user like: `GRANT SELECT ON EMP TO U2;`

If U1 gives limited permissions to U2 like: `GRANT SELECT, INSERT ON EMP TO U2;`

Then U2 won't able to give any permission on that object to any other user.

150. In which cases you will get no_data_found..?

- When you fetching some data into a variable and that data is not existing then no_data_found.
- In PL/SQL table if you asking one record and that record not existing in that PL/SQL table collection variable then it gives no_data_found.
- From any collection variable you asking deleted record then it gives no_data_found.
- '**utl_file.fgetattr**' will give no_data_found when that data not exist in the directory.

151. until run time you don't know the table name to select a query which method you should fallow..?

When we are fetching the data from the table and which table we are fetching we don't know then we should go for ref cursor.

Else if table name they are passing dynamically then we should go for dynamic SQL (or) ref cursor.

Ex:

```
CREATE OR REPLACE PROCEDURE SP_1 (P_TABLE VARCHAR2,P_COL VARCHAR2)
AS
  X NUMBER;
  LV_QUERY VARCHAR2 (1000);
BEGIN
    LV_QUERY:= 'SELECT '||P_COL||' FROM '||P_TABLE||
                  ' WHERE ROWNUM=1';
     EXECUTE IMMEDIATE LV_QUERY INTO X;
END;
```

152. How many triggers can we create on a table..?

12 types of triggers we can create on a table.

Before insert/update/delete for statement,
After insert/update/delete for statement,
Before insert/update/delete for each row,
After insert/update/delete for each row,

153. What is bulk exception..?

When we are going for the bulk bind concept we have to do multiple operations at one shot then there is a chance to get bulk of errors at a time. So if we want to catch those errors then we should go for bulk exception.

154. What are the errors in collections, cursors, triggers..?

Collections:

- Subscript beyond count.
- Subscript out of limit (varray).
- No_data_found.
- Reference to uninitialized collections.

Cursors:

- Invalid cursor.
- Cursor already open.

Triggers:

- Mutating error.
- Recursive error.

155. What is cursor for update..?

If we write for update along with cursor select statement exclusively row level lock will be applied for all the rows which are identified by that cursor select statement. Until we give commit no one cannot be modify those rows.

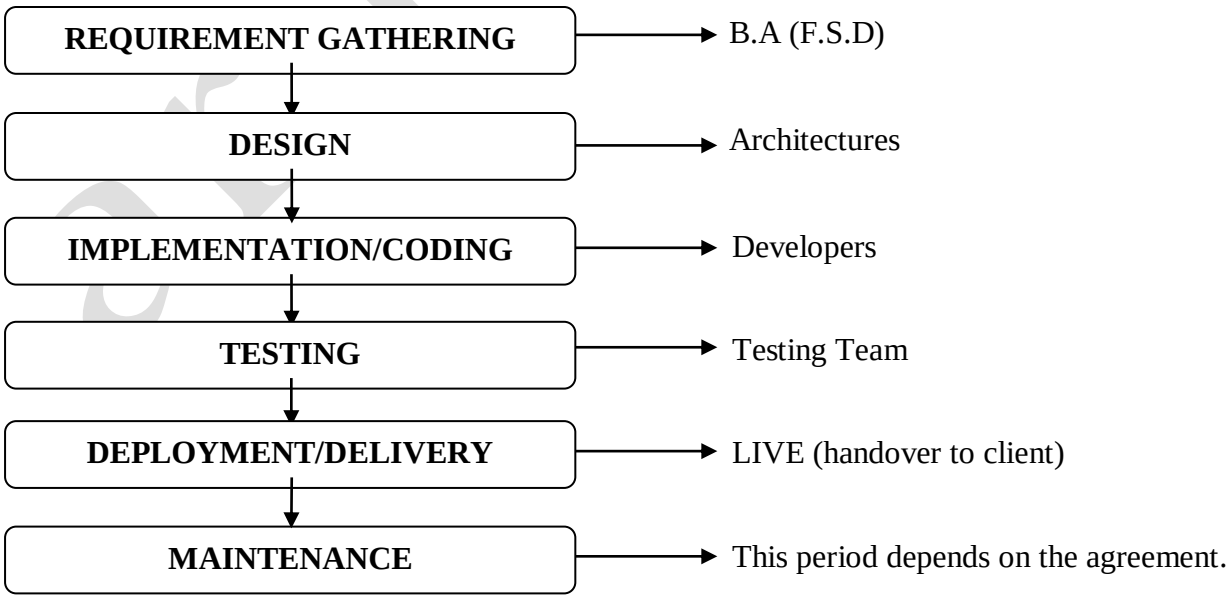
156. What is 'where current of'..?

Generally we cannot perform DML operations when we using cursors for update in our program if we want to perform DML operations on which are locked by the cursor for update the we should go for the "where current of".

158. Tell me the system development cycle..?

There are three types of development cycles are there water fall model, zebra model and Azalea model, In my previous company they are following Azalea model.

AZALEA MODEL:



159. Errors in triggers without Pragma+TCL:

BEFORE EVENT				AFTER EVENT		
STATEMENT	**ACTION**	**ERROR**		**STATEMENT**	**ACTION**	**ERROR**
INSERT	INSERT	RECURSIVE		INSERT	INSERT	MUTATING
	UPDATE	NO ISSUE			UPDATE	MUTATING
	DELETE	NO ISSUE			DELETE	MUTATING
	SELECT	NO ISSUE			SELECT	MUTATING
UPDATE	INSERT	MUTATING		UPDATE	INSERT	MUTATING
	UPDATE	MUTATING			UPDATE	MUTATING
	DELETE	MUTATING			DELETE	MUTATING
	SELECT	MUTATING			SELECT	MUTATING
DELETE	INSERT	MUTATING		DELETE	INSERT	MUTATING
	UPDATE	MUTATING			UPDATE	MUTATING
	DELETE	MUTATING			DELETE	MUTATING
	SELECT	MUTATING			SELECT	MUTATING

160. Errors in triggers with Pragma+TCL:

PRAGMA + TCL						
BEFORE EVENT				AFTER EVENT		
STATEMENT	**ACTION**	**ERROR**		**STATEMENT**	**ACTION**	**ERROR**
INSERT	INSERT	RECURSIVE		INSERT	INSERT	RECURSIVE
	UPDATE	NO ISSUE			UPDATE	NO ISSUE
	DELETE	NO ISSUE			DELETE	NO ISSUE
	SELECT	NO ISSUE			SELECT	NO ISSUE
UPDATE	INSERT	NO ISSUE		UPDATE	INSERT	NO ISSUE
	UPDATE	DEAD LOCK			UPDATE	DEAD LOCK
	DELETE SAME ROWS	NO ISSUE			DELETE SAME ROWS	DEAD LOCK
	DELETE OTHER ROWS	NO ISSUE			DELETE OTHER ROWS	NO ISSUE
	SELECT	NO ISSUE			SELECT	NO ISSUE
DELETE	INSERT	NO ISSUE		DELETE	INSERT	NO ISSUE
	UPDATE	NO ISSUE			UPDATE SAME ROWS	DEAD LOCK
	UPDATE OTHER ROWS	NO ISSUE			UPDATE OTHER ROWS	NO ISSUE
					DELETE OTHER ROWS	NO ISSUE
	DELETE	DEAD LOCK			DELETE	DEAD LOCK
	SELECT	NO ISSUE			SELECT	NO ISSUE